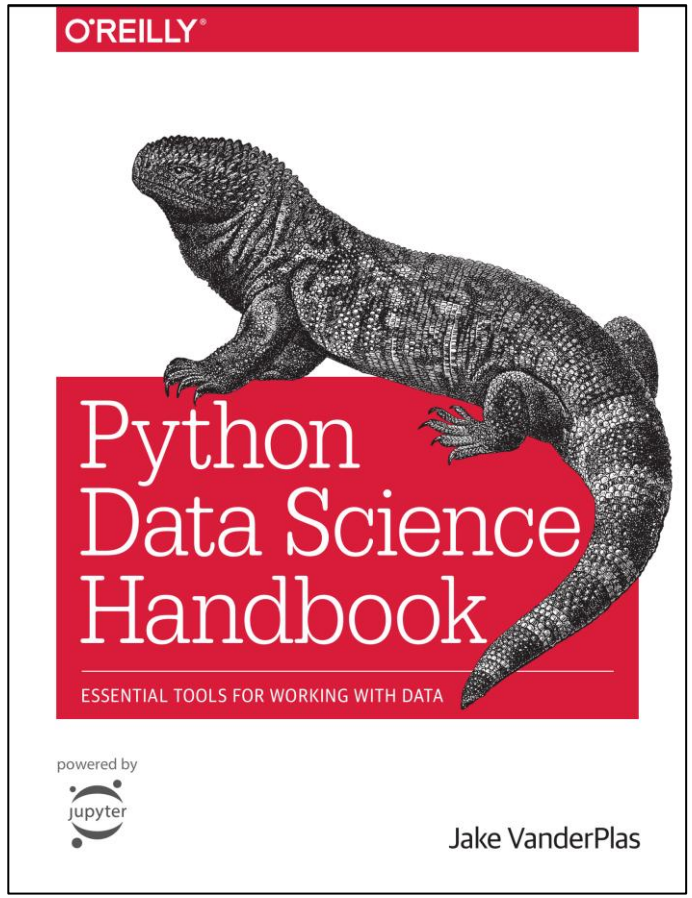
澳門理工大學
Universidade Politécnica de Macau
Macao Polytechnic University

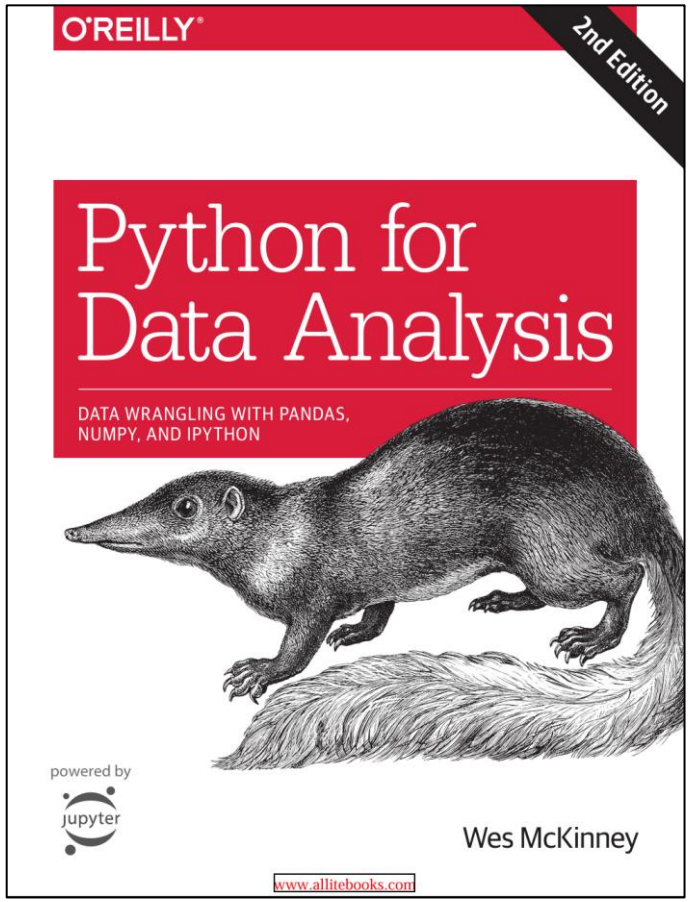
Introduction to Data Science

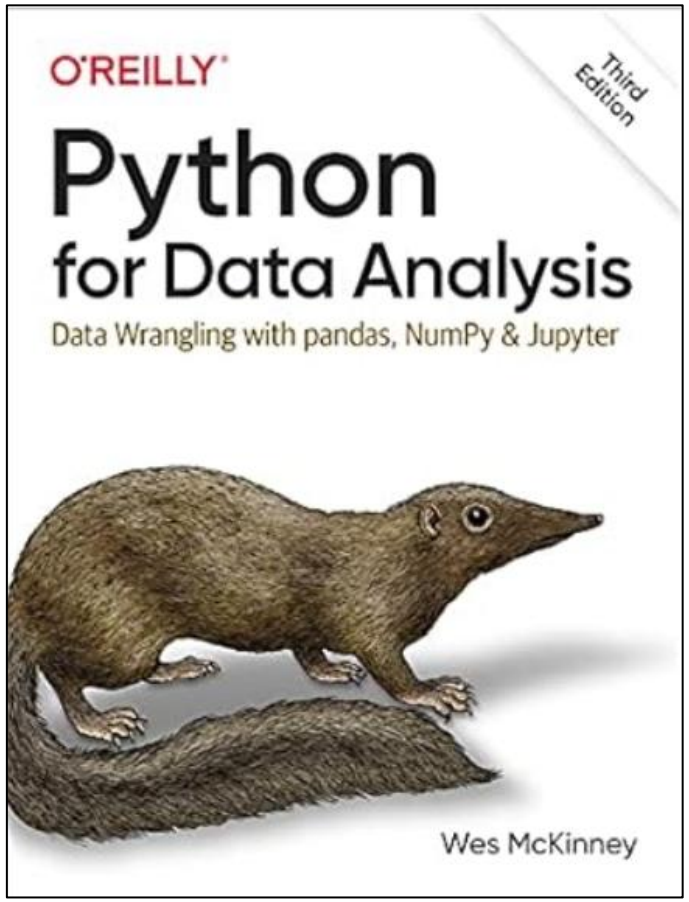
Jie Zhang
Faculty of Applied Sciences, Macao Polytechnic University
Email: jpeter.zhang@mpu.edu.mo
Personal Profile: easy-shu.github.io



Textbooks


O'REILLY
Python Data Science Handbook
ESSENTIAL TOOLS FOR WORKING WITH DATA
powered by
jupyter
Jake VanderPlas


O'REILLY
2nd Edition
Python for Data Analysis
DATA WRANGLING WITH PANDAS, NUMPY, AND IPYTHON
powered by
jupyter
Wes McKinney
www.allitebooks.com


O'REILLY
Third Edition
Python for Data Analysis
Data Wrangling with pandas, NumPy & Jupyter
Wes McKinney

Subjects Overview (Using Python)

COMP1122	CSAI2123	CSAI2122	CSAI3121	CSAI3122	CSAI3123
Introduction to Programming	Introduction to Data Science	Introduction to Artificial Intelligence	Machine Learning and IntelligentData Analysis	Natural Language Processing	Neural Networks and Deep Learning

Python foundation:
Numpy, Pandas,
Matplotlib, Plotnine

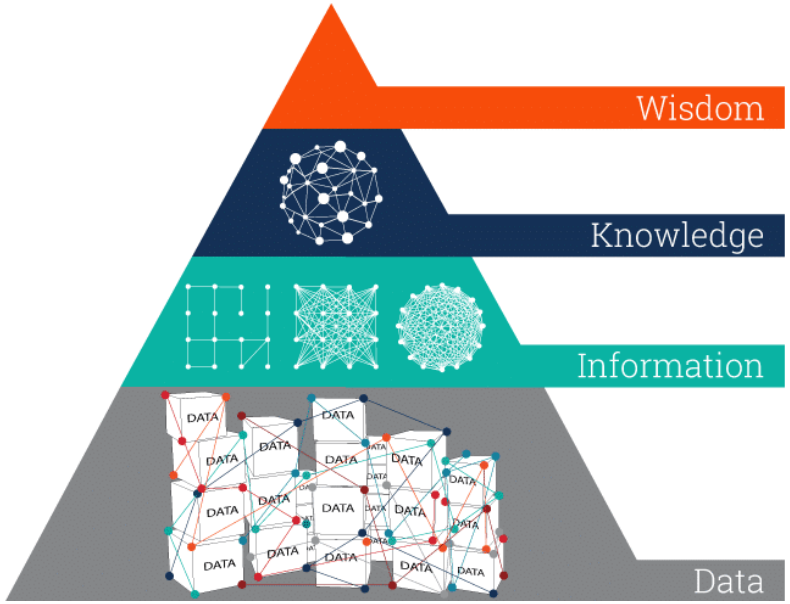
Advanced Python:
Scikit-learn (machine learning),
Pytorch (deep learning)

CSA14299	COMP4116	CSAI0112	CSAI0114	CSAI0117-118	CSAI3124
Final Year Project	Digital Image and Multimedia Processing (Elective Subjects)	Computer VIsion and Imaging (Elective Subjects)	Speech Recognition Technologyand Application (Elective Subjects)	Advanced Topics in A.I. (Elective Subjects)	Artificial Intelligence Application Project

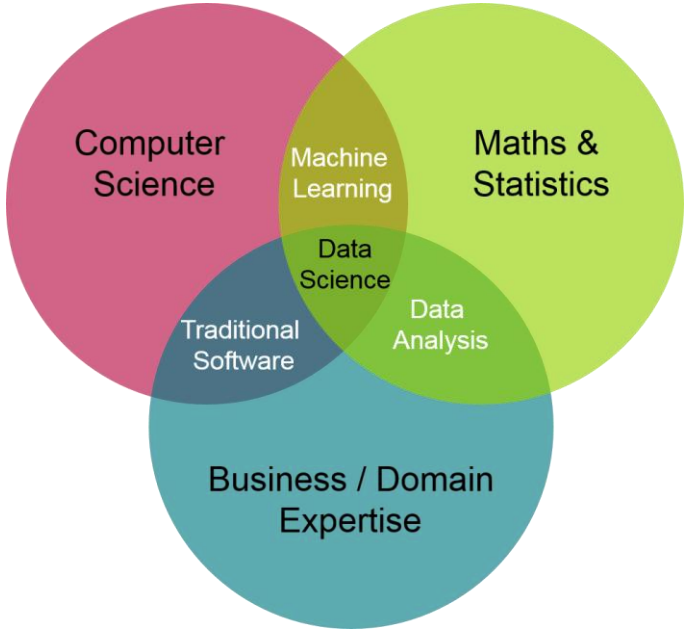
Only 4 Elective Subjects are shown and more Elective Subjects can be founded.



Basic Concepts



Data Science

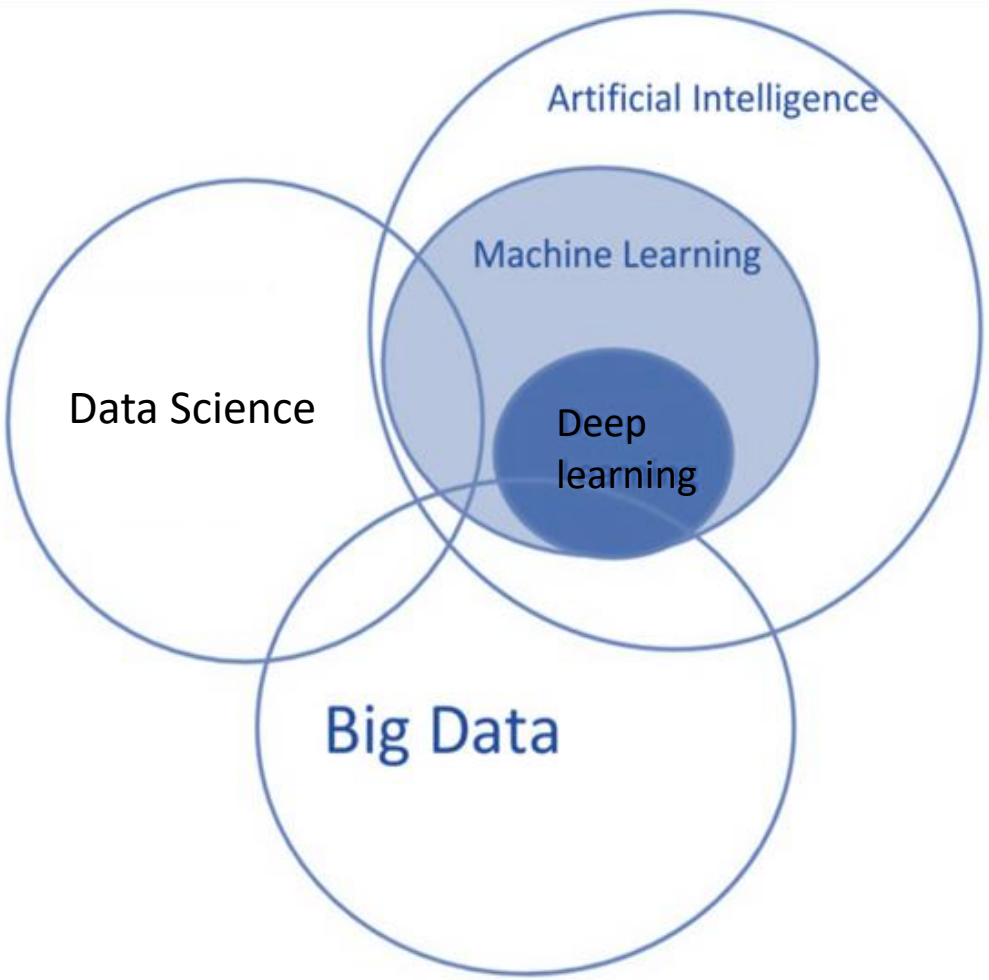


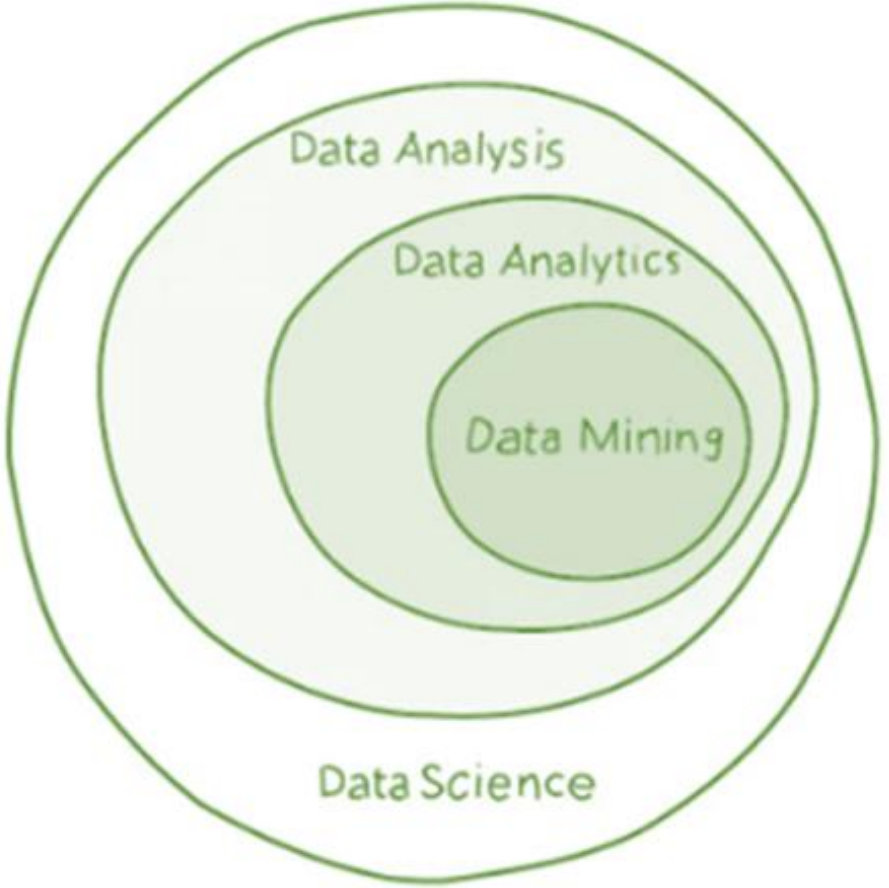
Data science is an interdisciplinary field that combines mathematics, statistics, programming, and domain expertise to extract meaningful insights from data.

key stages:

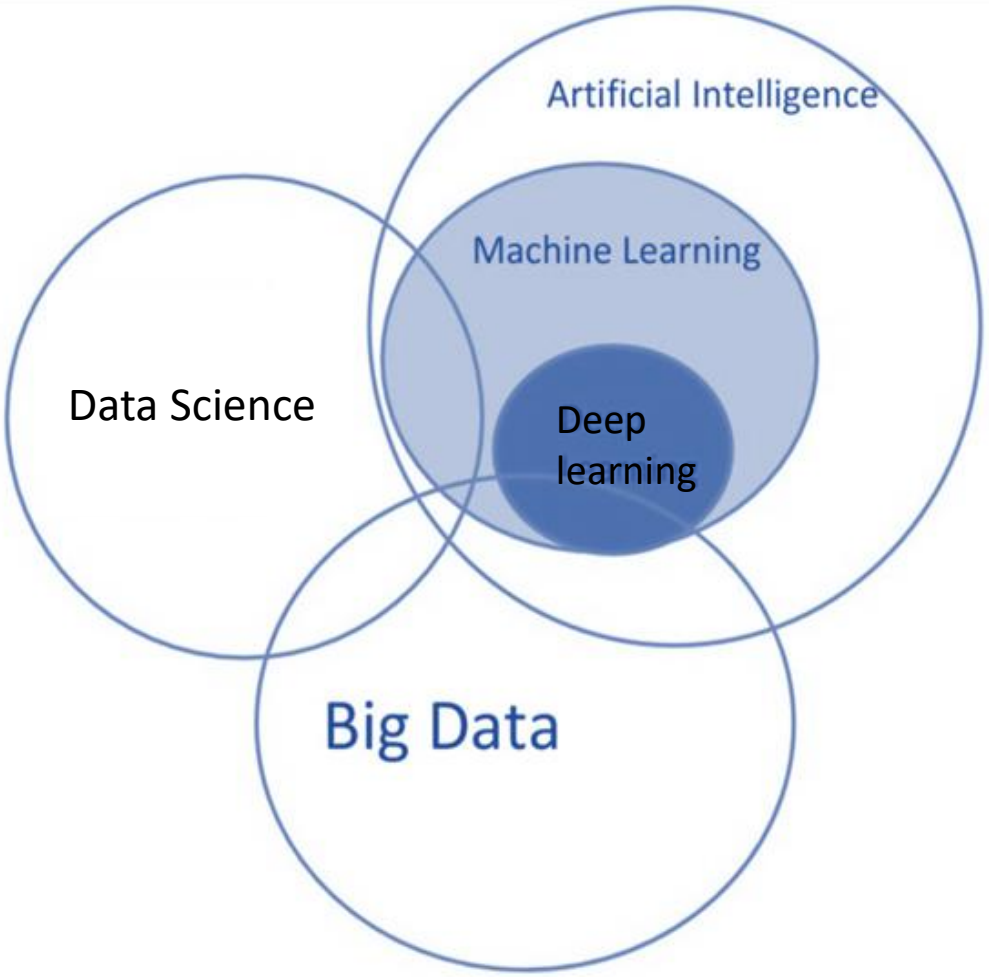
- **Data Collection**: Gathering raw data from various sources, including databases, web scraping, and real-time streaming.
- **Data Processing**: Cleaning and transforming data to ensure quality and consistency.
- **Data Analysis**: Using statistical methods and machine learning algorithms to identify patterns and trends.
- **Data Visualization**: Presenting insights through charts, graphs, and reports to make them understandable for decision-makers.

Data Science



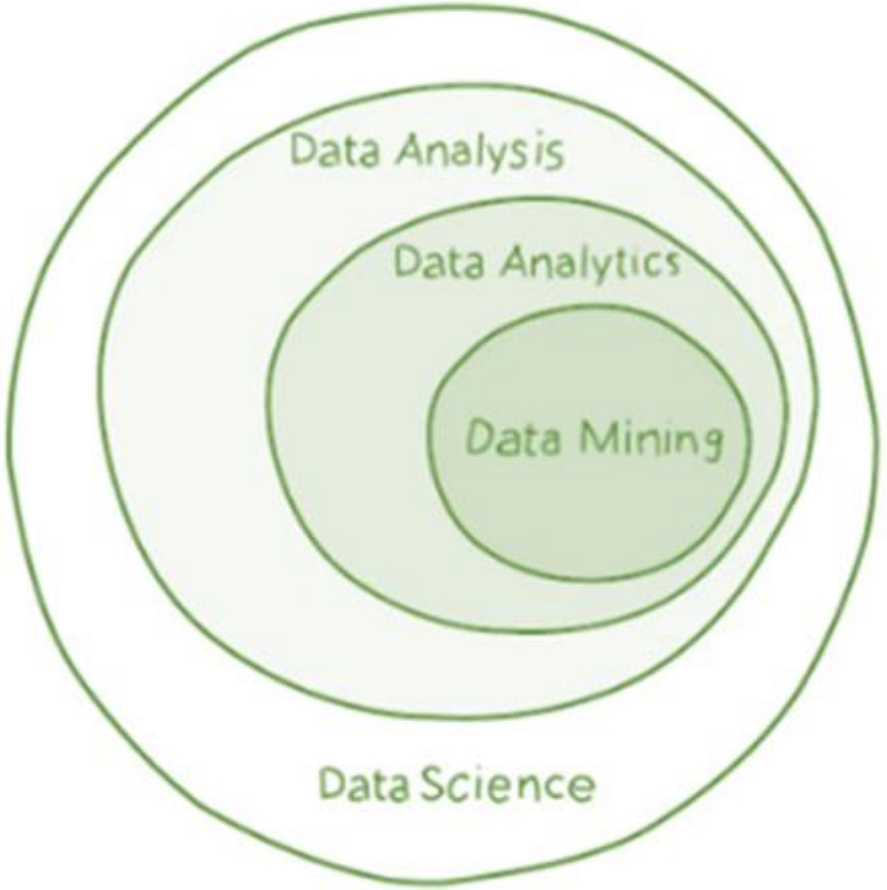


Data Science



- **Big data** refers to large data sets that are too complex or voluminous for traditional data processing **software** to handle.
- **Machine learning** is a subset of artificial intelligence that involves the development of algorithms that enable machines to learn from and make decisions based on data inputs.
- **Data science** is an interdisciplinary field that involves the use of scientific methods, processes, algorithms, and systems to extract knowledge and insights from structured and unstructured data.

Data Science



Data Science > Data Analysis > Data Analytics > Data Mining

- **Data mining** is the process of discovering patterns, correlations, and trends in large datasets and involves the use of algorithms and statistical models.
- **Data analytics** is the process of examining data sets to draw conclusions about the information they contain, with the aid of specialized software and systems.
- **Data analysis**, is the process of inspecting, cleaning, transforming, and modeling data to discover useful information, conclusions, and support decision-making.
- **Data science** is an interdisciplinary field that involves the use of scientific methods, processes, algorithms, and systems to extract knowledge and insights from structured and unstructured data.

Data Science

- **Data Mining:**

Data mining is the process of uncovering patterns, trends, and insights from large datasets using statistical and machine learning techniques. It involves exploring and analyzing data to discover hidden patterns and relationships that can be used to make informed decisions, predict future outcomes, and optimize business processes.

- **Data Warehousing:**

Data warehousing is the process of collecting, storing, and managing large volumes of data from various sources to facilitate business decision-making. It involves the extraction, transformation, and loading (ETL) of data into a centralized repository known as a data warehouse. Data warehouses are designed to support complex queries, reporting, and analysis by consolidating data from multiple systems into a single source of truth.

How they complement each other:

Data warehousing provides the infrastructure for storing and organizing data, making it accessible for analysis. Data mining utilizes the data stored in a data warehouse to extract meaningful insights and patterns that can lead to informed decision-making.



Data Science

Similarities and Differences among Data Science, Data Analysis, Data Analytics, and Data Mining

Similarities:

1. **Data Utilization:** All four fields involve working with data to extract insights, patterns, and valuable information.
2. **Statistical Techniques:** They utilize statistical methods to analyze and interpret data, uncover trends, and support decision-making processes.
3. **Tools and Technologies:** They rely on tools and technologies such as programming languages, statistical software, and data visualization tools to process and analyze data effectively.
4. **Decision Support:** Each of these disciplines aims to provide organizations with insights that can guide decision-making processes based on data-driven evidence.



Data Science

Similarities and Differences among Data Science, Data Analysis, Data Analytics, and Data Mining

Differences-Focus**:**

1. **Data Science**: Encompasses a broader set of skills and techniques to not only analyze data but also develop sophisticated models, algorithms, and data-driven solutions for complex challenges.
2. **Data Analysis**: Primarily involves examining and interpreting data to derive insights and inform decisions.
3. **Data Analytics**: Concentrates on using specialized software and systems to analyze data for decision-making.
4. **Data Mining**: Specifically focuses on discovering patterns and relationships in large datasets using algorithms and statistical models.



Data Science

Similarities and Differences among Data Science, Data Analysis, Data Analytics, and Data Mining

Differences-Scope**:**

1. **Data Science:** Includes predictive modeling, machine learning, and the development of data-driven solutions to address complex problems and predict future trends.
2. **Data Analysis:** Focuses on descriptive statistics, data cleaning, and visualization to understand historical data patterns.
3. **Data Analytics:** Emphasizes using data analysis tools to gain insights and make informed decisions
4. **Data Mining:** Concentrates on pattern discovery and knowledge extraction from large datasets.



Data Science

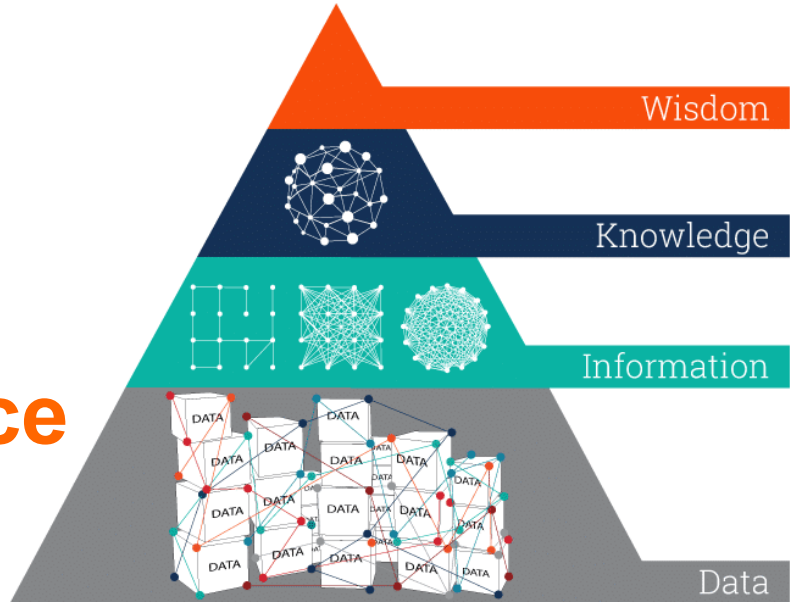
Similarities and Differences among Data Science, Data Analysis, Data Analytics, and Data Mining

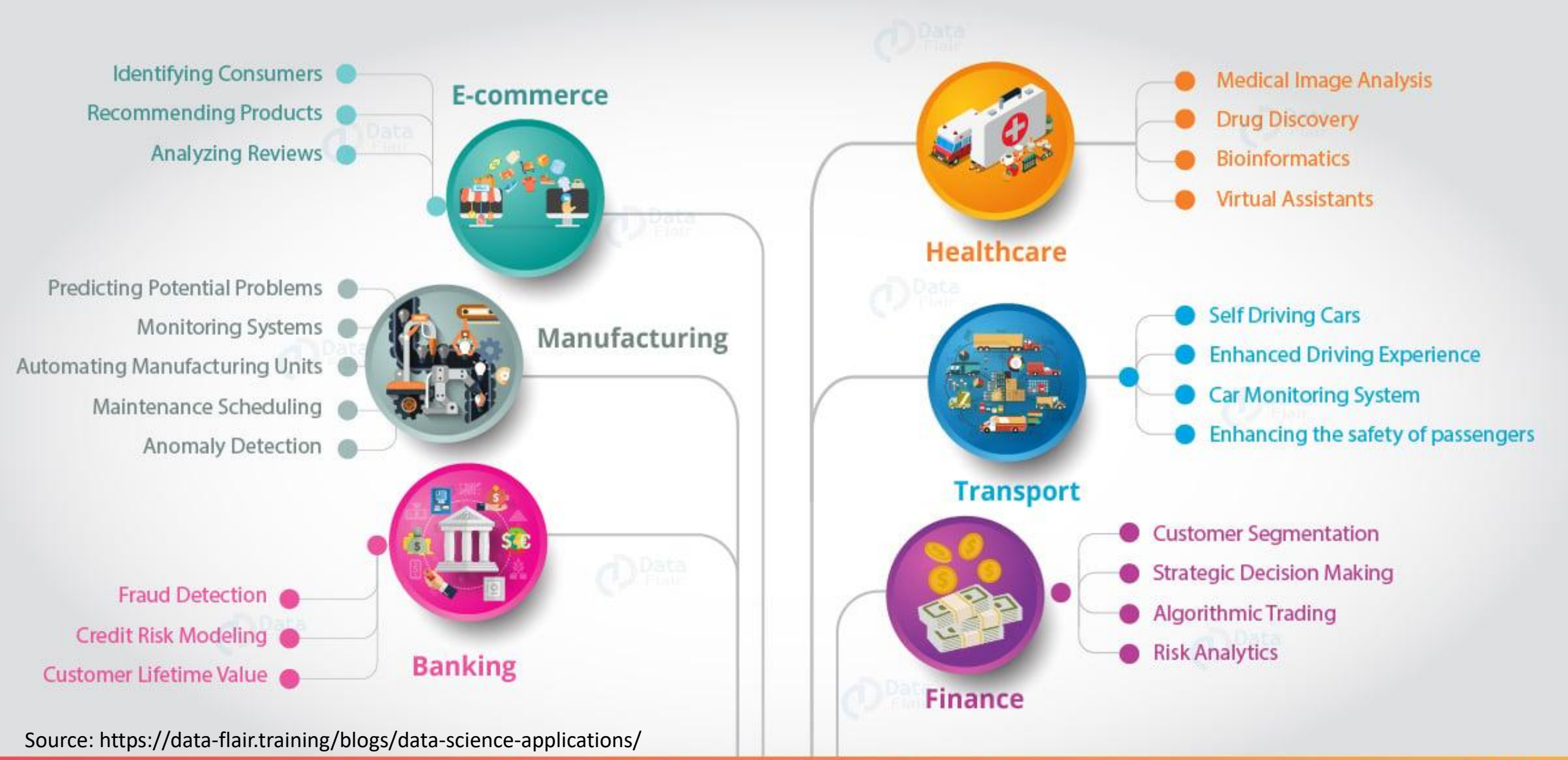
Differences-Skillset**:**

1. **Data Science:** Demands a diverse skill set, including programming, machine learning, data engineering, and domain expertise.
2. **Data Analysis:** Requires proficiency in statistical analysis, data manipulation, and visualization techniques.
3. **Data Analytics:** Focuses on using specialized software systems and tools for analyzing data efficiently.
4. **Data Mining:** Involves expertise in algorithms, statistical models, and pattern recognition techniques.



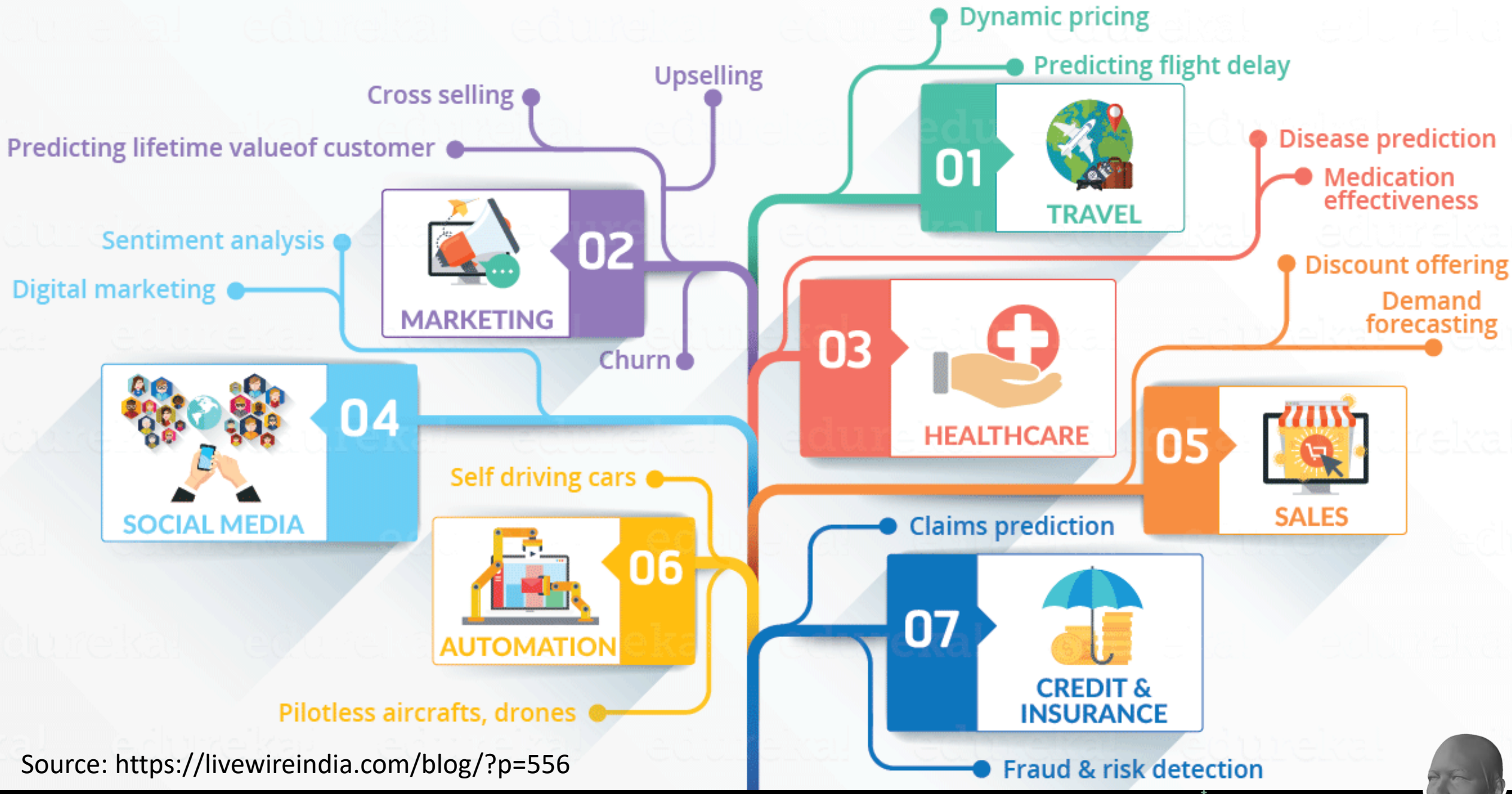
The Power of Data Science



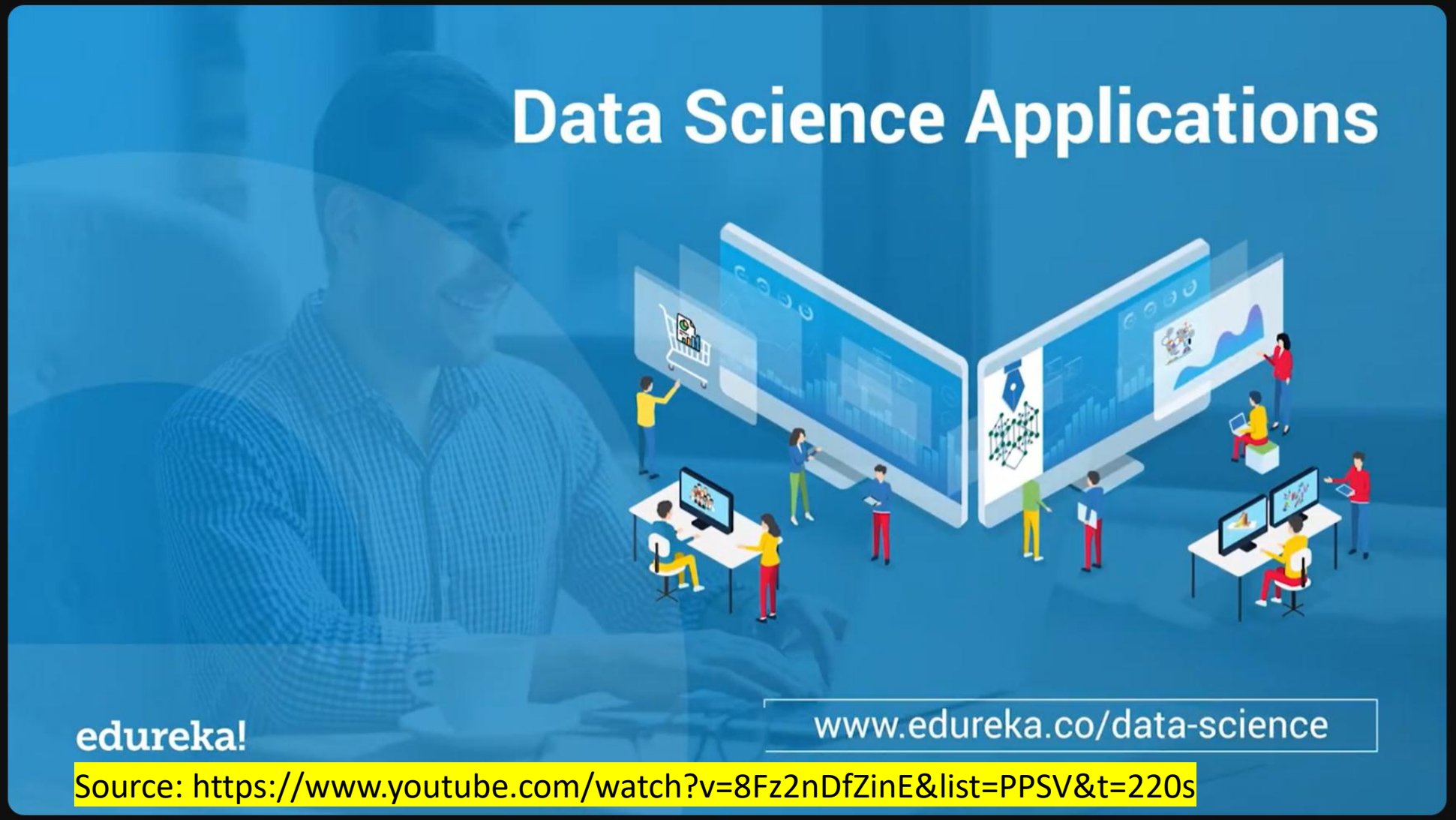


Source: https://data-flair.training/blogs/data-science-applications/

Data Science Applications



Source: https://livewireindia.com/blog/?p=556



Source: https://www.youtube.com/watch?v=8Fz2nDfZinE&list=PPSV&t=220s

Data Science Applications | Data Science For Beginners | Data Science Training | Edureka

Healthcare



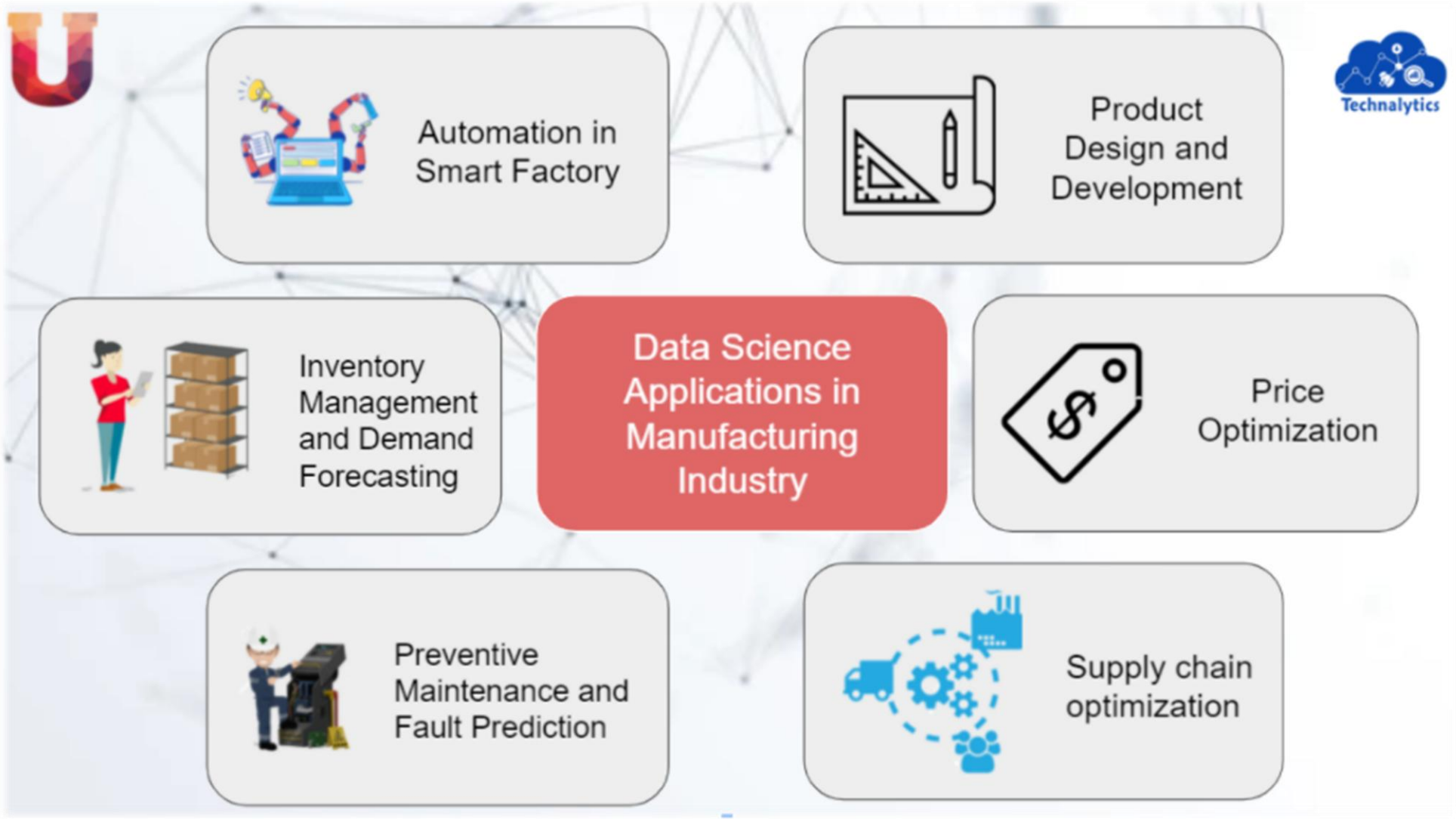
Source: https://www.bacancytechnology.com/blog/data-science-use-cases

Education

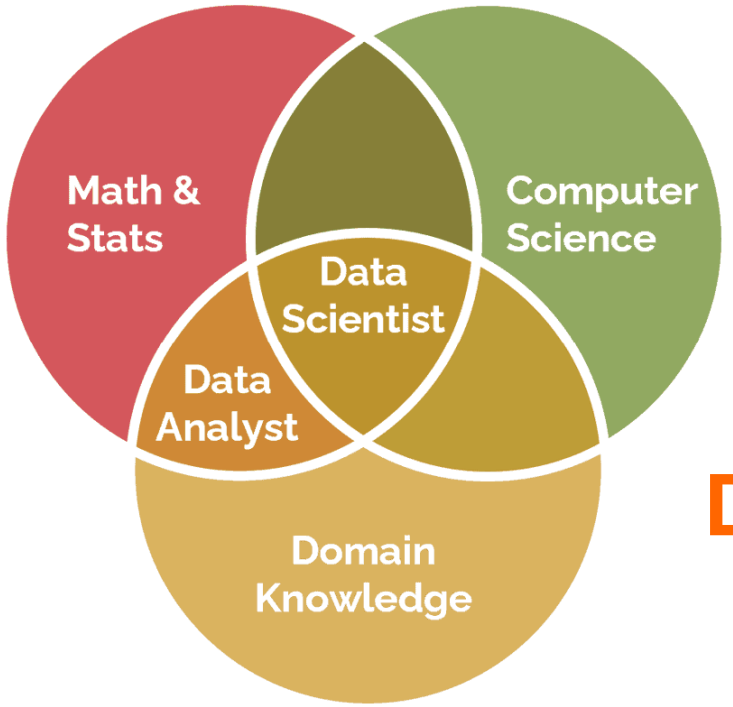


Source: https://www.linkedin.com/pulse/data-science-education-enhancing-learning-outcomes-narasimharao-raz0c/

Manufacturing



Source: https://www.linkedin.com/pulse/data-science-applications-manufacturing-industry-technalytics/


Math &
Stats
Computer
Science
Data
Scientist
Data
Analyst
Domain
Knowledge

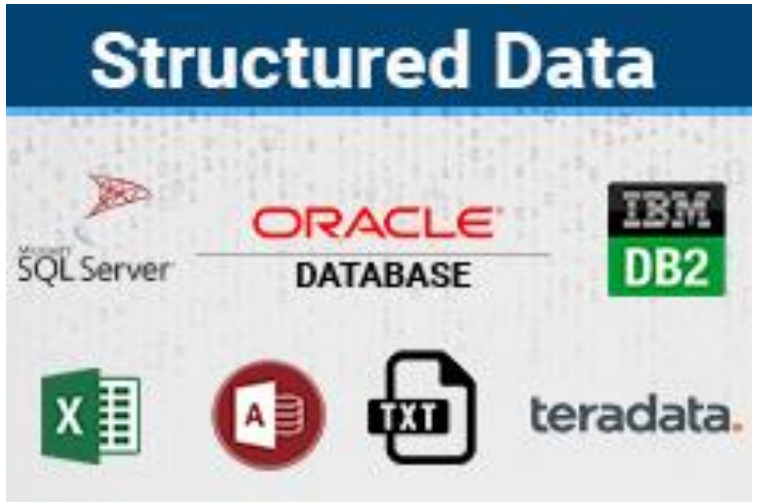
Data Types

Data Types

Structured Data: Organized data with a fixed schema (e.g., SQL tables).

Semi-structured Data: Data with some structure but no fixed schema (e.g., JSON, XML).

Unstructured Data: Data with no predefined structure (e.g., images, videos, text).







- Well defined structure
- Easier to work with

- No apparent structure and look like a stream of bytes, e.g, text file, audio, image
- Harder to work with

Decreasing ← → Increasing

Structure Data

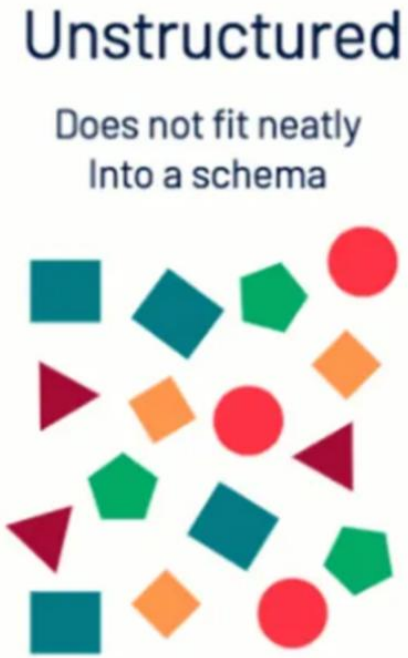
- **Structured data** are data organized in a table format
  - **Data items (records)** in a table have identical structure, and follows a strict format.
  - The **schema** is usually separated from the data.
  - When **adding** a new record, there is no need to store its structure separately.
- It's **quick to locate** an element in structured data

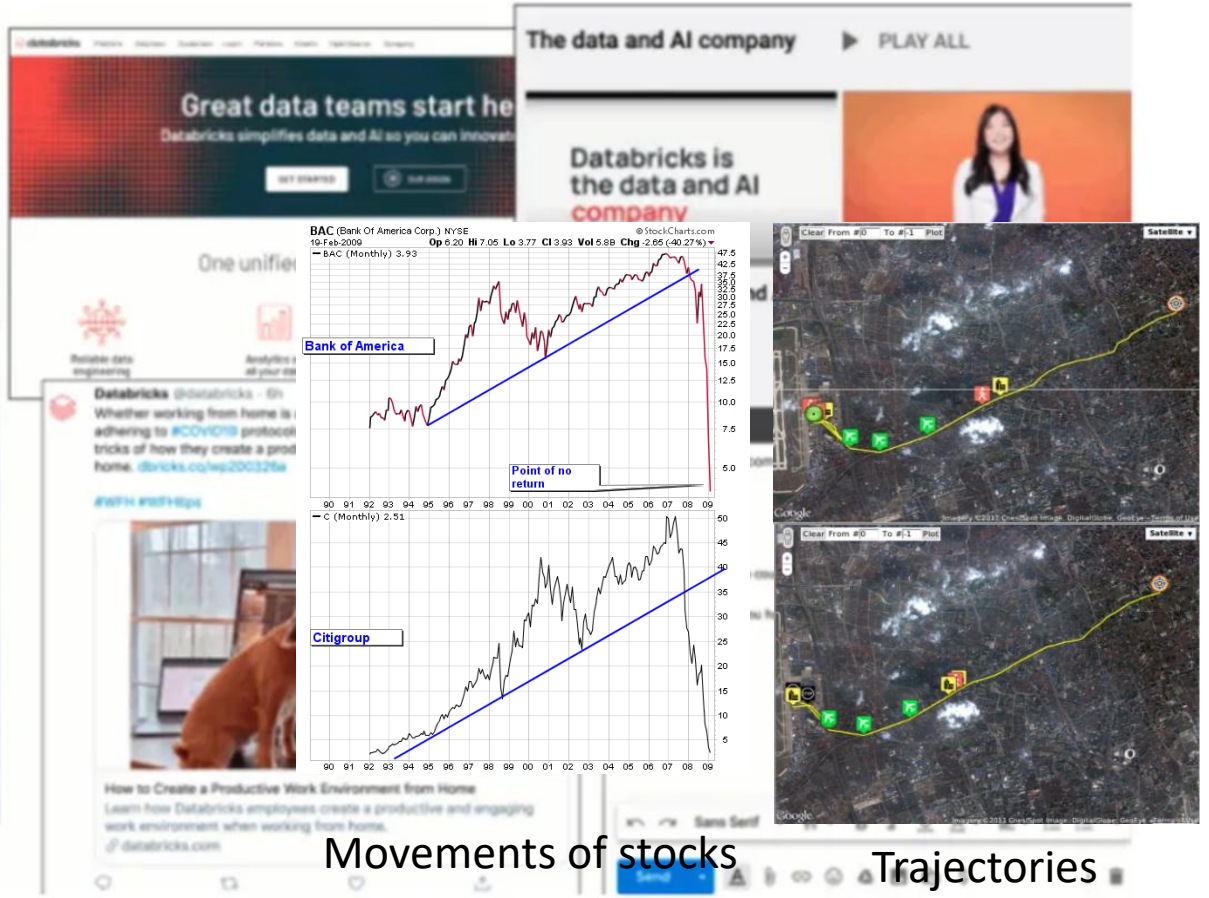


Order	CustID	Month	Item	Color	Price
101	20051	Dec	Pen	Red	2.99
102	20045	Mar	Pencil	Blue|Yellow|Red	3.99
103	29584	May	Eraser	Blue	1.25
104	29584	May	Pen	White	2.25
105	29584	May	Pencil	Blue|Yellow|Red	2.99
106	27485	Jan	Eraser	Blue|Yellow	2.75
107	29574	Jan	Marker	Green	1.75
108	24447	Feb	Marker	Yellow|Blue	7.25
109	26466	Jul	Pen	Black|Red	5.25
110	27467	Jun	Pencil	Black	2.95

Unstructured Data

- **Unstructured data** has internal structures, but it NOT structured via pre-defined data models or schema.
- Examples:
  - Social Media Posts
  - Movements of stocks
  - Trajectories
  - Images
  - Audio Files
  - Emails

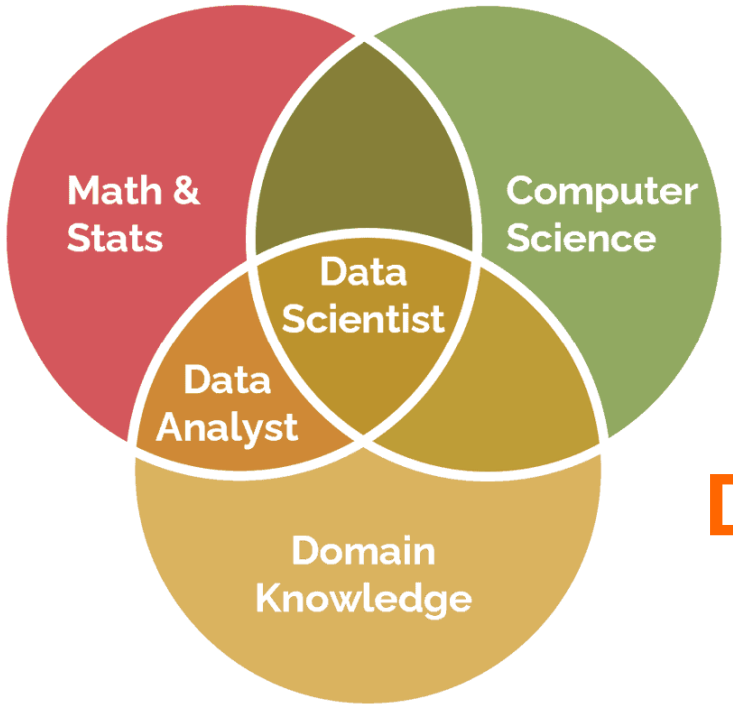




Semi-Structure Data

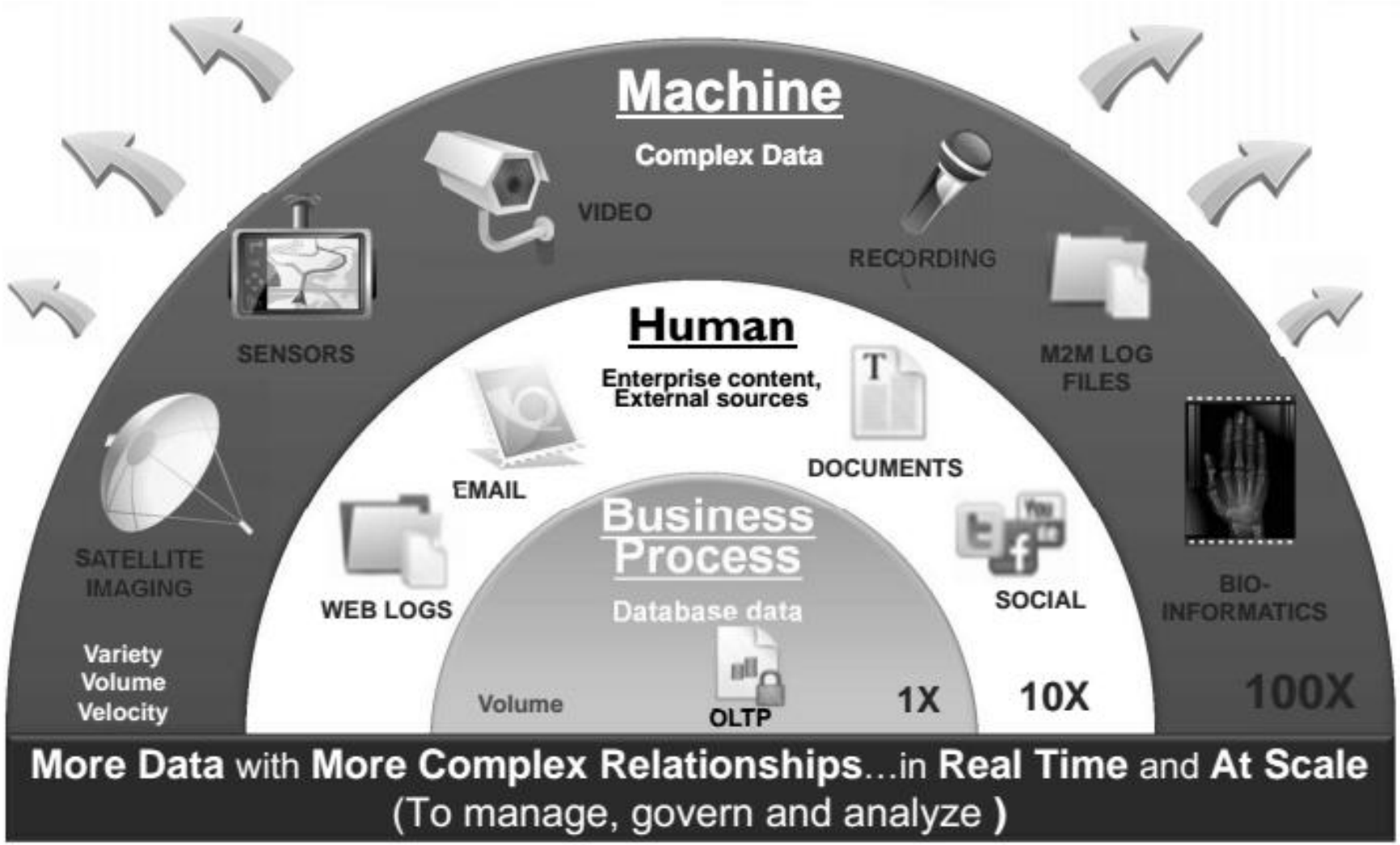
- **Semi-structured data** is structured but not obey the tabular structure of RDBMS.
- Schema info is mixed with the data values
- Sometimes known as self-describing data
- Entities in one class may have different attributes
- Examples: JSON, XML



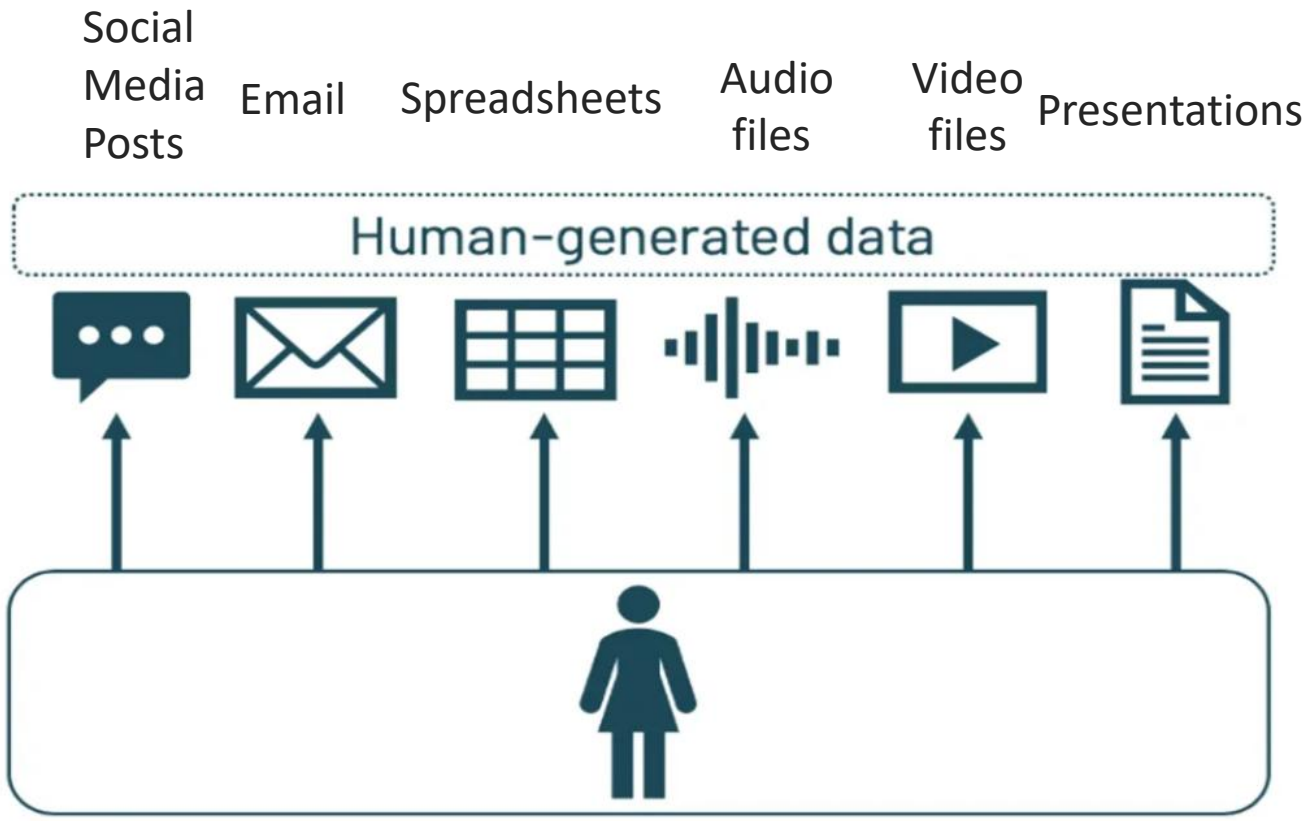


Data Sources

Where Does Data Come From?



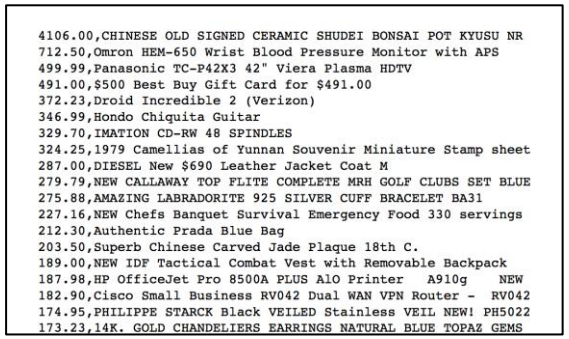
Human-Generated Data



Source: https://faun.pub/introduction-to-big-data-d9fd0389db3a

Human-Generated Data

- Challenges:
  - Large volume
  - Unstructured, relational databases cannot handle well

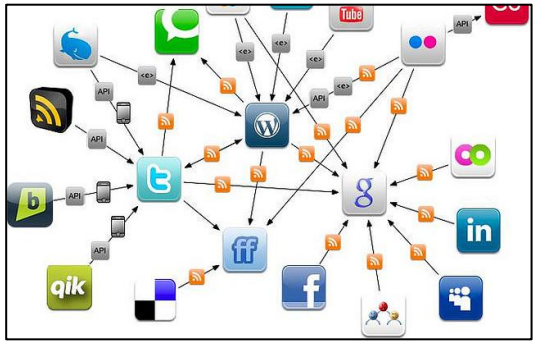


Text data

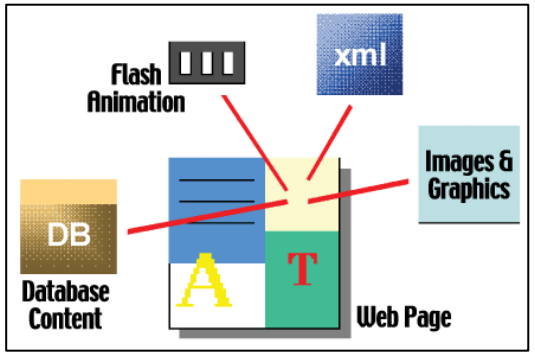


Image data

Unstructured Data



Graph data

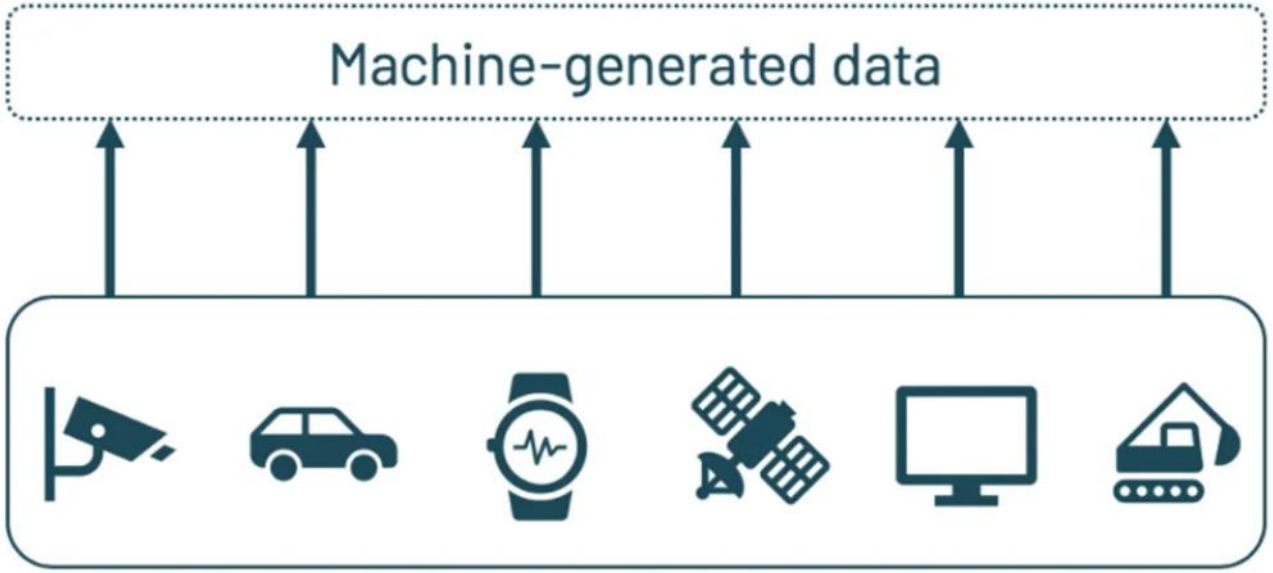


And many more…

Machine-Generated Data



Security Cameras | *Sensors on Vehicles* | *Smartphone App*, or, *Fitness Tracker* *Personal Tools* | *Satellites* | *Medical Devices* | *Industrial Machinery*



Source: https://faun.pub/introduction-to-big-data-d9fd0389db3a

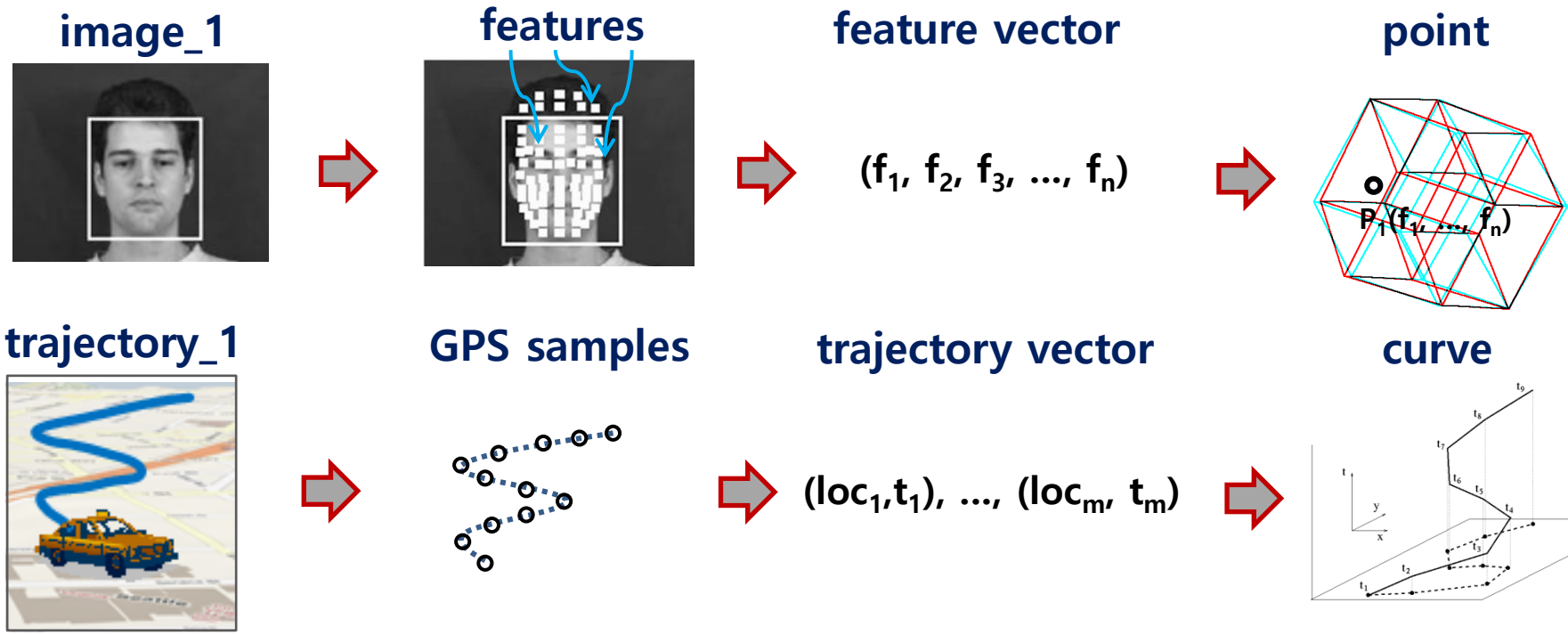
- Internet of Things: sensor data are collected from cars, machines, weather monitor stations, traffic monitors, etc.
- Fitness trackers record body temperature, heartbeat, stairs climbed and distance walked
- GPS-equipped devices and mobile phone (and Wifi connection) track customers by time and location.



Machine-Generated Data

Challenges:

- Data missing and data error
- Complex data structure, E.g, multi-dimensional data:
  An object is usually transformed into a point (shape) in a multi-dimensional space



Organization-Generated Data

- Massive data are generated by organizations when carrying out their operations and business practice.
  - E.g., commercial transactions, credit cards, banking/stock records, government open data, e-commerce, medical records
- Usually structured data, and stored in relational databases



Sale ID	Time	Customer	Product ID	Quantity
S00001	12/1/2012 9:00:00 AM	C0001	P025	1
S00002	12/1/2012 9:05:58 AM	C0025	P025	3
S00003	12/1/2012 9:11:33 AM	C0010	P001	2
S00004	12/1/2012 9:17:16 AM	C0017	P023	4
S00005	12/1/2012 9:23:04 AM	C0018	P016	5
S00006	12/1/2012 9:28:43 AM	C0011	P018	4
S0007	12/1/2012 9:34:07 AM	C0015	P006	4

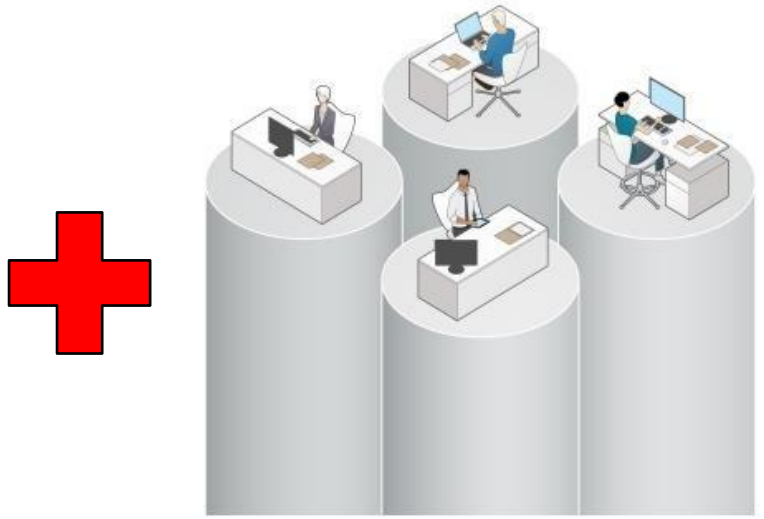
Sales Transaction Records

SQL

Organization-Generated Data

- May also generate non-structured data
  - E.g., for Web transactions, we can capture customer navigation in the Web sites, product searching, conversion (from visitors to purchases)
- **Challenges:** involves too many data sources and data types. Integration becomes very difficult and time consuming

Highly-structured organizational data



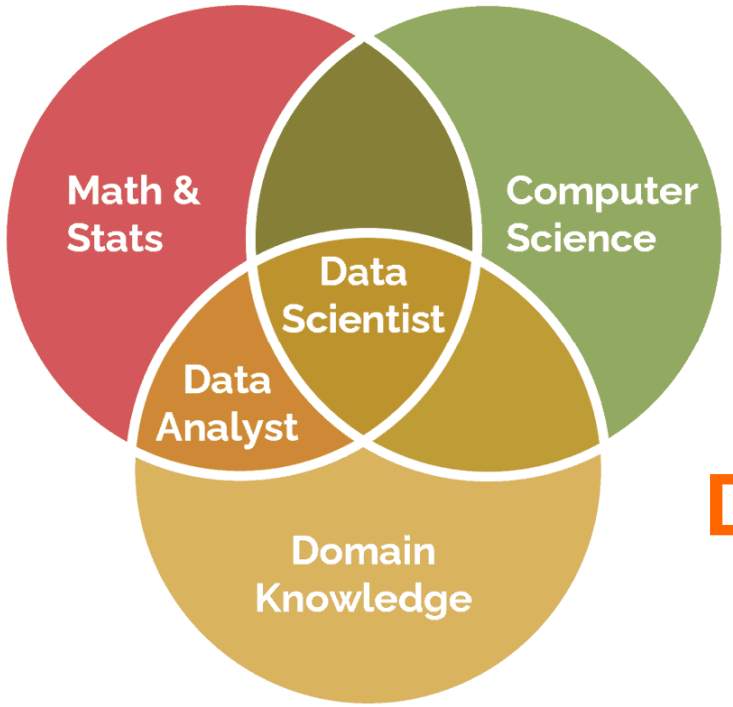
Value!

Organization-Generated Data

- May also generate non-structured data
  - E.g., for Web transactions, we can capture customer navigation in the Web sites, product searching, conversion (from visitors to purchases)
- **Challenges:** involves too many data sources and data types. Integration becomes very difficult and time consuming

If such silos are left untouched, organizations risk having outdated, unsynchronized, and even invisible datasets

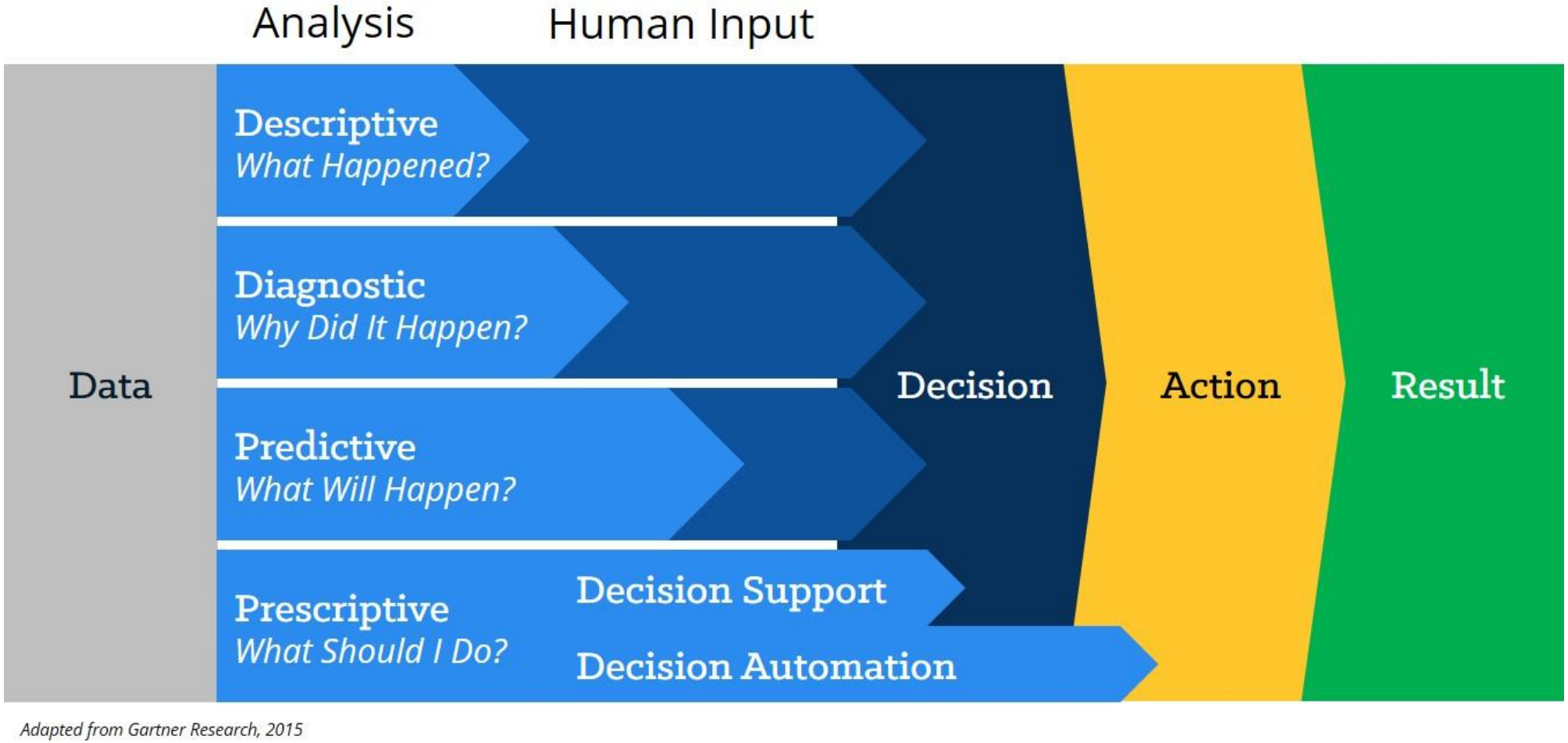




Data Analytics

Data Analytics

Data analytics is the science of analyzing raw data in order to make conclusions about that information.



Adapted from Gartner Research, 2015

Data Analytics

Data analytics is a valuable tool for businesses aiming to increase revenue, improve products, and retain customers. According to research by global management consulting firm McKinsey & Company, companies that use data analytics are 23 times more likely to outperform competitors in terms of new customer acquisition than non-data-driven companies. They were also nine times more likely to surpass them in measures of customer loyalty and 19 times more likely to achieve above-average profitability.

Data analytics can be broken into four key types:

- Descriptive, which answers the question, "**What happened?**"
- Diagnostic, which answers the question, "**Why did this happen?**"
- Predictive, which answers the question, "**What might happen in the future?**"
- Prescriptive, which answers the question, "**What should we do next?**"

Each type of data analysis can help you reach specific goals and be used in tandem to create a full picture of data that informs your organization's strategy formulation and decision-making.

Source: https://online.hbs.edu/blog/post/types-of-data-analysis



Descriptive Analytics

- **Descriptive analytics** describes what has happened over a given period of time.
- Perform well-defined computations or asking well-define questions. Ex…
  - Average Jan low temperature for each country over last 20 years
  - Number of items over $100 bought by females between ages 20 and 30
  - Frequency of specific medicine relieving specific symptoms
  - The ten stocks whose price varied the most over the past years
- Usable tools:
  - Spreadsheets, SQL, NoSQL, programming languages with big data supports (e.g. Python, Scala, R)



Descriptive Analytics: "What happened?"

Descriptive analytics aims to summarize historical data to uncover trends, patterns, or performance insights.

Key methods and tools include:

- **Statistical Descriptive Methods**: Utilizes measures like mean, median, mode, variance, and standard deviation for summarizing data distributions.
- **Clustering**: Helps in summarizing data segments using algorithms like K-Means and DBSCAN.
- **Dimensionality Reduction**: Simplifies large datasets by reducing input variables while retaining important information for better data understanding and visualization.
- **Time Series Analysis**: Describes trends over time, often utilizing moving averages or seasonal decomposition techniques.



Diagnostic Analytics

- **Diagnostic analytics** focuses on why something happened. This involves a bit of hypothesizing.
- Looking for patterns in data. Ex…
  - Did the weather affect beer sales?
  - Did that latest marketing campaign impact sales?
  - What items are bought together frequently?
  - Patients who respond well to medicines X and Y also respond well to medicine Z
- Usable tools:
  - Correlation analysis (from Statistics)
  - Data mining tools (frequent item-sets, association rules, …)



Diagnostic Analytics: "Why did this happen?"

Diagnostic analytics focuses on identifying underlying causes behind data trends or patterns.

Common methods and tools include:

- **Correlation and Regression Analysis**: Determines relationships between variables using techniques like Linear Regression and Logistic Regression.
- **Classification**: Uses labeled data to diagnose categories for new data points with algorithms like Decision Trees, Random Forest, and Support Vector Machines (SVM).
- **Root Cause Analysis (RCA) :** Identifies root causes of specific issues using methods like Ishikawa (Fishbone) diagrams and the 5 Whys technique.
- **Anomaly Detection**: Identifies outliers or unusual data points that may indicate problems using algorithms like Z-score and Isolation Forest.
- **Time Series Decomposition**: Breaks down time series data to identify trend, seasonality, and irregular components.

Predictive Analytics

- **Predictive analytics** forecasts what is likely going to happen in the near term.
- Using data to build models and make predictions. Ex…
  - What happens to sales if we have a hot summer?
  - Customers who are women over age 20 are likely to respond to an advertisement
  - Weather forecast, demand forecast (e.g. for retail, oil, agricultural products)
- Usable tools:
  - Time series prediction (regression)
  - Machine learning (classification, clustering, neural networks …)



Predictive Analytics: "What might happen in the future?"

Predictive analytics leverages historical data to make forecasts about future trends or behaviors.

Common methods and algorithms include:

- **Regression Analysis**: Predicts continuous outcomes by identifying relationships between independent and dependent variables with techniques like Multiple Linear Regression and Ridge Regression.
- **Classification and Predictive Modeling**: Categorizes data into predefined classes using algorithms like Random Forest, Gradient Boosting Machines (GBM), and XGBoost.
- **Neural Networks (Deep Learning)**: Utilizes models like Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and Convolutional Neural Networks (CNN) for capturing complex relationships.
- **Ensemble Models**: Combines multiple predictive models to enhance accuracy with methods like Bagging and Boosting (e.g., AdaBoost, CatBoost).
- **Time Series Forecasting**: Predicts future values based on historical data using algorithms like ARIMA, Prophet, and Exponential Smoothing.



Prescriptive Analytics

- **Prescriptive analytics** factors information about possible situations or scenarios, available resources, past performance, and current performance, and suggests a course of action or strategy.
  - Ex: If the summer is likely to be hot, we should add an evening shift to the brewery and rent an additional tank to increase output.
  - Ex: Based on historical data on the usage of the road network, and recent measures from traffic sensors and reports of traffic accidents, reschedule the traffic signal system.
- Usable tools:
  - Machine learning
  - Mathematical models, Computational models
  - Operations research, Optimization algorithms, Approximation algorithms, Probabilistic algorithms, Heuristic algorithms



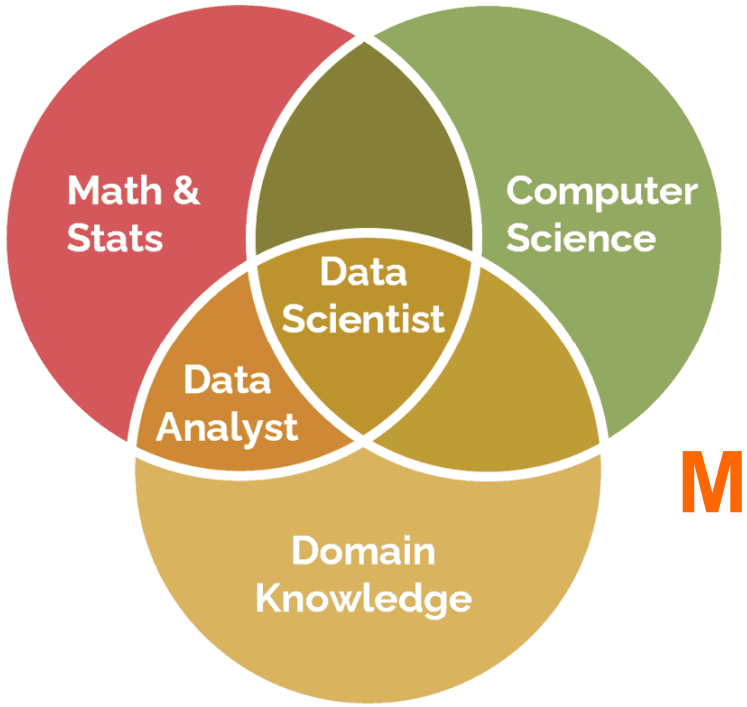
Prescriptive Analytics: "What should we do next?"

Prescriptive analytics provides recommendations based on data insights to facilitate decision making.

Common methods and tools include:

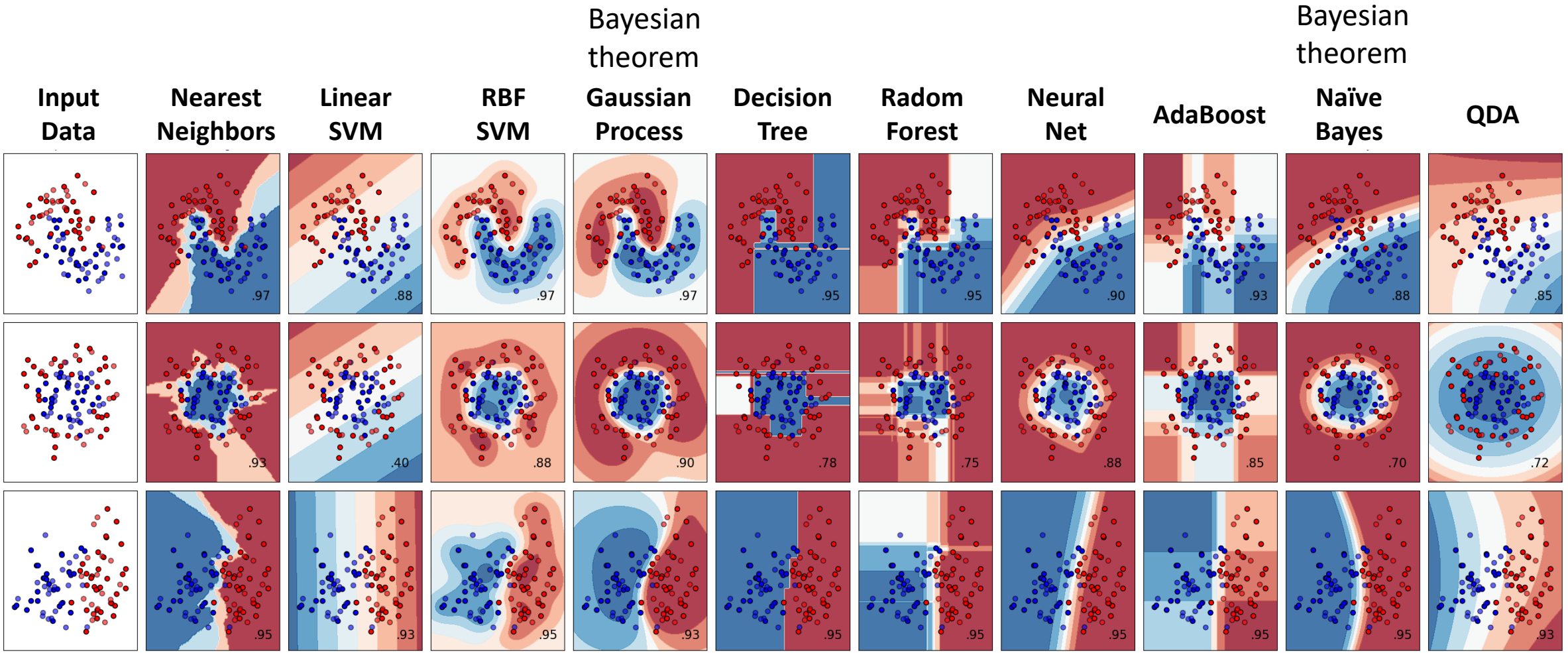
- **Optimization Models**: Helps in making optimal decisions under constraints using algorithms like Linear Programming (LP), Integer Programming (IP), and Genetic Algorithms.
- **Simulation Modeling**: Uses models to simulate different scenarios and outcomes with methods like Monte Carlo Simulations and Agent-based Models.
- **Reinforcement Learning (RL):** Learns optimal actions through trial and error with models such as Q-learning and Deep Q Networks (DQN).
- **Decision Trees**: Aids in decision-making by providing structured paths of possible outcomes using algorithms like CART and C4.5.
- **Markov Decision Processes (MDP):** Models decision-making in partially random environments, useful for policy optimization and complex system recommendations.





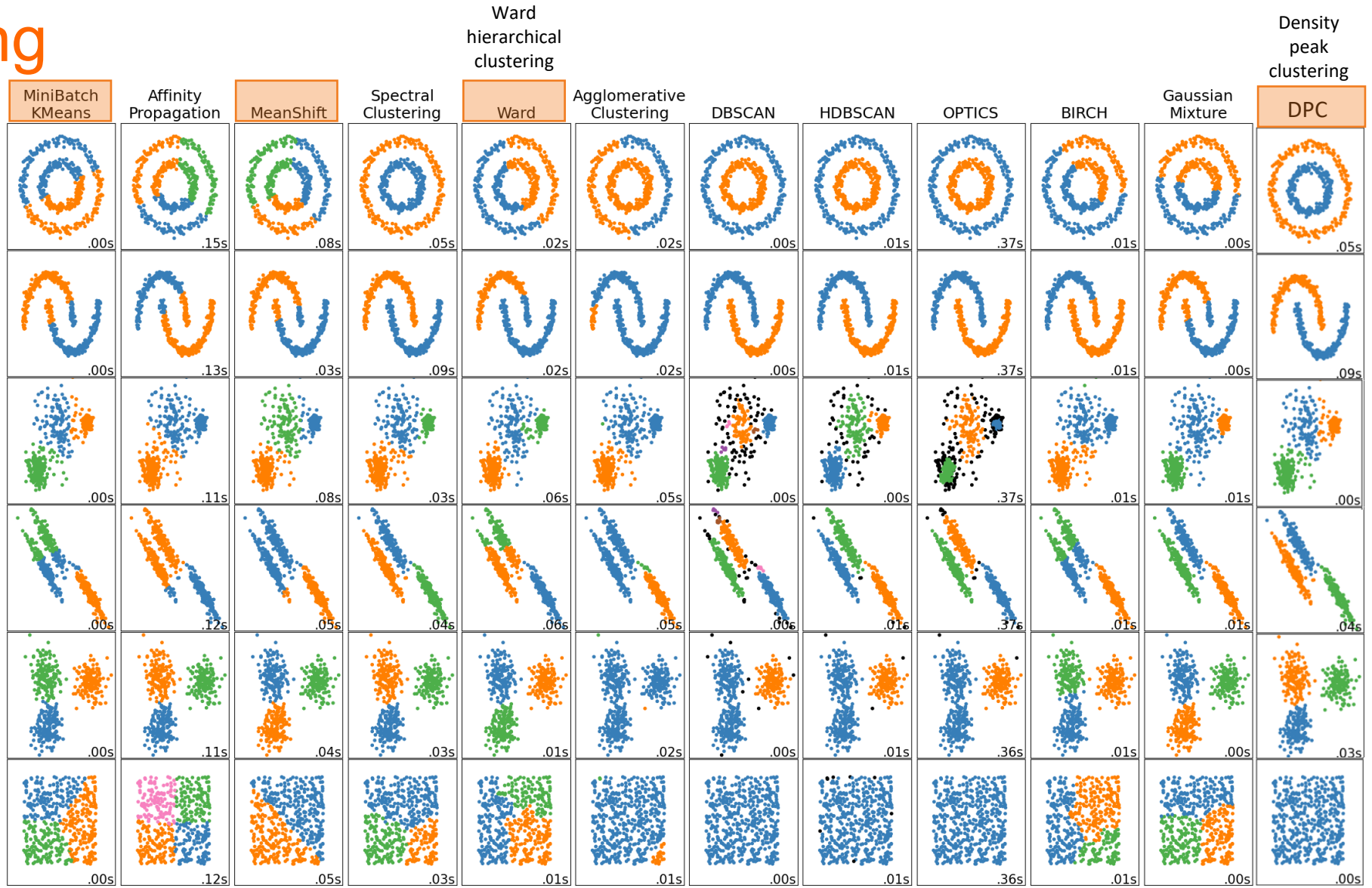
Machine Learning

Classification



Source: https://scikit-learn.org/stable/

Clustering

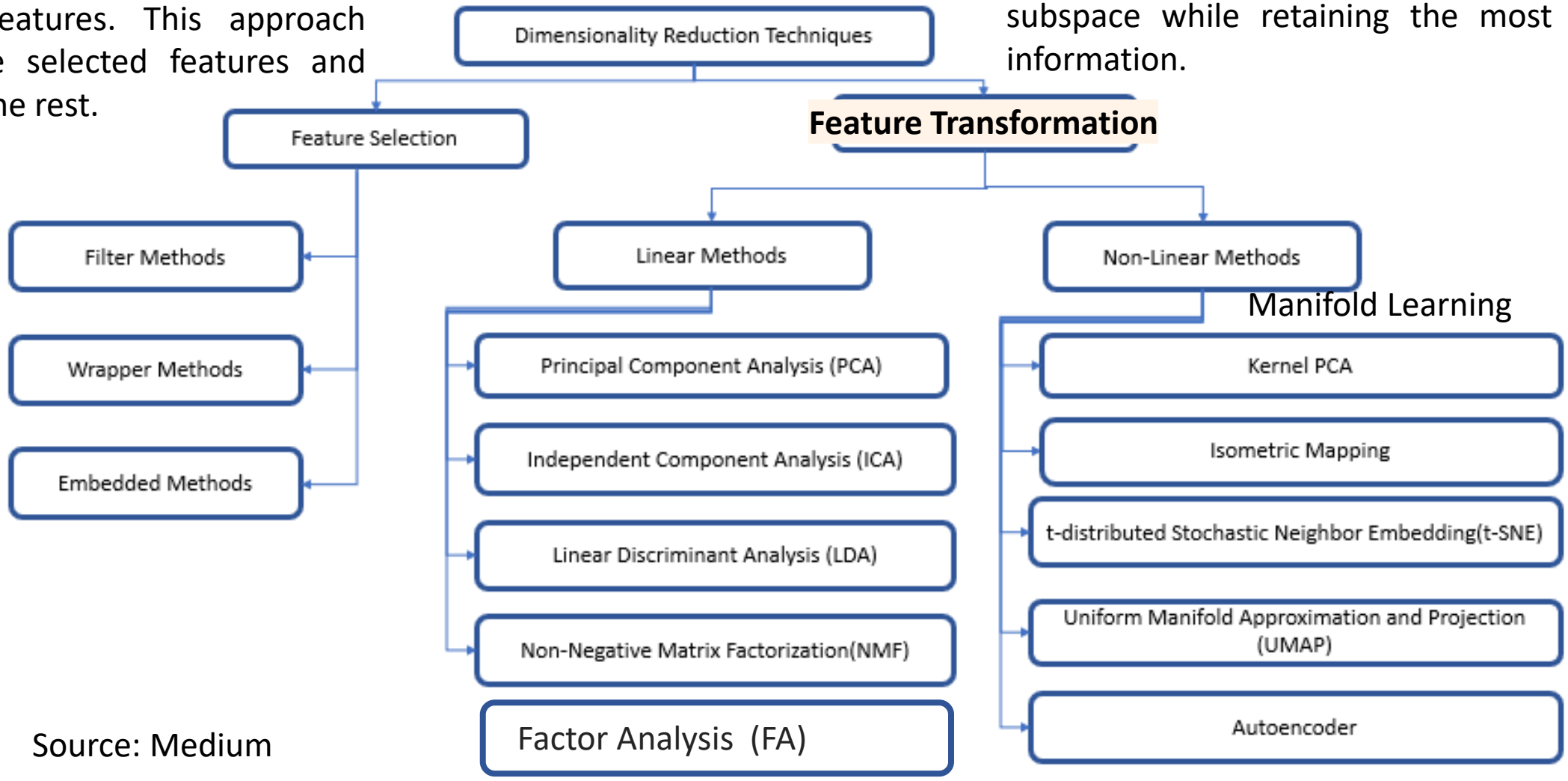


Source: https://scikit-learn.org/stable/modules/clustering

Dimensionality Reduction

Involves selecting a subset of the original features. This approach keeps the selected features and discards the rest.

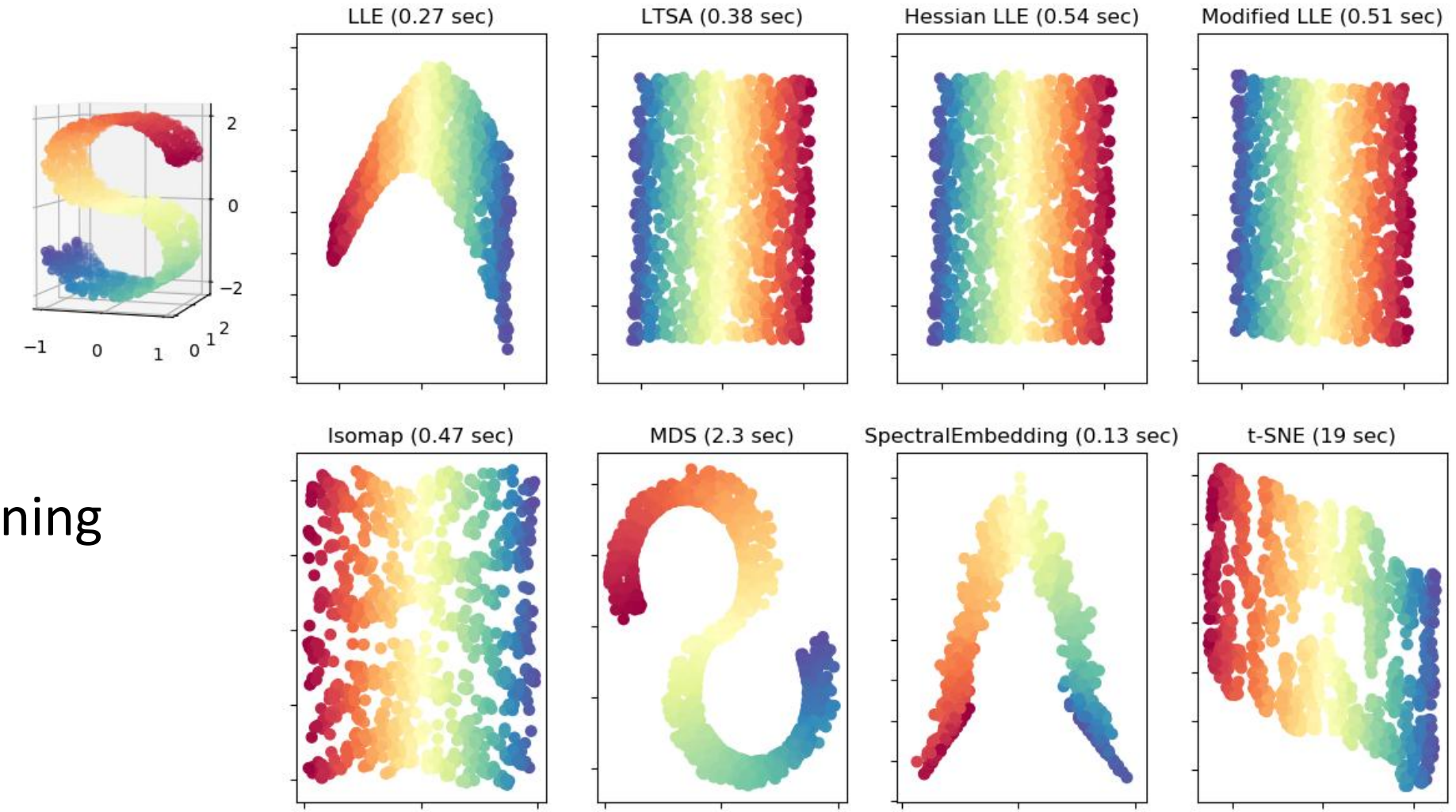
Involves transforming the original features into a lower-dimensional space. This transformation is typically done by projecting the data onto a new subspace while retaining the most important information.



Source: Medium

Dimensionality Reduction

Source: https://scikit-learn.org/

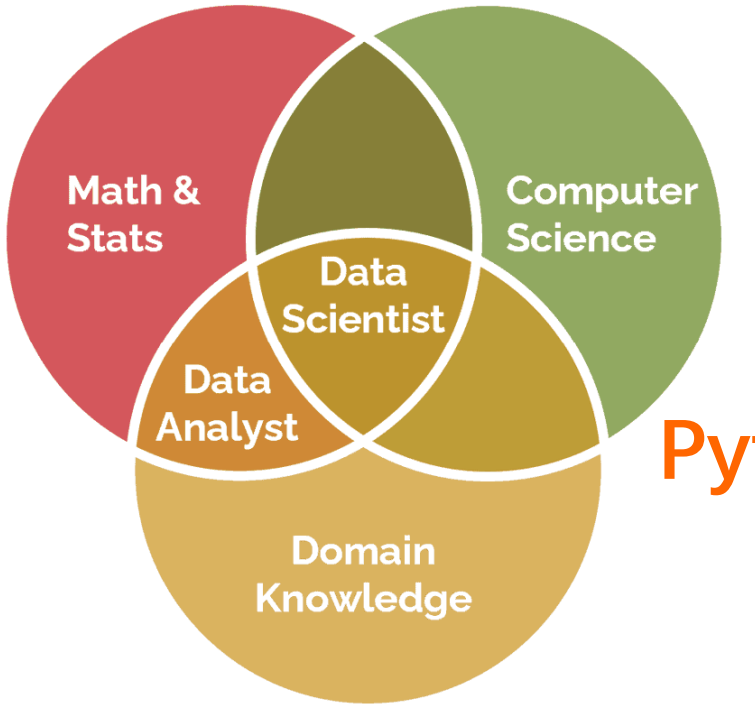


Manifold Learning

Regression

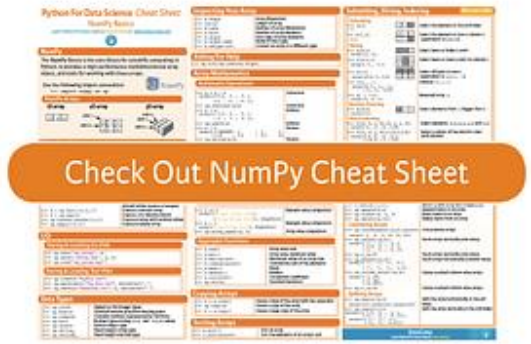


Source: https://gist.github.com/kwinkunks/48f86f4c368f5003d13dee734a16800c

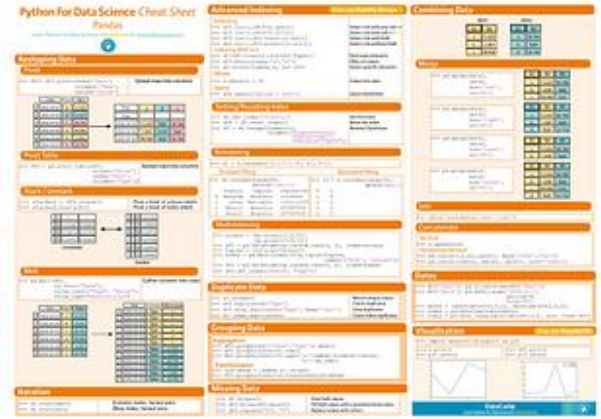


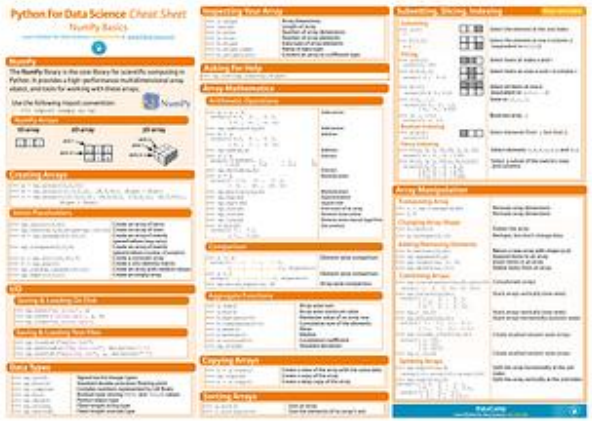
Python Packages



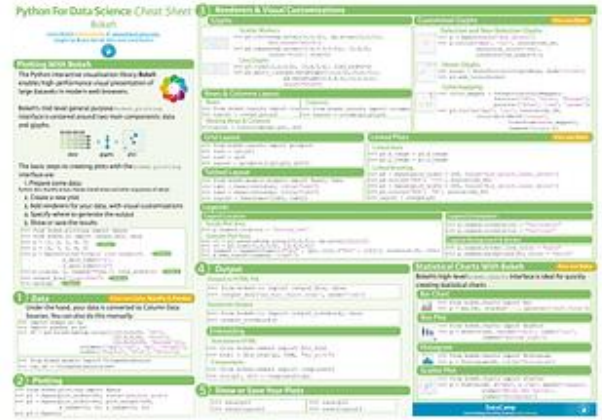










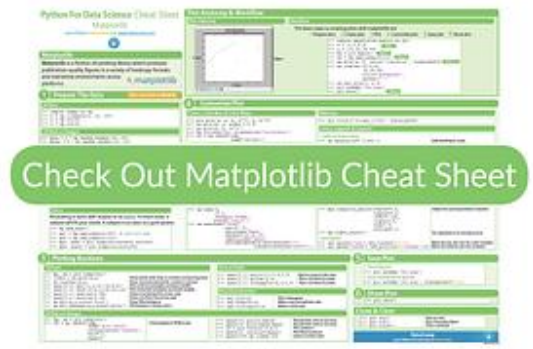












Source: https://practice2code.blogspot.com/2017/07/cheat-sheets-for-data-science-machine.html

Python For Data Science *Cheat Sheet*

NumPy Basics

Learn Python for Data Science Interactively at www.DataCamp.com

NumPy

The **NumPy** library is the core library for scientific computing in Python. It provides a high-performance multidimensional array object, and tools for working with these arrays.

Use the following import convention:

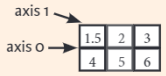
```
>>> import numpy as np
```

NumPy Arrays

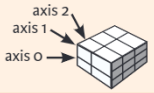
1D array



2D array



3D array



Creating Arrays

```
>>> a = np.array([1,2,3])
>>> b = np.array([(1.5,2,3), (4,5,6)], dtype = float)
>>> c = np.array([[(1.5,2,3), (4,5,6)], [(3,2,1), (4,5,6)]],
                 dtype = float)
```

Initial Placeholders

```
>>> np.zeros((3,4))                  Create an array of zeros
>>> np.ones((2,3,4),dtype=np.int16)  Create an array of ones
>>> d = np.arange(10,25,5)           Create an array of evenly
                                     spaced values (step value)
>>> np.linspace(0,2,9)               Create an array of evenly
                                     spaced values (number of samples)
>>> e = np.full((2,2),7)             Create a constant array
>>> f = np.eye(2)                    Create a 2X2 identity matrix
>>> np.random.random((2,2))          Create an array with random values
>>> np.empty((3,2))                  Create an empty array
```

I/O

Saving & Loading On Disk

```
>>> np.save('my_array', a)
>>> np.savez('array.npz', a, b)
>>> np.load('my_array.npy')
```

Saving & Loading Text Files

```
>>> np.loadtxt("myfile.txt")
>>> np.genfromtxt("my_file.csv", delimiter=',')
>>> np.savetxt("myarray.txt", a, delimiter=" ")
```

Data Types

```
>>> np.int64       Signed 64-bit integer types
>>> np.float32     Standard double-precision floating point
>>> np.complex     Complex numbers represented by 128 floats
>>> np.bool        Boolean type storing TRUE and FALSE values
>>> np.object      Python object type
>>> np.string_     Fixed-length string type
>>> np.unicode_    Fixed-length unicode type
```

Inspecting Your Array

```
>>> a.shape          Array dimensions
>>> len(a)           Length of array
>>> b.ndim           Number of array dimensions
>>> e.size           Number of array elements
>>> b.dtype          Data type of array elements
>>> b.dtype.name     Name of data type
>>> b.astype(int)    Convert an array to a different type
```

Asking For Help

```
>>> np.info(np.ndarray.dtype)
```

Array Mathematics

Arithmetic Operations

```
>>> g = a - b                        Subtraction
  array([[-0.5,  0. ,  0. ],
         [-3. , -3. , -3. ]])
>>> np.subtract(a,b)                 Subtraction
>>> b + a                            Addition
  array([[ 2.5,  4. ,  6. ],
         [ 5. ,  7. ,  9. ]])
>>> np.add(b,a)                      Addition
>>> a / b                            Division
  array([[ 0.66666667,  1.        ,  1.        ],
         [ 0.25      ,  0.4       ,  0.5       ]])
>>> np.divide(a,b)                   Division
>>> a * b                            Multiplication
  array([[  1.5,   4. ,   9. ],
         [  4. ,  10. ,  18. ]])
>>> np.multiply(a,b)                 Multiplication
>>> np.exp(b)                        Exponentiation
>>> np.sqrt(b)                       Square root
>>> np.sin(a)                        Print sines of an array
>>> np.cos(b)                        Element-wise cosine
>>> np.log(a)                        Element-wise natural logarithm
>>> e.dot(f)                         Dot product
  array([[ 7.,  7.],
         [ 7.,  7.]])
```

Comparison

```
>>> a == b                           Element-wise comparison
  array([[False,  True,  True],
         [False, False, False]], dtype=bool)
>>> a < 2                            Element-wise comparison
  array([True, False, False], dtype=bool)
>>> np.array_equal(a, b)             Array-wise comparison
```

Aggregate Functions

```
>>> a.sum()                          Array-wise sum
>>> a.min()                          Array-wise minimum value
>>> b.max(axis=0)                    Maximum value of an array row
>>> b.cumsum(axis=1)                 Cumulative sum of the elements
>>> a.mean()                         Mean
>>> b.median()                       Median
>>> a.corrcoef()                     Correlation coefficient
>>> np.std(b)                        Standard deviation
```

Copying Arrays

```
>>> h = a.view()          Create a view of the array with the same data
>>> np.copy(a)            Create a copy of the array
>>> h = a.copy()          Create a deep copy of the array
```

Sorting Arrays

```
>>> a.sort()              Sort an array
>>> c.sort(axis=0)        Sort the elements of an array's axis
```

Subsetting, Slicing, Indexing

Also see Lists

Subsetting

```
>>> a[2]                  Select the element at the 2nd index
  3
>>> b[1,2]                Select the element at row 1 column 2
  6.0                     (equivalent to b[1][2])
```

Slicing

```
>>> a[0:2]                Select items at index 0 and 1
  array([1, 2])
>>> b[0:2,1]              Select items at rows 0 and 1 in column 1
  array([ 2.,  5.])
>>> b[:1]                 Select all items at row 0
  array([[1.5, 2., 3.]])  (equivalent to b[0:1, :])
>>> c[1,...]              Same as [1,:,:]
  array([[[ 3.,  2.,  1.],
          [ 4.,  5.,  6.]]])
>>> a[ : :-1]             Reversed array a
  array([3, 2, 1])
```

Boolean Indexing

```
>>> a[a<2]                Select elements from a less than 2
  array([1])
```

Fancy Indexing

```
>>> b[[1, 0, 1, 0],[0, 1, 2, 0]]   Select elements (1,0),(0,1),(1,2) and (0,0)
  array([ 4. ,  2. ,  6. ,  1.5])
>>> b[[1, 0, 1, 0]][:,[0,1,2,0]]   Select a subset of the matrix's rows
  array([[ 4. ,  5. ,  6. ,  4. ],  and columns
         [ 1.5,  2. ,  3. ,  1.5],
         [ 4. ,  5. ,  6. ,  4. ],
         [ 1.5,  2. ,  3. ,  1.5]])
```

Array Manipulation

Transposing Array

```
>>> i = np.transpose(b)      Permute array dimensions
>>> i.T                      Permute array dimensions
```

Changing Array Shape

```
>>> b.ravel()                Flatten the array
>>> g.reshape(3,-2)          Reshape, but don't change data
```

Adding/Removing Elements

```
>>> h.resize((2,6))          Return a new array with shape (2,6)
>>> np.append(h,g)           Append items to an array
>>> np.insert(a, 1, 5)       Insert items in an array
>>> np.delete(a,[1])         Delete items from an array
```

Combining Arrays

```
>>> np.concatenate((a,d),axis=0)   Concatenate arrays
  array([ 1,  2,  3, 10, 15, 20])
>>> np.vstack((a,b))               Stack arrays vertically (row-wise)
  array([[ 1. ,  2. ,  3. ],
         [ 1.5,  2. ,  3. ],
         [ 4. ,  5. ,  6. ]])
>>> np.r_[e,f]                     Stack arrays vertically (row-wise)
>>> np.hstack((e,f))               Stack arrays horizontally (column-wise)
  array([[ 7.,  7.,  1.,  0.],
         [ 7.,  7.,  0.,  1.]])
>>> np.column_stack((a,d))         Create stacked column-wise arrays
  array([[ 1, 10],
         [ 2, 15],
         [ 3, 20]])
>>> np.c_[a,d]                     Create stacked column-wise arrays
```

Splitting Arrays

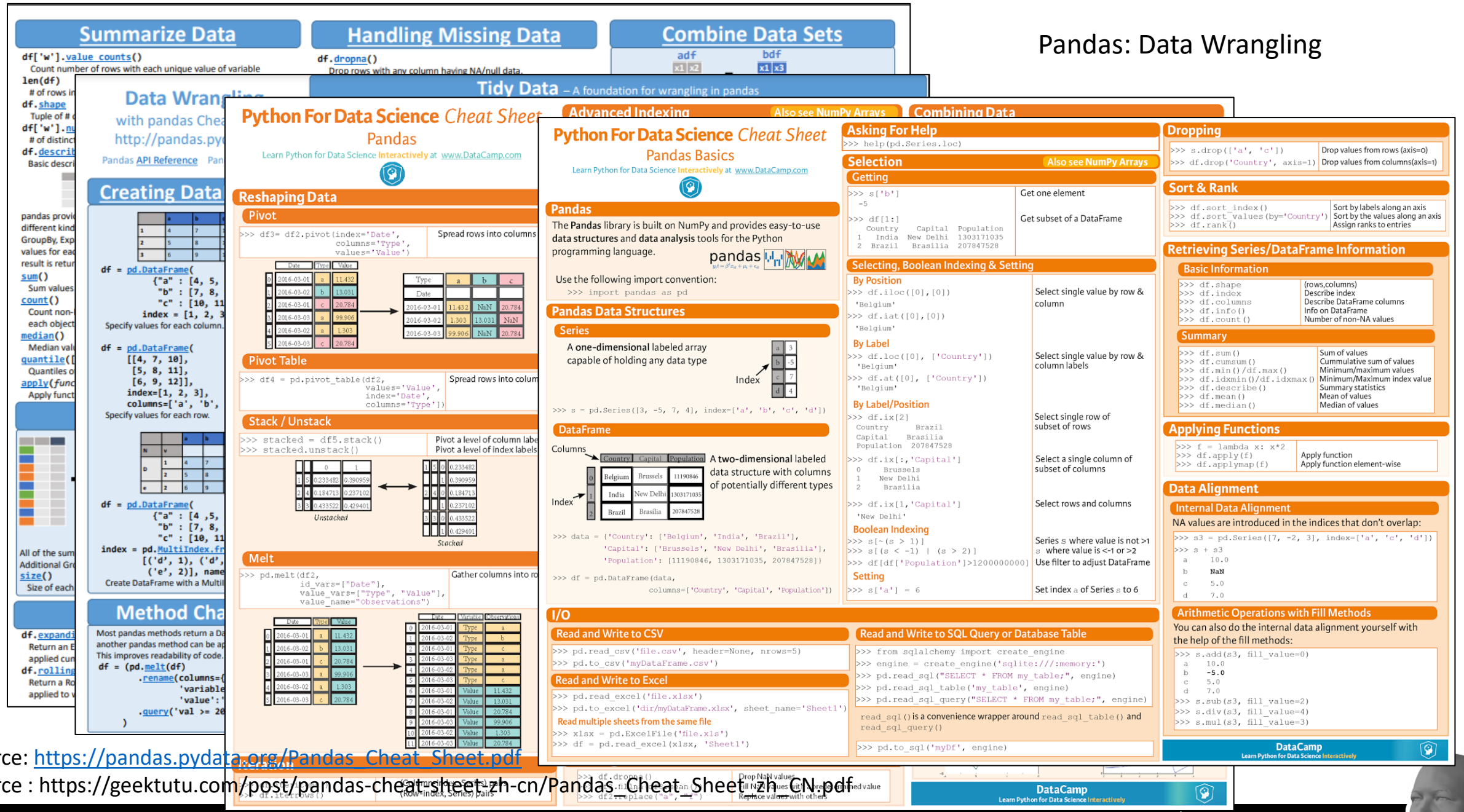
```
>>> np.hsplit(a,3)           Split the array horizontally at the 3rd
  [array([1]),array([2]),array([3])]  index
>>> np.vsplit(c,2)           Split the array vertically at the 2nd index
[array([[[ 1.5,  2. ,  1. ],
         [ 4. ,  5. ,  6. ]]]),
 array([[[ 3. ,  2. ,  3. ],
         [ 4. ,  5. ,  6. ]]])]
```

DataCamp
Learn Python for Data Science Interactively

Numpy: Numeric Calculation

澳門理工大學
Universidade Politécnica de Macau
Macao Polytechnic University

Pandas: Data Wrangling



Source: https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf

Source : https://geektutu.com/post/pandas-cheat-sheet-zh-cn/Pandas_Cheat_Sheet_ZH_CN.pdf

Python For Data Science *Cheat Sheet*

SciPy - Linear Algebra

Learn More Python for Data Science Interactively at www.datacamp.com

SciPy

The **SciPy** library is one of the core packages for scientific computing that provides mathematical algorithms and convenience functions built on the NumPy extension of Python.

Interacting With NumPy

Also see NumPy

```
>>> import numpy as np
>>> a = np.array([1,2,3])
>>> b = np.array([(1+5j,2j,3j), (4j,5j,6j)])
>>> c = np.array([[(1.5,2,3), (4,5,6)], [(3,2,1), (4,5,6)]])
```

Index Tricks

`>>> np.mgrid[0:5,0:5]`	Create a dense meshgrid
`>>> np.ogrid[0:2,0:2]`	Create an open meshgrid
`>>> np.r_[3,[0]*5,-1:1:10j]`	Stack arrays vertically (row-wise)
`>>> np.c_[b,c]`	Create stacked column-wise arrays

Shape Manipulation

`>>> np.transpose(b)`	Permute array dimensions
`>>> b.flatten()`	Flatten the array
`>>> np.hstack((b,c))`	Stack arrays horizontally (column-wise)
`>>> np.vstack((a,b))`	Stack arrays vertically (row-wise)
`>>> np.hsplit(c,2)`	Split the array horizontally at the 2nd index
`>>> np.vpslit(d,2)`	Split the array vertically at the 2nd index

Polynomials

`>>> from numpy import polyld`	
`>>> p = polyld([3,4,5])`	Create a polynomial object

Vectorizing Functions

`>>> def myfunc(a):`
`  if a < 0:`
`  return a*2`
`  else:`
`  return a/2`	

`>>> np.vectorize(myfunc)`	Vectorize functions

Type Handling

`>>> np.real(b)`	Return the real part of the array elements
`>>> np.imag(b)`	Return the imaginary part of the array elements
`>>> np.real_if_close(c,tol=1000)`	Return a real array if complex parts close to 0
`>>> np.cast['f'](np.pi)`	Cast object to a data type

Other Useful Functions

`>>> np.angle(b,deg=True)`	Return the angle of the complex argument
`>>> g = np.linspace(0,np.pi,num=5)`	Create an array of evenly spaced values (number of samples)
`>>> g [3:] += np.pi`	
`>>> np.unwrap(g)`	Unwrap
`>>> np.logspace(0,10,3)`	Create an array of evenly spaced values (log scale)
`>>> np.select([c<4],[c*2])`	Return values from a list of arrays depending on conditions
`>>> misc.factorial(a)`	Factorial
`>>> misc.comb(10,3,exact=True)`	Combine N things taken at k time
`>>> misc.central_diff_weights(3)`	Weights for Np-point central derivative
`>>> misc.derivative(myfunc,1.0)`	Find the n-th derivative of a function at a point

Linear Algebra

Also see NumPy

You'll use the `linalg` and `sparse` modules. Note that `scipy.linalg` contains and expands on `numpy.linalg`.

```
>>> from scipy import linalg, sparse
```

Creating Matrices

```
>>> A = np.matrix(np.random.random((2,2)))
>>> B = np.asmatrix(b)
>>> C = np.mat(np.random.random((10,5)))
>>> D = np.mat([[3,4], [5,6]])
```

Basic Matrix Routines

Inverse

`>>> A.I`	Inverse
`>>> linalg.inv(A)`	Inverse

Transposition

`>>> A.T`	Tranpose matrix
`>>> A.H`	Conjugate transposition

Trace

`>>> np.trace(A)`	Trace

Norm

`>>> linalg.norm(A)`	Frobenius norm
`>>> linalg.norm(A,1)`	L1 norm (max column sum)
`>>> linalg.norm(A,np.inf)`	L inf norm (max row sum)

Rank

`>>> np.linalg.matrix_rank(C)`	Matrix rank

Determinant

`>>> linalg.det(A)`	Determinant

Solving linear problems

`>>> linalg.solve(A,b)`	Solver for dense matrices
`>>> E = np.mat(a).T`	Solver for dense matrices
`>>> linalg.lstsq(F,E)`	Least-squares solution to linear matrix equation

Generalized inverse

`>>> linalg.pinv(C)`	Compute the pseudo-inverse of a matrix (least-squares solver)
`>>> linalg.pinv2(C)`	Compute the pseudo-inverse of a matrix (SVD)

Creating Sparse Matrices

`>>> F = np.eye(3, k=1)`	Create a 2X2 identity matrix
`>>> G = np.mat(np.identity(2))`	Create a 2x2 identity matrix
`>>> C[C > 0.5] = 0`	
`>>> H = sparse.csr_matrix(C)`	Compressed Sparse Row matrix
`>>> I = sparse.csc_matrix(D)`	Compressed Sparse Column matrix
`>>> J = sparse.dok_matrix(A)`	Dictionary Of Keys matrix
`>>> E.todense()`	Sparse matrix to full matrix
`>>> sparse.isspmatrix_csc(A)`	Identify sparse matrix

Sparse Matrix Routines

Inverse

`>>> sparse.linalg.inv(I)`	Inverse

Norm

`>>> sparse.linalg.norm(I)`	Norm

Solving linear problems

`>>> sparse.linalg.spsolve(H,I)`	Solver for sparse matrices

Sparse Matrix Functions

`>>> sparse.linalg.expm(I)`	Sparse matrix exponential

Matrix Functions

Addition

`>>> np.add(A,D)`	Addition

Subtraction

`>>> np.subtract(A,D)`	Subtraction

Division

`>>> np.divide(A,D)`	Division

Multiplication

`>>> A @ D`	Multiplication operator (Python 3)
`>>> np.multiply(D,A)`	Multiplication
`>>> np.dot(A,D)`	Dot product
`>>> np.vdot(A,D)`	Vector dot product
`>>> np.inner(A,D)`	Inner product
`>>> np.outer(A,D)`	Outer product
`>>> np.tensordot(A,D)`	Tensor dot product
`>>> np.kron(A,D)`	Kronecker product

Exponential Functions

`>>> linalg.expm(A)`	Matrix exponential
`>>> linalg.expm2(A)`	Matrix exponential (Taylor Series)
`>>> linalg.expm3(D)`	Matrix exponential (eigenvalue decomposition)

Logarithm Function

`>>> linalg.logm(A)`	Matrix logarithm

Trigonometric Functions

`>>> linalg.sinm(D)`	Matrix sine
`>>> linalg.cosm(D)`	Matrix cosine
`>>> linalg.tanm(A)`	Matrix tangent

Hyperbolic Trigonometric Functions

`>>> linalg.sinhm(D)`	Hypberbolic matrix sine
`>>> linalg.coshm(D)`	Hyperbolic matrix cosine
`>>> linalg.tanhm(A)`	Hyperbolic matrix tangent

Matrix Sign Function

`>>> np.signm(A)`	Matrix sign function

Matrix Square Root

`>>> linalg.sqrtm(A)`	Matrix square root

Arbitrary Functions

`>>> linalg.funm(A, lambda x: x*x)`	Evaluate matrix function

Decompositions

Eigenvalues and Eigenvectors

`>>> la, v = linalg.eig(A)`	Solve ordinary or generalized eigenvalue problem for square matrix
`>>> l1, l2 = la`	Unpack eigenvalues
`>>> v[:,0]`	First eigenvector
`>>> v[:,1]`	Second eigenvector
`>>> linalg.eigvals(A)`	Unpack eigenvalues

Singular Value Decomposition

`>>> U,s,Vh = linalg.svd(B)`	Singular Value Decomposition (SVD)
`>>> M,N = B.shape`	
`>>> Sig = linalg.diagsvd(s,M,N)`	Construct sigma matrix in SVD

LU Decomposition

`>>> P,L,U = linalg.lu(C)`	LU Decomposition

Sparse Matrix Decompositions

`>>> la, v = sparse.linalg.eigs(F,1)`	Eigenvalues and eigenvectors
`>>> sparse.linalg.svds(H, 2)`	SVD

Asking For Help

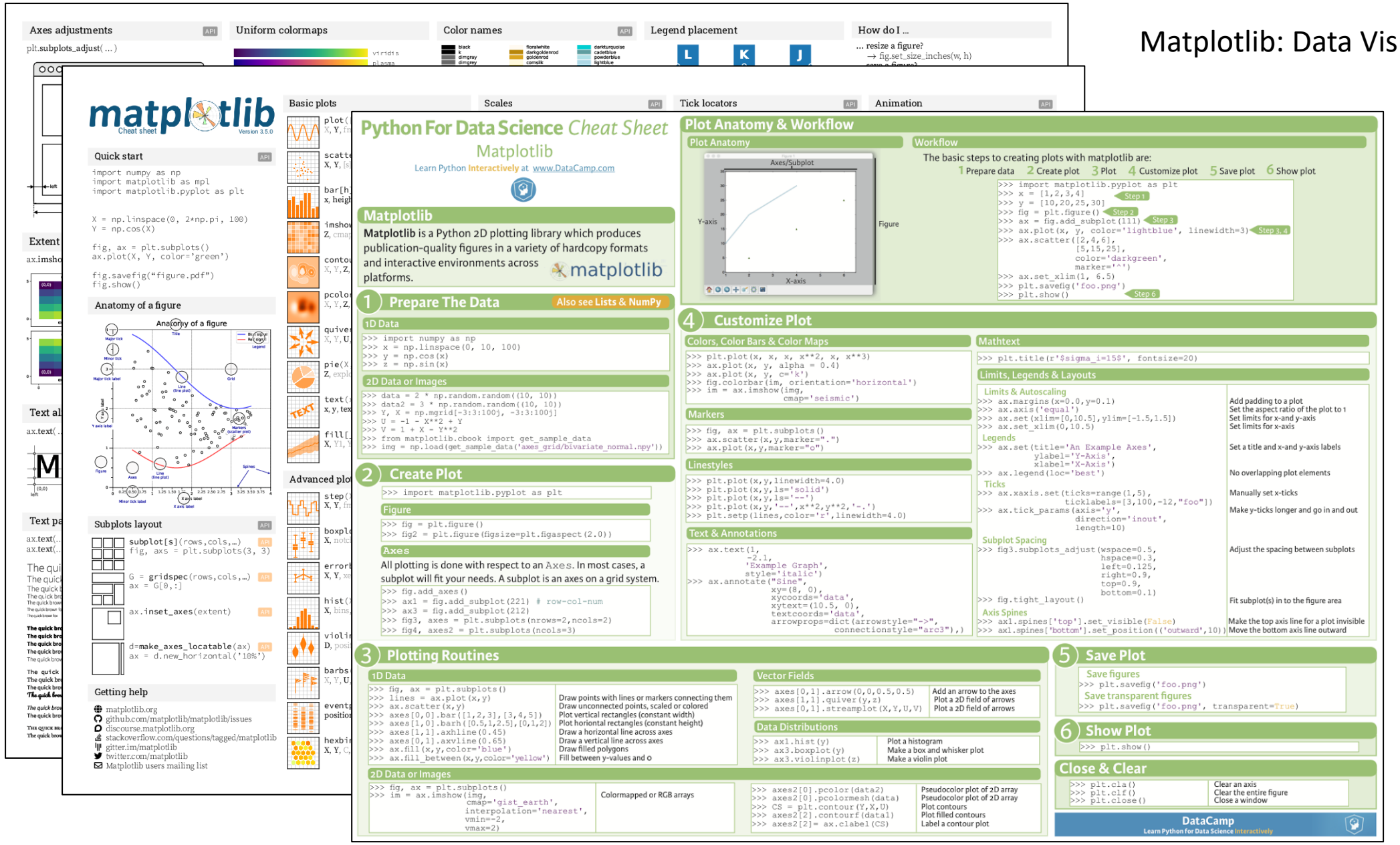
```
>>> help(scipy.linalg.diagsvd)
>>> np.info(np.matrix)
```

DataCamp
Learn Python for Data Science Interactively

Scipy: Linear Algebra



Matplotlib: Data Visualization



Python For Data Science *Cheat Sheet*

Seaborn

Learn Data Science Interactively at www.DataCamp.com

Statistical Data Visualization With Seaborn

The Python visualization library **Seaborn** is based on matplotlib and provides a high-level interface for drawing attractive statistical graphics.

Make use of the following aliases to import the libraries:

```
>>> import matplotlib.pyplot as plt
>>> import seaborn as sns
```

The basic steps to creating plots with Seaborn are:

1. Prepare some data
2. Control figure aesthetics
3. Plot with Seaborn
4. Further customize your plot

```
>>> import matplotlib.pyplot as plt
>>> import seaborn as sns
>>> tips = sns.load_dataset("tips")                    Step 1
>>> sns.set_style("whitegrid")                         Step 2
>>> g = sns.lmplot(x="tip",                            Step 3
                   y="total_bill",
                   data=tips,
                   aspect=2)
>>> g = (g.set_axis_labels("Tip","Total bill(USD)").
set(xlim=(0,10),ylim=(0,100)))                         Step 4
>>> plt.title("title")
>>> plt.show(g)                                        Step 5
```

1 Data — Also see Lists, NumPy & Pandas

```
>>> import pandas as pd
>>> import numpy as np
>>> uniform_data = np.random.rand(10, 12)
>>> data = pd.DataFrame({'x':np.arange(1,101),
                         'y':np.random.normal(0,4,100)})
```

Seaborn also offers built-in data sets:

```
>>> titanic = sns.load_dataset("titanic")
>>> iris = sns.load_dataset("iris")
```

2 Figure Aesthetics — Also see Matplotlib

```
>>> f, ax = plt.subplots(figsize=(5,6))
```
Create a figure and one subplot

Seaborn styles

```
>>> sns.set()
>>> sns.set_style("whitegrid")
>>> sns.set_style("ticks",
                  {"xtick.major.size":8,
                   "ytick.major.size":8})
>>> sns.axes_style("whitegrid")
```
(Re)set the seaborn default
Set the matplotlib parameters
Set the matplotlib parameters
Return a dict of params or use with `with` to temporarily set the style

Context Functions

```
>>> sns.set_context("talk")
>>> sns.set_context("notebook",
                    font_scale=1.5,
                    rc={"lines.linewidth":2.5})
```
Set context to `"talk"`
Set context to `"notebook"`, scale font elements and override param mapping

Color Palette

```
>>> sns.set_palette("husl",3)
>>> sns.color_palette("husl")
>>> flatui = ["#9b59b6","#3498db","#95a5a6","#e74c3c","#34495e","#2ecc71"]
>>> sns.set_palette(flatui)
```
Define the color palette
Use with `with` to temporarily set palette
Set your own color palette

3 Plotting With Seaborn

Axis Grids

```
>>> g = sns.FacetGrid(titanic,
                      col="survived",
                      row="sex")
>>> g = g.map(plt.hist,"age")
```
Subplot grid for plotting conditional relationships

```
>>> sns.factorplot(x="pclass",
                   y="survived",
                   hue="sex",
                   data=titanic)
```
Draw a categorical plot onto a Facetgrid

```
>>> sns.lmplot(x="sepal_width",
               y="sepal_length",
               hue="species",
               data=iris)
```
Plot data and regression model fits across a FacetGrid

```
>>> h = sns.PairGrid(iris)
>>> h = h.map(plt.scatter)
>>> sns.pairplot(iris)
```
Subplot grid for plotting pairwise relationships
Plot pairwise bivariate distributions

```
>>> i = sns.JointGrid(x="x",
                      y="y",
                      data=data)
>>> i = i.plot(sns.regplot,
               sns.distplot)
```
Grid for bivariate plot with marginal univariate plots

```
>>> sns.jointplot("sepal_length",
                  "sepal_width",
                  data=iris,
                  kind='kde')
```
Plot bivariate distribution

Categorical Plots

Scatterplot

```
>>> sns.stripplot(x="species",
                  y="petal_length",
                  data=iris)
```
Scatterplot with one categorical variable

```
>>> sns.swarmplot(x="species",
                  y="petal_length",
                  data=iris)
```
Categorical scatterplot with non-overlapping points

Bar Chart

```
>>> sns.barplot(x="sex",
                y="survived",
                hue="class",
                data=titanic)
```
Show point estimates and confidence intervals with scatterplot glyphs

Count Plot

```
>>> sns.countplot(x="deck",
                  data=titanic,
                  palette="Greens_d")
```
Show count of observations

Point Plot

```
>>> sns.pointplot(x="class",
                  y="survived",
                  hue="sex",
                  data=titanic,
                  palette={"male":"g",
                           "female":"m"},
                  markers=["^","o"],
                  linestyles=["-","--"])
```
Show point estimates and confidence intervals as rectangular bars

Boxplot

```
>>> sns.boxplot(x="alive",
                y="age",
                hue="adult_male",
                data=titanic)
```
Boxplot

```
>>> sns.boxplot(data=iris,orient="h")
```
Boxplot with wide-form data

Violinplot

```
>>> sns.violinplot(x="age",
                   y="sex",
                   hue="survived",
                   data=titanic)
```
Violin plot

Regression Plots

```
>>> sns.regplot(x="sepal_width",
                y="sepal_length",
                data=iris,
                ax=ax)
```
Plot data and a linear regression model fit

Distribution Plots

```
>>> plot = sns.distplot(data.y,
                        kde=False,
                        color="b")
```
Plot univariate distribution

Matrix Plots

```
>>> sns.heatmap(uniform_data,vmin=0,vmax=1)
```
Heatmap

4 Further Customizations — Also see Matplotlib

Axisgrid Objects

```
>>> g.despine(left=True)
>>> g.set_ylabels("Survived")
>>> g.set_xticklabels(rotation=45)
>>> g.set_axis_labels("Survived",
                      "Sex")
```
Remove left spine
Set the labels of the y-axis
Set the tick labels for x
Set the axis labels

```
>>> h.set(xlim=(0,5),
          ylim=(0,5),
          xticks=[0,2.5,5],
          yticks=[0,2.5,5])
```
Set the limit and ticks of the x-and y-axis

Plot

```
>>> plt.title("A Title")
>>> plt.ylabel("Survived")
>>> plt.xlabel("Sex")
>>> plt.ylim(0,100)
>>> plt.xlim(0,10)
>>> plt.setp(ax,yticks=[0,5])
>>> plt.tight_layout()
```
Add plot title
Adjust the label of the y-axis
Adjust the label of the x-axis
Adjust the limits of the y-axis
Adjust the limits of the x-axis
Adjust a plot property
Adjust subplot params

5 Show or Save Plot — Also see Matplotlib

```
>>> plt.show()
>>> plt.savefig("foo.png")
>>> plt.savefig("foo.png",
                transparent=True)
```
Show the plot
Save the plot as a figure
Save transparent figure

Close & Clear — Also see Matplotlib

```
>>> plt.cla()
>>> plt.clf()
>>> plt.close()
```
Clear an axis
Clear an entire figure
Close a window

DataCamp
Learn Python for Data Science Interactively

Seaborn: Data Visualization



Plotnine: Data Visualization

f + geom_dotplot(binaxis = "y", stackdir = "center",dotsize=0.1)
x, y, alpha, color, fill, group

f + geom_violin(scale = "area")
x, y, alpha, color, fill, group, linety

2个变量：x-离散型，y-离散

g = ggplot(diamonds, aes('cut', 'color'

g + geom_count()
x, y, alpha, color, fill, shape, size

二元连续变量分布

h=ggplot(diamonds, aes('carat', 'price'))

h + geom_bin2d(binwidth = (0.2
x, y, alpha, color, fill, linetype, siz

h + geom_density2d()
x, y, alpha, colour, group, linetyp

连续函数

i=ggplot(economics, aes(date, unemploy))

i + geom_area()
x, y, alpha, color, fill, linetype, siz

i + geom_line()
x, y, alpha, color, group, linetype,

i + geom_step(direction = "hv")
x, y, alpha, color, group, linetype,

误差绘制

```
import pandas as pd
df = pd.DataFrame({'grp':  ["A", "B"], 'fit': ran
j = ggplot(df, aes('grp', 'fit', ymin = 'fit-se', ym
```

j + geom_crossbar(fatten = 2)
x, y, ymax, ymin, alpha, color, fill,

j + geom_errorbar()
x, ymax, ymin, alpha, color, group

j + geom_linerange()
x, ymin, ymax, alpha, color, group, linetype, size

c + stat_density(adjust = 1, kernel = "gaussian")
x, y, | ..count.., ..density.., ..scaled..
c2+stat_qq(aes(sample='hwy'))
c2+stat_qq_line (aes(sample='hwy'))

Python For Data Science *Cheat Sheet*

plotnine

EasyShu团队-张杰出品

Basics ES

Plotnine是基于图层语法开发的python数据可视化包，主张模块间的协调与分工，各司其职，上帝的归上帝，凯撒的归凯撒：数据集，坐标系和geom可视化数据。



为展示数据，可以通过geom把数据集的变量映射到数据可视化的属性，比如：size(尺寸), color(颜色)和x、y轴的位置。



图表绘制的标准格式：

```
ggplot (data = <DATA> ) +                    ┐ 必需
<GEOM_FUNCTION> (mapping = aes( <MAPPINGS> ),
  stat = <STAT> , position = <POSITION> ) +  ┐
<COORDINATE_FUNCTION> +                      │
<FACET_FUNCTION>      +                      │ 可选
<SCALE_FUNCTION>      +                      │
<THEME_FUNCTION>                             ┘
```

```
from plotnine import *        #导入plotnine包的绘图函数
from plotnine.data import * #导入plotnine自带的数据集

base_plot=ggplot(data = mpg, aes(x = 'cty', y = 'hwy')) #使用ggplot()函数开始绘图，然后通过geom()函数可以添加图层。

print(base_plot)  #图表的显示

base_plot.save("plot.png", width = 5, height = 5)  #保存绘制的图表为5' x 5' 的png格式的图片 "plot.png"，图片的保存格式也可选择为pdf。
```

Geoms ES

使用geom函数绘制数据，使用geom函数的美学属性映射数据的变量，每个函数都返回一个图层。

图元 ES

```
a = ggplot(economics, aes('date', 'unemploy'))
b = ggplot(seals, aes(x = 'long', y = 'lat'))
```

a + geom_blank()
(用于扩大限制)

a + geom_path(lineend="butt", linejoin="round")
x, y, alpha, color, group, linetype, size

a + geom_polygon(aes(group = 'date'))
x, y, alpha, color, fill, group, linetype, size

b + geom_rect(aes(xmin = 'long', ymin='lat', xmax= 'long + 1', ymax = 'lat + 1'))
xmax, xmin, ymax, ymin, alpha, color, fill, linetype, size

a + geom_ribbon(aes(ymin='unemploy - 900', ymax= 'unemploy + 900'))
x, ymax, ymin, alpha, color, fill, group, linetype, size

线条: 公用的映射属性：x, y, alpha, color, linetype, size

b + geom_abline(aes(intercept=0, slope=1))
b + geom_hline(aes(yintercept = 'lat'))
b + geom_vline(aes(xintercept = 'long'))

b + geom_segment(aes(yend='lat+1', xend='long+1'))
b + geom_spoke(aes(angle = range(0,seals.shape[0]), radius = 1))

1个变量：连续型 ES

```
c = ggplot(mpg, aes('hwy')); c2 = ggplot(mpg)
```

c + geom_area(stat = "bin")
x, y, alpha, color, fill, linetype, size

c + geom_density(kernel = "gaussian")
x, y, alpha, color, fill, group, linetype, size, weight

c + geom_dotplot()
x, y, alpha, color, fill

c + geom_freqpoly() x, y, alpha, color, group, linetype, size

c + geom_histogram(binwidth = 5) x, y, alpha, color, fill, linetype, size, weight

c2 + geom_qq(aes(sample = 'hwy'))
x, y, alpha, color, fill, linetype, size, weight

1个变量：离散型 ES

```
d =ggplot(mpg, aes('fl'))
```

d + geom_bar()
x, alpha, color, fill, linetype, size, weight

2个变量：x-连续型，y-连续型 ES

```
e = ggplot(mpg, aes('cty', 'hwy'))
```

e + geom_label(aes(label = 'cty'), nudge_x = 1, nudge_y = 1)
x, y, label, alpha, angle, color, family, fontface, hjust, size, vjust

e + geom_jitter(height = 2, width = 2)
x, y, alpha, color, fill, shape, size

e + geom_point()
x, y, alpha, color, fill, shape, size, stroke

e + geom_quantile()
x, y, alpha, color, group, linetype, size, weight

e + geom_rug(sides = "bl")
x, y, alpha, color, linetype, size

e + geom_smooth(method = 'lm')
x, y, alpha, color, fill, group, linetype, size, weight

e + geom_text(aes(label = 'cty'), nudge_x = 1, nudge_y = 1)
x, y, label, alpha, angle, color, family, fontface, hjust, size, vjust

2个变量：x-离散型，y-连续型 ES

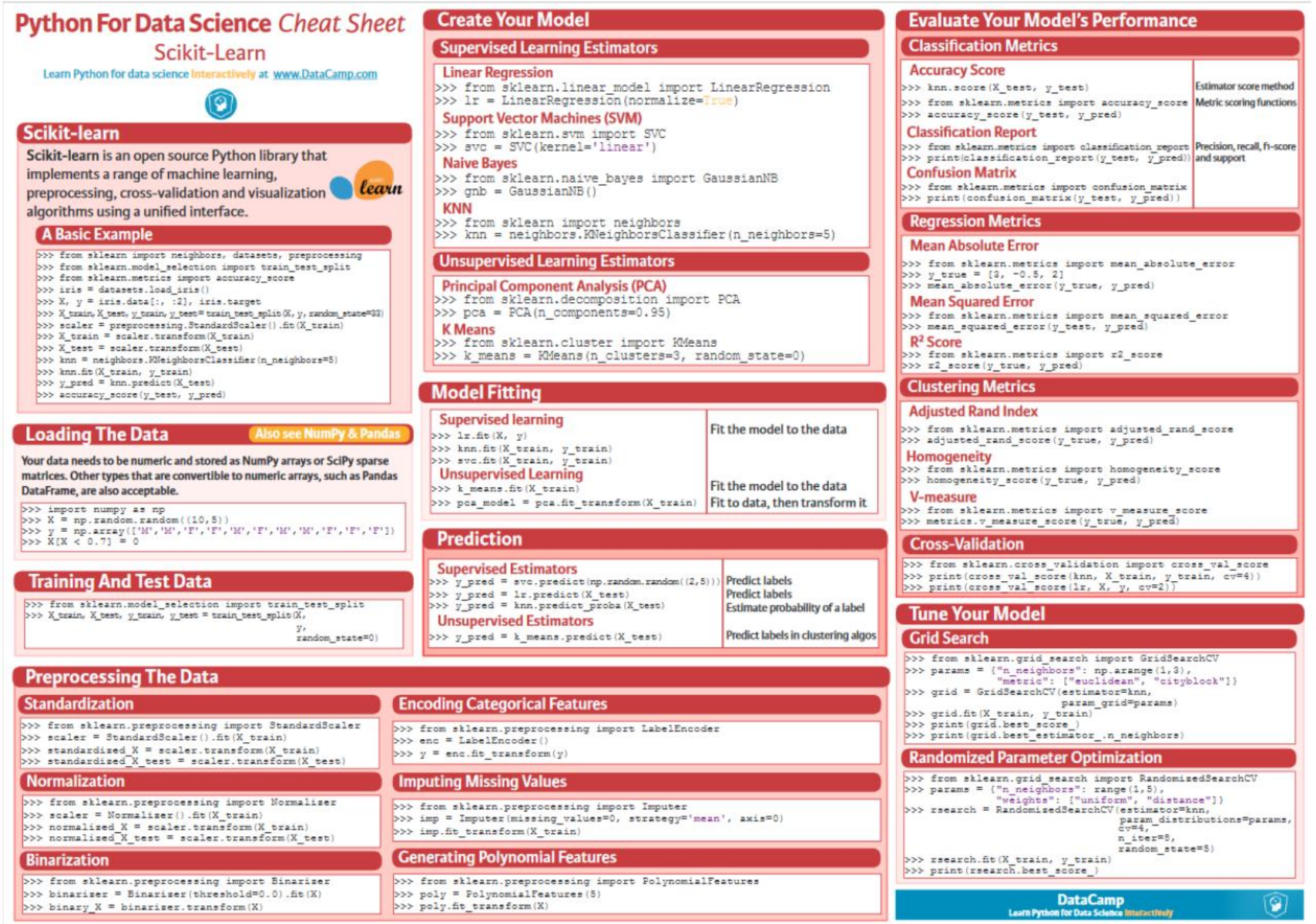
```
f=ggplot(mpg, aes('class', 'hwy'))
```

f + geom_col()
x, y, alpha, color, fill, group, linetype, size

f + geom_boxplot()
x, y, lower, middle, upper, ymax, ymin, alpha, color, fill, group, linetype, shape, size, weight

澳門理工大學
Universidade Politécnica de Macau
Macao Polytechnic University





Python For Data Science *Cheat Sheet*

Scikit-Learn

Learn Python for data science Interactively at www.DataCamp.com

Scikit-learn

Scikit-learn is an open source Python library that implements a range of machine learning, preprocessing, cross-validation and visualization algorithms using a unified interface.

A Basic Example

```
>>> from sklearn import neighbors, datasets, preprocessing
>>> from sklearn.model_selection import train_test_split
>>> from sklearn.metrics import accuracy_score
>>> iris = datasets.load_iris()
>>> X, y = iris.data[:, :2], iris.target
>>> X_train, X_test, y_train, y_test= train_test_split(X, y, random_state=33)
>>> scaler = preprocessing.StandardScaler().fit(X_train)
>>> X_train = scaler.transform(X_train)
>>> X_test = scaler.transform(X_test)
>>> knn = neighbors.KNeighborsClassifier(n_neighbors=5)
>>> knn.fit(X_train, y_train)
>>> y_pred = knn.predict(X_test)
>>> accuracy_score(y_test, y_pred)
```

Loading The Data

Also see NumPy & Pandas

Your data needs to be numeric and stored as NumPy arrays or SciPy sparse matrices. Other types that are convertible to numeric arrays, such as Pandas DataFrame, are also acceptable.

```
>>> import numpy as np
>>> X = np.random.random((10,5))
>>> y = np.array(['M','M','F','F','M','F','M','M','F','F','F'])
>>> X[X < 0.7] = 0
```

Training And Test Data

```
>>> from sklearn.model_selection import train_test_split
>>> X_train, X_test, y_train, y_test = train_test_split(X,
                                                        y,
                                                        random_state=0)
```

Create Your Model

Supervised Learning Estimators

Linear Regression

```
>>> from sklearn.linear_model import LinearRegression
>>> lr = LinearRegression(normalize=True)
```

Support Vector Machines (SVM)

```
>>> from sklearn.svm import SVC
>>> svc = SVC(kernel='linear')
```

Naive Bayes

```
>>> from sklearn.naive_bayes import GaussianNB
>>> gnb = GaussianNB()
```

KNN

```
>>> from sklearn import neighbors
>>> knn = neighbors.KNeighborsClassifier(n_neighbors=5)
```

Unsupervised Learning Estimators

Principal Component Analysis (PCA)

```
>>> from sklearn.decomposition import PCA
>>> pca = PCA(n_components=0.95)
```

K Means

```
>>> from sklearn.cluster import KMeans
>>> k_means = KMeans(n_clusters=3, random_state=0)
```

Model Fitting

Supervised learning	Fit the model to the data
`>>> lr.fit(X, y)`	
`>>> knn.fit(X_train, y_train)`	
`>>> svc.fit(X_train, y_train)`	
Unsupervised Learning	
`>>> k_means.fit(X_train)`	Fit the model to the data
`>>> pca_model = pca.fit_transform(X_train)`	Fit to data, then transform it

Prediction

Supervised Estimators	
`>>> y_pred = svc.predict(np.random.random((2,5)))`	Predict labels
`>>> y_pred = lr.predict(X_test)`	Predict labels
`>>> y_pred = knn.predict_proba(X_test)`	Estimate probability of a label
Unsupervised Estimators	
`>>> y_pred = k_means.predict(X_test)`	Predict labels in clustering algos

Evaluate Your Model's Performance

Classification Metrics

Accuracy Score — Estimator score method

```
>>> knn.score(X_test, y_test)
```

Metric scoring functions

```
>>> from sklearn.metrics import accuracy_score
>>> accuracy_score(y_test, y_pred)
```

Classification Report — Precision, recall, f1-score and support

```
>>> from sklearn.metrics import classification_report
>>> print(classification_report(y_test, y_pred))
```

Confusion Matrix

```
>>> from sklearn.metrics import confusion_matrix
>>> print(confusion_matrix(y_test, y_pred))
```

Regression Metrics

Mean Absolute Error

```
>>> from sklearn.metrics import mean_absolute_error
>>> y_true = [3, -0.5, 2]
>>> mean_absolute_error(y_true, y_pred)
```

Mean Squared Error

```
>>> from sklearn.metrics import mean_squared_error
>>> mean_squared_error(y_test, y_pred)
```

R^2 Score

```
>>> from sklearn.metrics import r2_score
>>> r2_score(y_true, y_pred)
```

Clustering Metrics

Adjusted Rand Index

```
>>> from sklearn.metrics import adjusted_rand_score
>>> adjusted_rand_score(y_true, y_pred)
```

Homogeneity

```
>>> from sklearn.metrics import homogeneity_score
>>> homogeneity_score(y_true, y_pred)
```

V-measure

```
>>> from sklearn.metrics import v_measure_score
>>> metrics.v_measure_score(y_true, y_pred)
```

Cross-Validation

```
>>> from sklearn.cross_validation import cross_val_score
>>> print(cross_val_score(knn, X_train, y_train, cv=4))
>>> print(cross_val_score(lr, X, y, cv=2))
```

Tune Your Model

Grid Search

```
>>> from sklearn.grid_search import GridSearchCV
>>> params = {"n_neighbors": np.arange(1,3),
              "metric": ["euclidean", "cityblock"]}
>>> grid = GridSearchCV(estimator=knn,
                        param_grid=params)
>>> grid.fit(X_train, y_train)
>>> print(grid.best_score_)
>>> print(grid.best_estimator_.n_neighbors)
```

Randomized Parameter Optimization

```
>>> from sklearn.grid_search import RandomizedSearchCV
>>> params = {"n_neighbors": range(1,5),
              "weights": ["uniform", "distance"]}
>>> rsearch = RandomizedSearchCV(estimator=knn,
                                 param_distributions=params,
                                 cv=4,
                                 n_iter=8,
                                 random_state=5)
>>> rsearch.fit(X_train, y_train)
>>> print(rsearch.best_score_)
```

Preprocessing The Data

Standardization

```
>>> from sklearn.preprocessing import StandardScaler
>>> scaler = StandardScaler().fit(X_train)
>>> standardized_X = scaler.transform(X_train)
>>> standardized_X_test = scaler.transform(X_test)
```

Normalization

```
>>> from sklearn.preprocessing import Normalizer
>>> scaler = Normalizer().fit(X_train)
>>> normalized_X = scaler.transform(X_train)
>>> normalized_X_test = scaler.transform(X_test)
```

Binarization

```
>>> from sklearn.preprocessing import Binarizer
>>> binarizer = Binarizer(threshold=0.0).fit(X)
>>> binary_X = binarizer.transform(X)
```

Encoding Categorical Features

```
>>> from sklearn.preprocessing import LabelEncoder
>>> enc = LabelEncoder()
>>> y = enc.fit_transform(y)
```

Imputing Missing Values

```
>>> from sklearn.preprocessing import Imputer
>>> imp = Imputer(missing_values=0, strategy='mean', axis=0)
>>> imp.fit_transform(X_train)
```

Generating Polynomial Features

```
>>> from sklearn.preprocessing import PolynomialFeatures
>>> poly = PolynomialFeatures(5)
>>> poly.fit_transform(X)
```

DataCamp
Learn Python for Data Science Interactively

scikit-learn:
Machine Learning

Source: https://scikit-learn.org/stable/



Statsmodels: Data Analysis

statsmodels 0.14.1 Stable Search statsmodels v0.14.2 9.9k 2.9k

statsmodels 0.14.1
Installing statsmodels
Getting started
User Guide
Background
Regression and Linear Models
Linear Regression
Module Reference
Generalized Linear Models
Generalized Estimating Equations
Generalized Additive Models (GAM)
Robust Linear Models
Linear Mixed Effects Models
Regression with Discrete Dependent Variable
Generalized Linear Mixed Effects Models

Linear Regression

Linear models with independently and identically distributed errors, and for errors with heteroscedasticity or autocorrelation. This module allows estimation by ordinary least squares (OLS), weighted least squares (WLS), generalized least squares (GLS), and feasible generalized least squares with autocorrelated AR(p) errors.

See Module Reference for commands and arguments.

Contents
Examples
Technical Documentation
References
Attributes
Module Reference
Model Classes
Results Classes

https://www.statsmodels.org/stable/regression.html
https://github.com/statsmodels/statsmodels

Examples

```
# Load modules and data
In [1]: import numpy as np

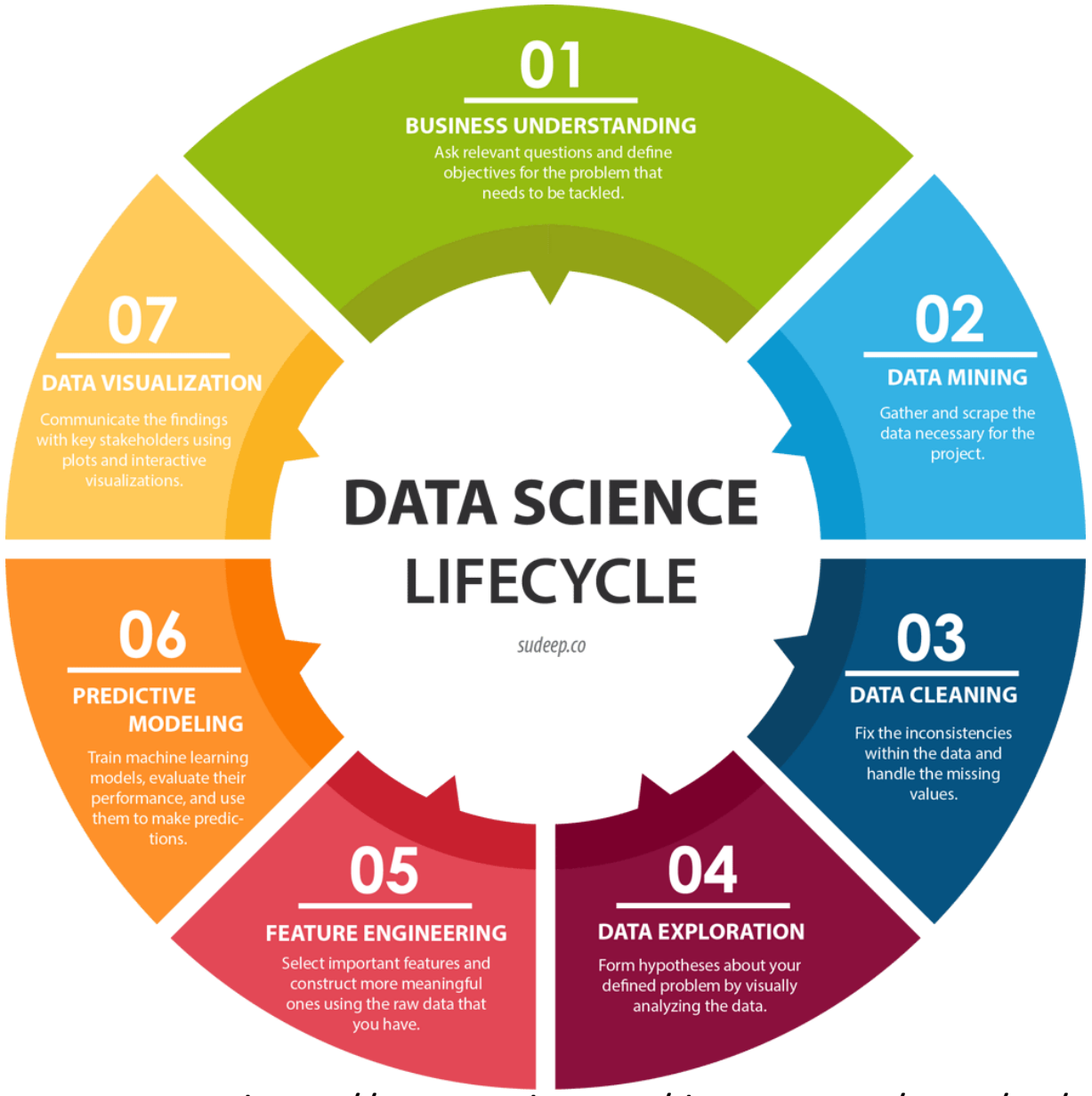
In [2]: import statsmodels.api as sm

In [3]: spector_data = sm.datasets.spector.load()

In [4]: spector_data.exog = sm.add_constant(spector_data.exog, prepend=False)

# Fit and summarize OLS model
```





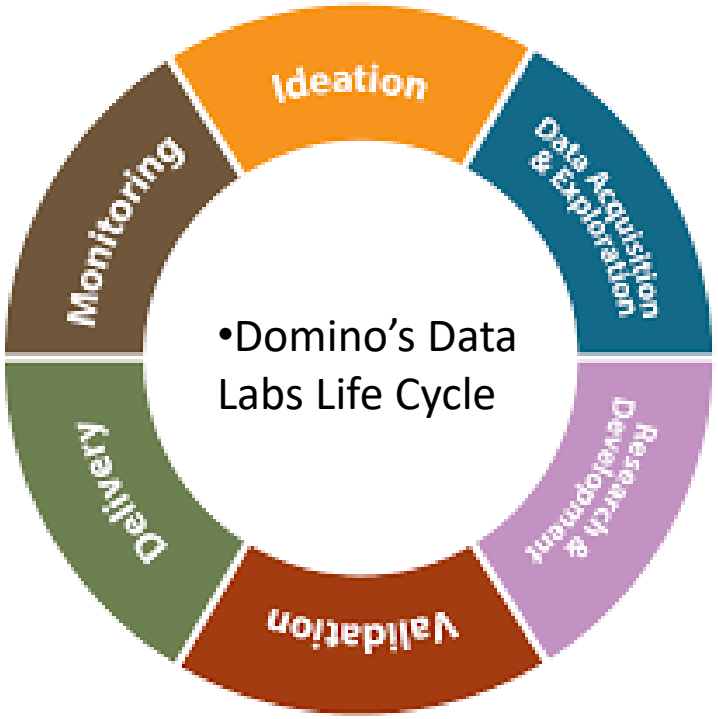
Data Science Life Cycle

Source: https://www.sudeep.co/data-science/2018/02/09/Understanding-the-Data-Science-Lifecycle.html

Data Science Life Cycle

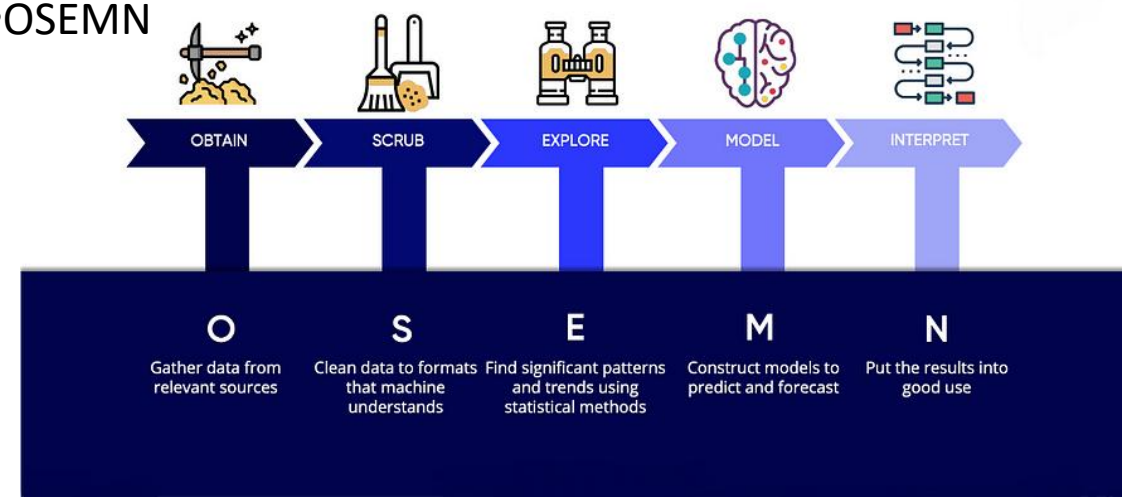
Modern Data Science Life Cycles

- OSEMN
- Microsoft's TDSP
- Domino's Data Labs Life Cycle

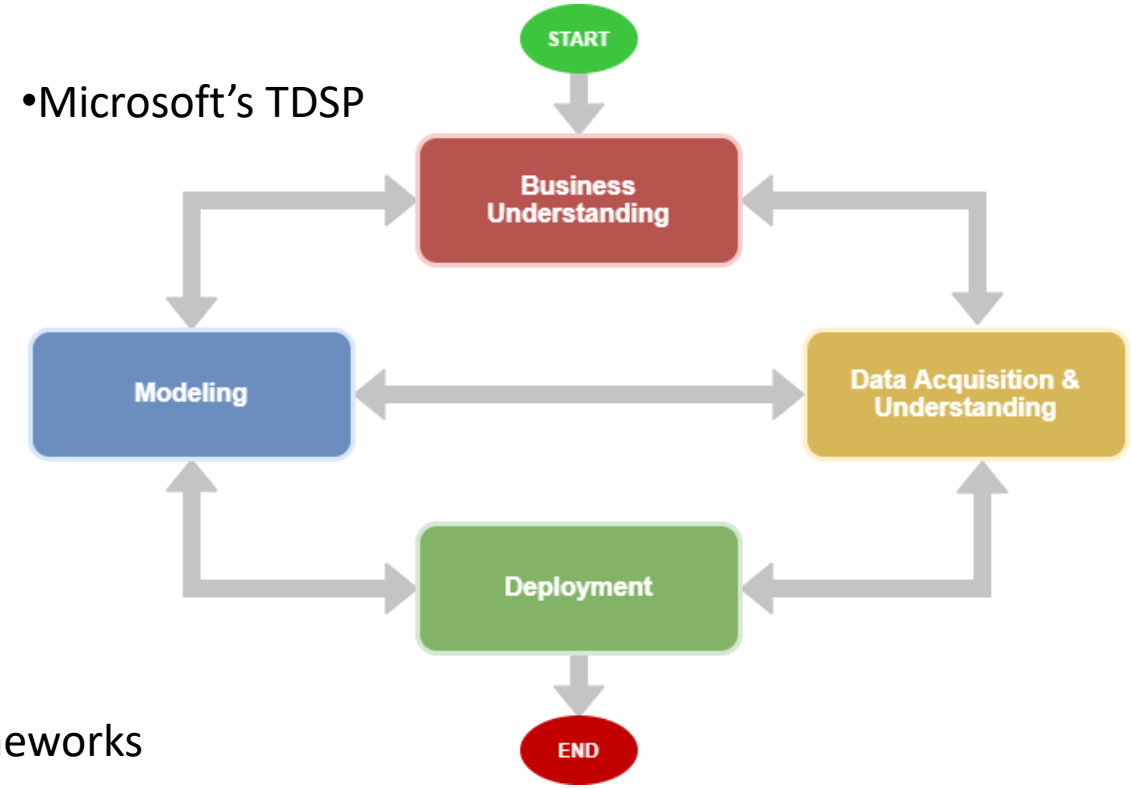


•Domino's Data Labs Life Cycle

•OSEMN



•Microsoft's TDSP

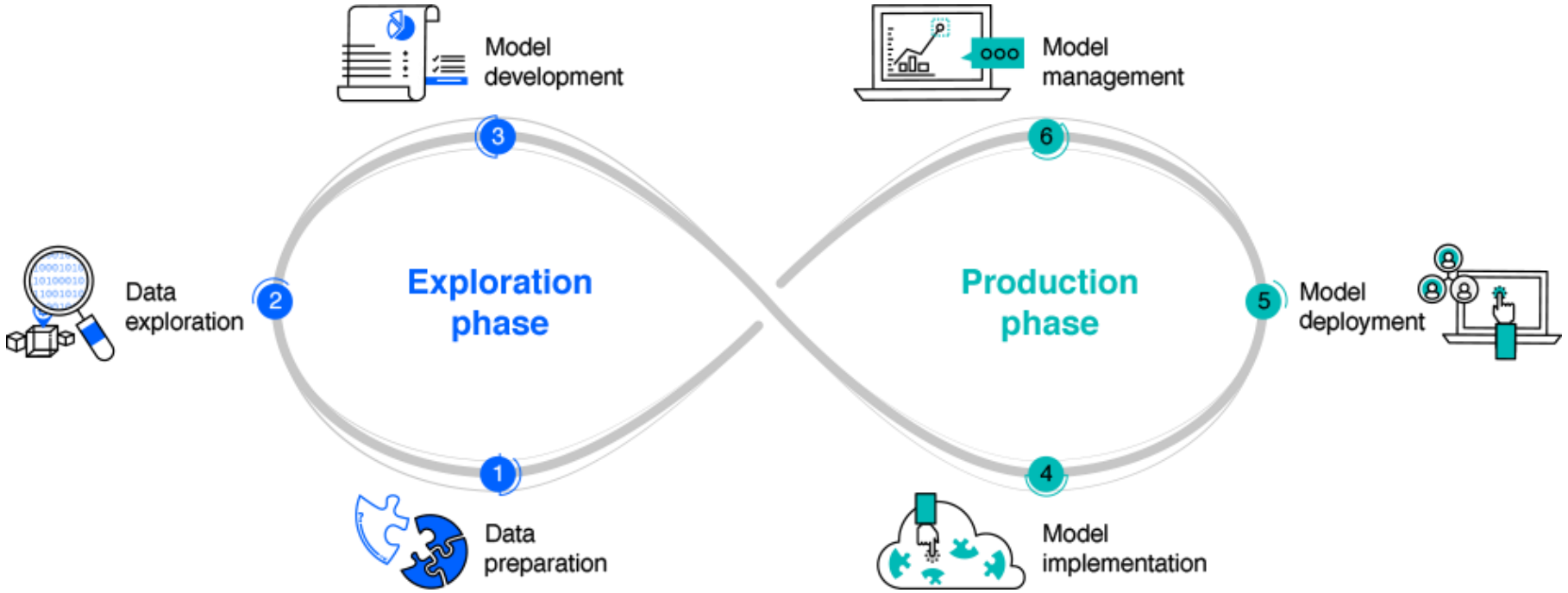


Source: https://harshvardhan.blog/data-science-process-frameworks

Data Science Life Cycle

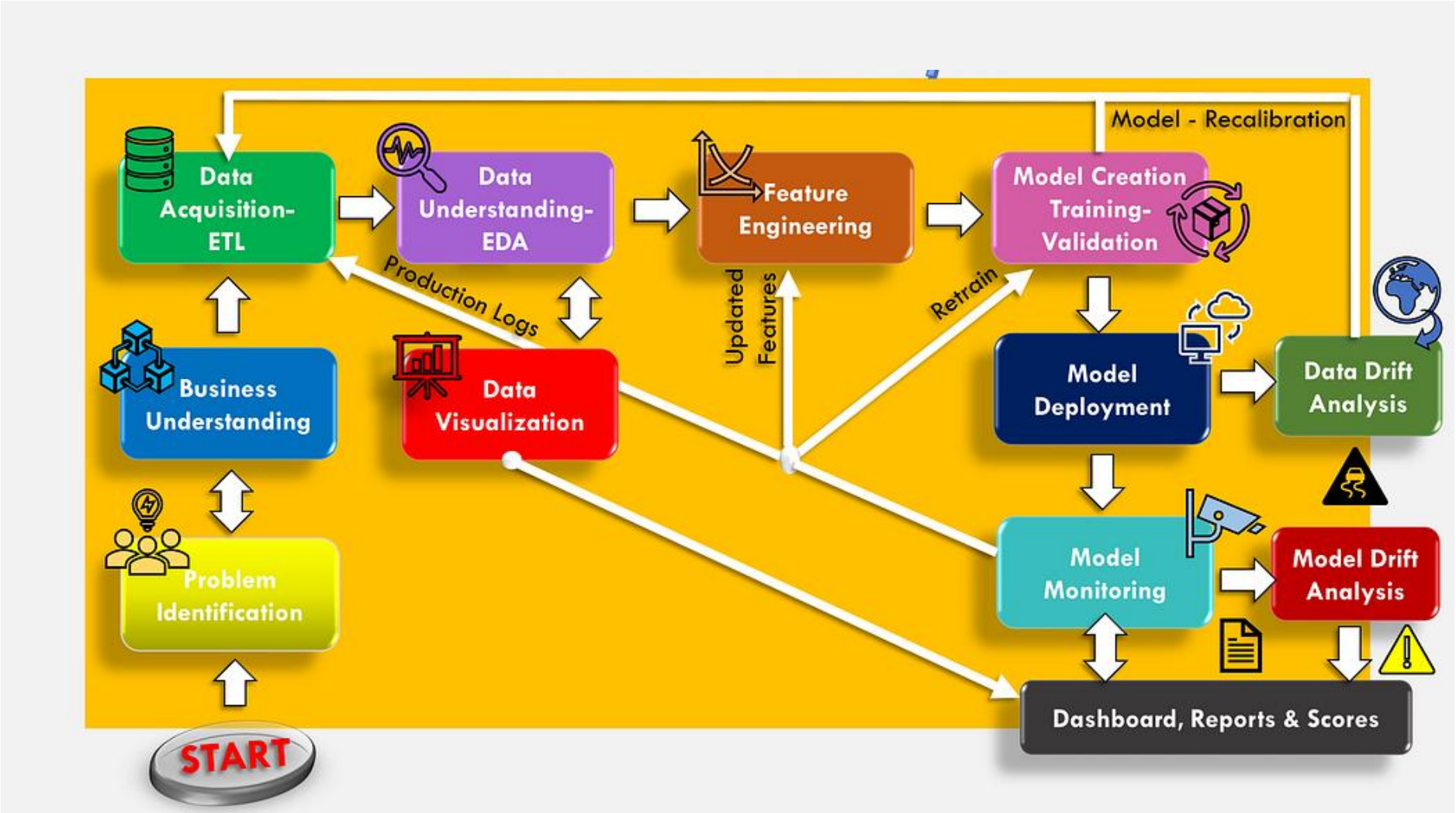
Exploration-to-Production

IBM Watson Studio embeds the practice of data science into your business, allowing you to manage the entire data science lifecycle from research to production.



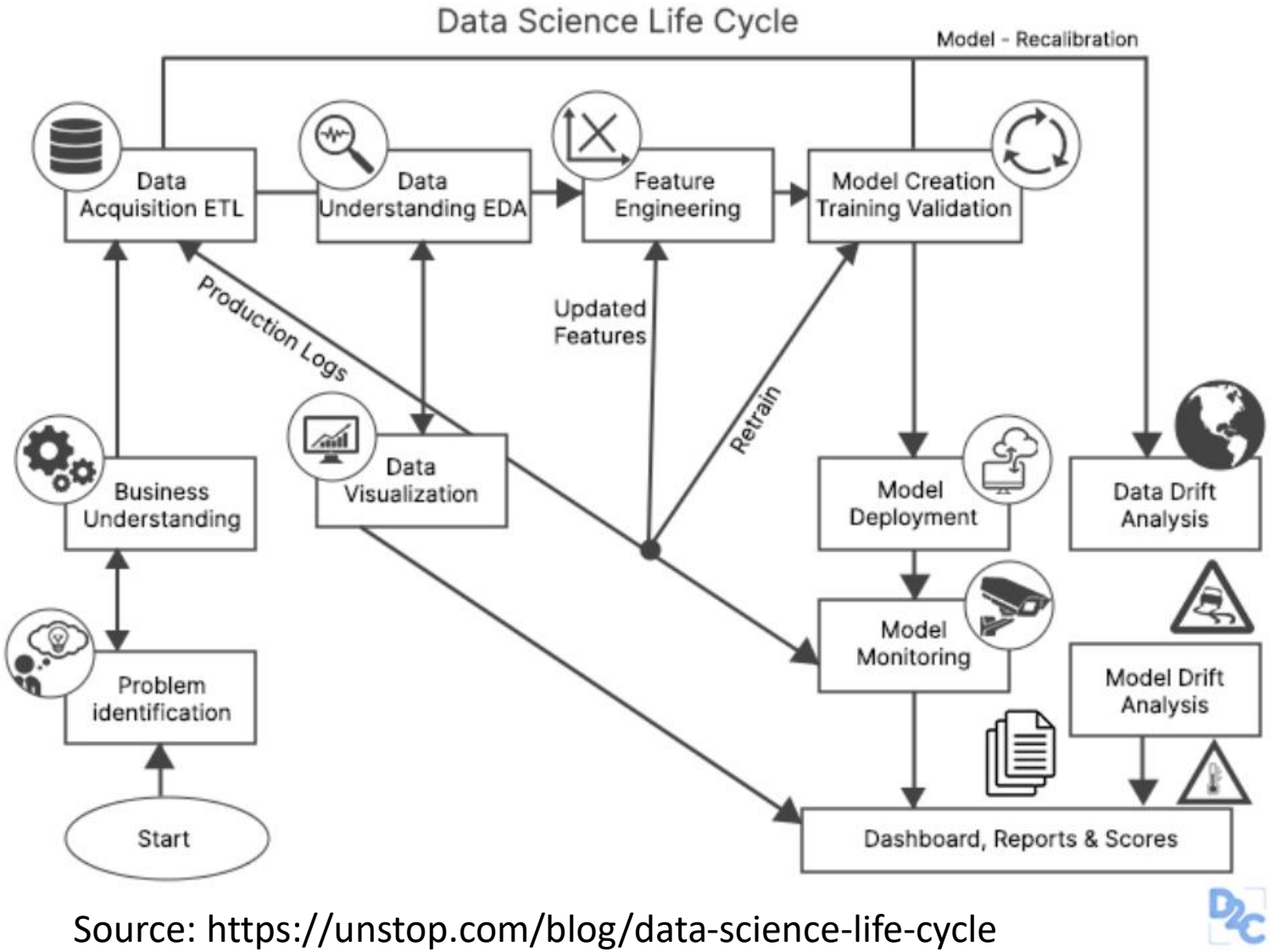
Source: https://public.dhe.ibm.com/software/data/sw-library/analytics/data-science-lifecycle/

Data Science Life Cycle



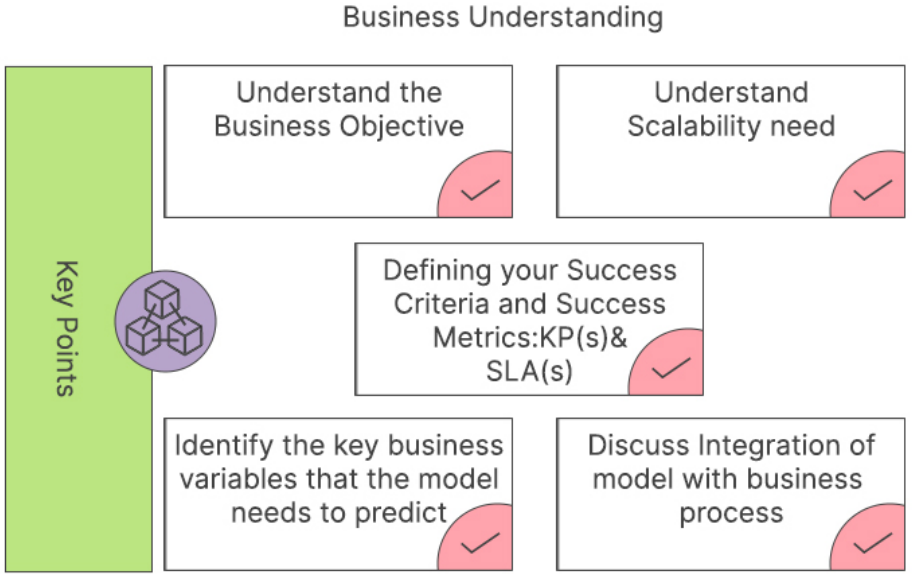
Source: https://www.genspark.ai/spark/data-science-life-cycle-diagram/30ec02a8-d550-3c9d-a61f-1289438be2c1

Data Science Life Cycle

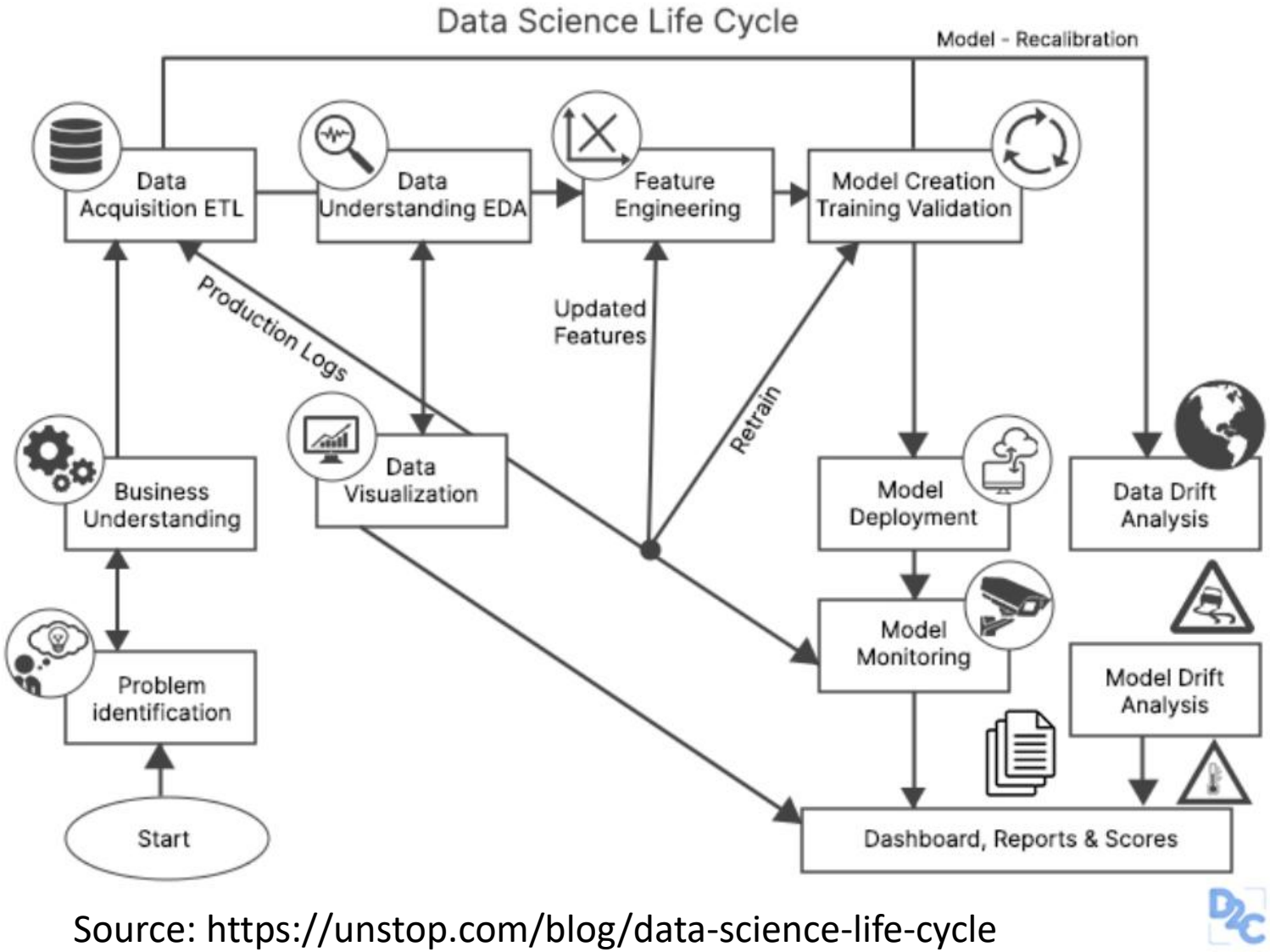


Source: https://unstop.com/blog/data-science-life-cycle

1. Problem Definition: The first step involves understanding the business problem or question that needs to be addressed using data analysis. It is crucial to clearly define the objectives and scope of the project.



Data Science Life Cycle



Source: https://unstop.com/blog/data-science-life-cycle

1. Problem Definition: The first step involves understanding the business problem or question that needs to be addressed using data analysis. It is crucial to clearly define the objectives and scope of the project.

According to Microsoft Azure's blog, we typically use data science to answer five types of questions:

1.How much or how many? (regression)

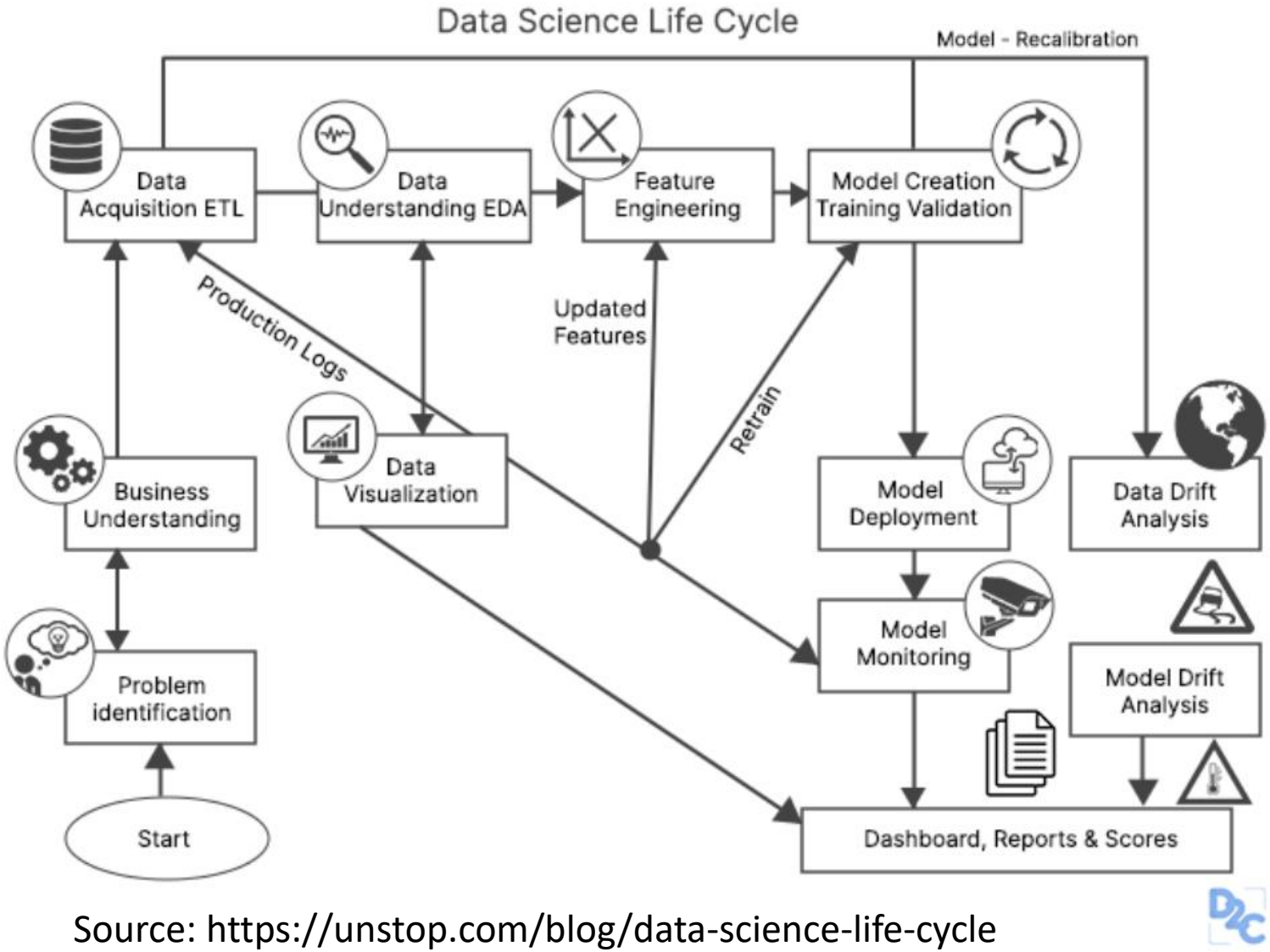
2.Which category? (classification)

3.Which group? (clustering)

4.Is this weird? (anomaly detection)

5.Which option should be taken? (recommendation)

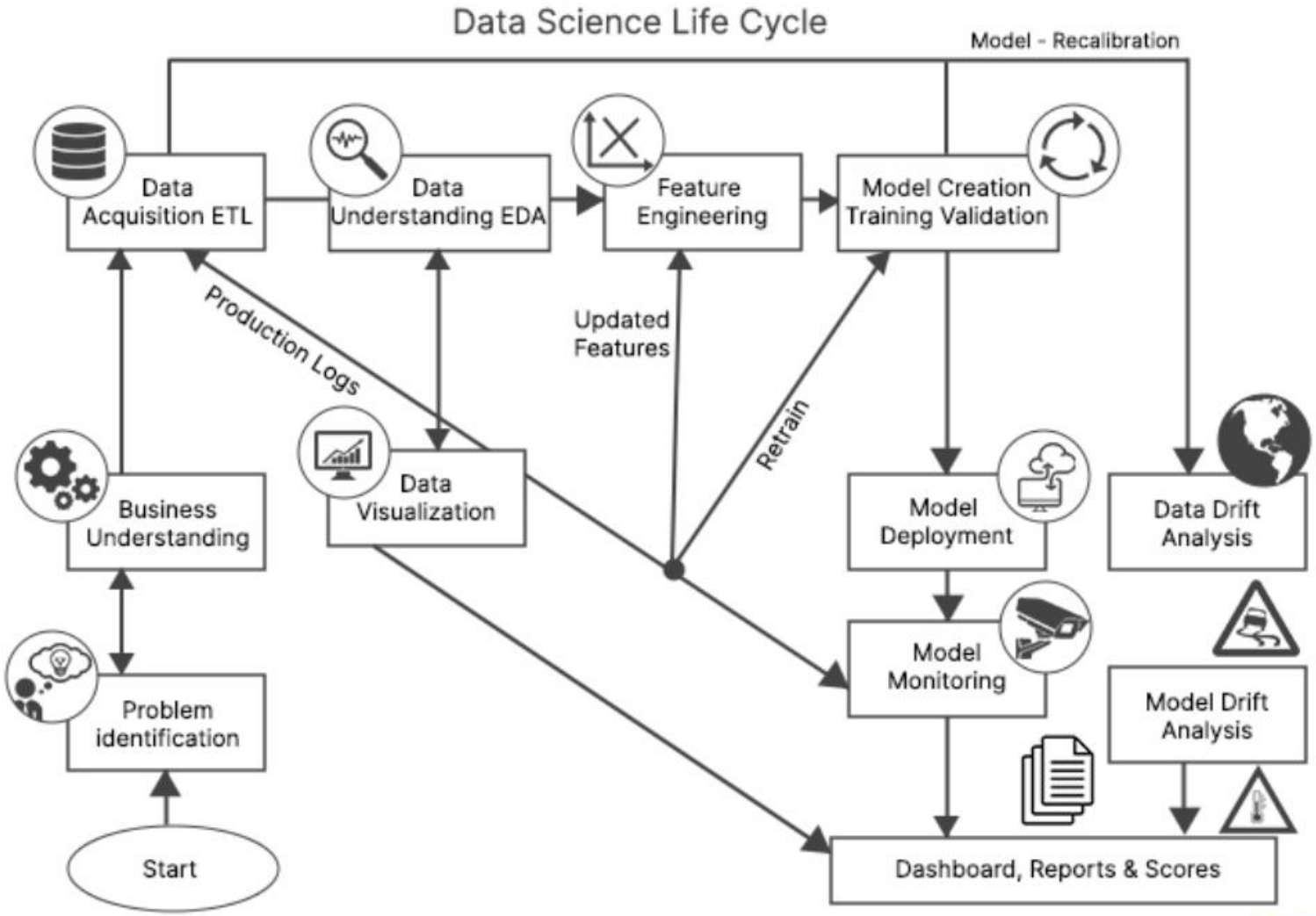
Data Science Life Cycle



Source: https://unstop.com/blog/data-science-life-cycle

2. Data Collection: In this stage, relevant data is gathered from various sources such as databases, APIs, files, and sensors. Data quality and quantity are important considerations at this stage.

Data Science Life Cycle

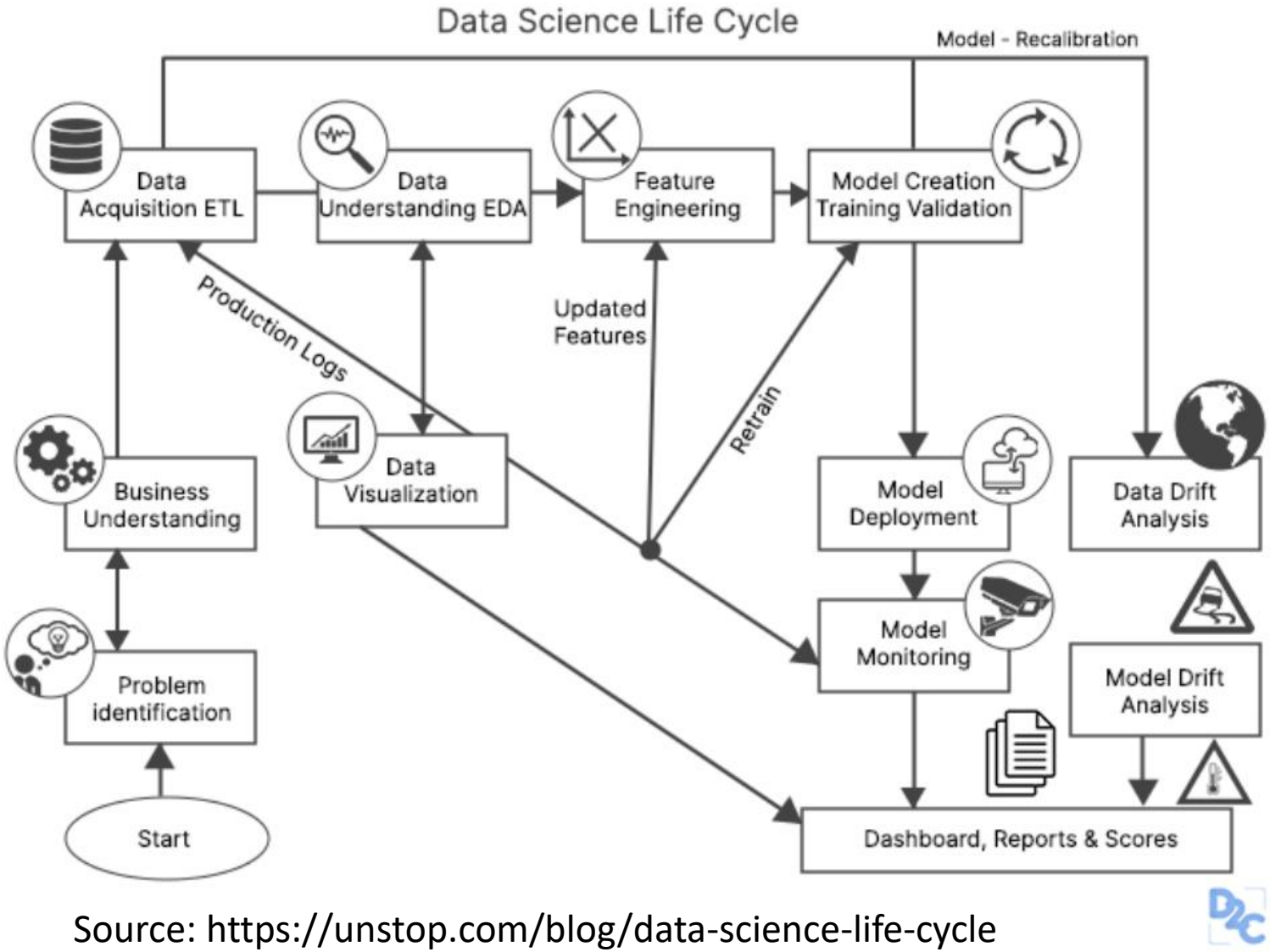


Source: https://unstop.com/blog/data-science-life-cycle

3. Data Preparation: Data cleaning and preprocessing are performed to ensure that the data is in a suitable format for analysis. Tasks include handling missing values, removing duplicates, and transforming the data into a usable form.

- **Data Cleaning**: Handle missing values, remove duplicates, and correct errors in the data.
- **Data Transformation**: Convert data into a format suitable for analysis, including reshaping, normalization, and encoding.
- **Data Integration**: Combine data from multiple sources to create a unified dataset.
- **Data Sampling**: Selecting a representative subset of data: Sample data to create balanced training and test datasets.

Data Science Life Cycle

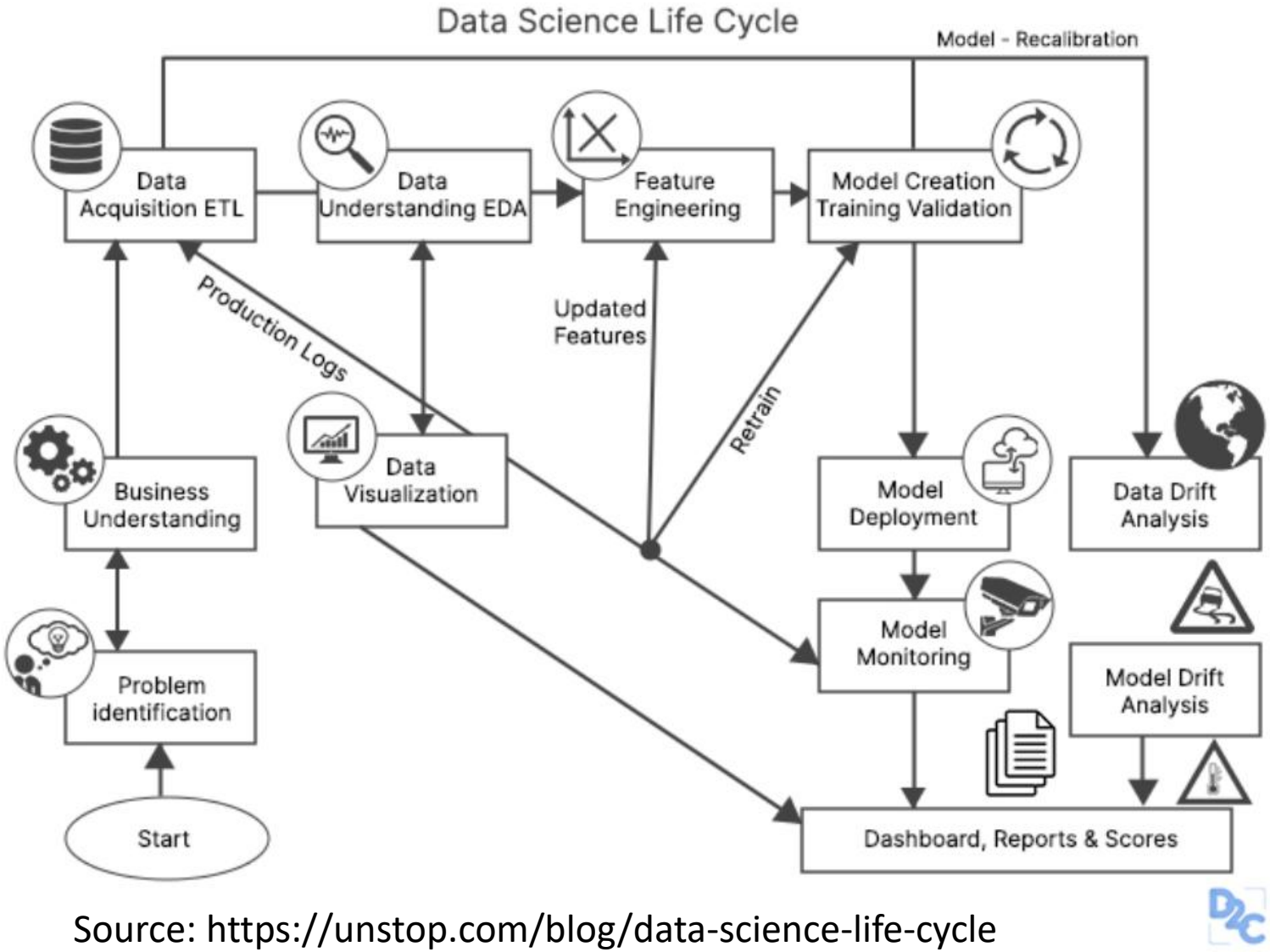


Source: https://unstop.com/blog/data-science-life-cycle

3. Data Preparation: Data cleaning and preprocessing are performed to ensure that the data is in a suitable format for analysis. Tasks include handling missing values, removing duplicates, and transforming the data into a usable form.

"Data wrangling" is a term often used interchangeably with "data munging" or "data cleaning" in the context of data science and data analysis. It refers to the process of cleaning, structuring, and enriching raw data into a format that is suitable for analysis.

Data Science Life Cycle



Source: https://unstop.com/blog/data-science-life-cycle

3. Data Preparation --- ETL

(1) Extract:

- **Definition**: The first step in ETL involves extracting data from various sources such as databases, applications, files, APIs, or streams.
- **Purpose**: This step retrieves raw data from source systems for further processing.

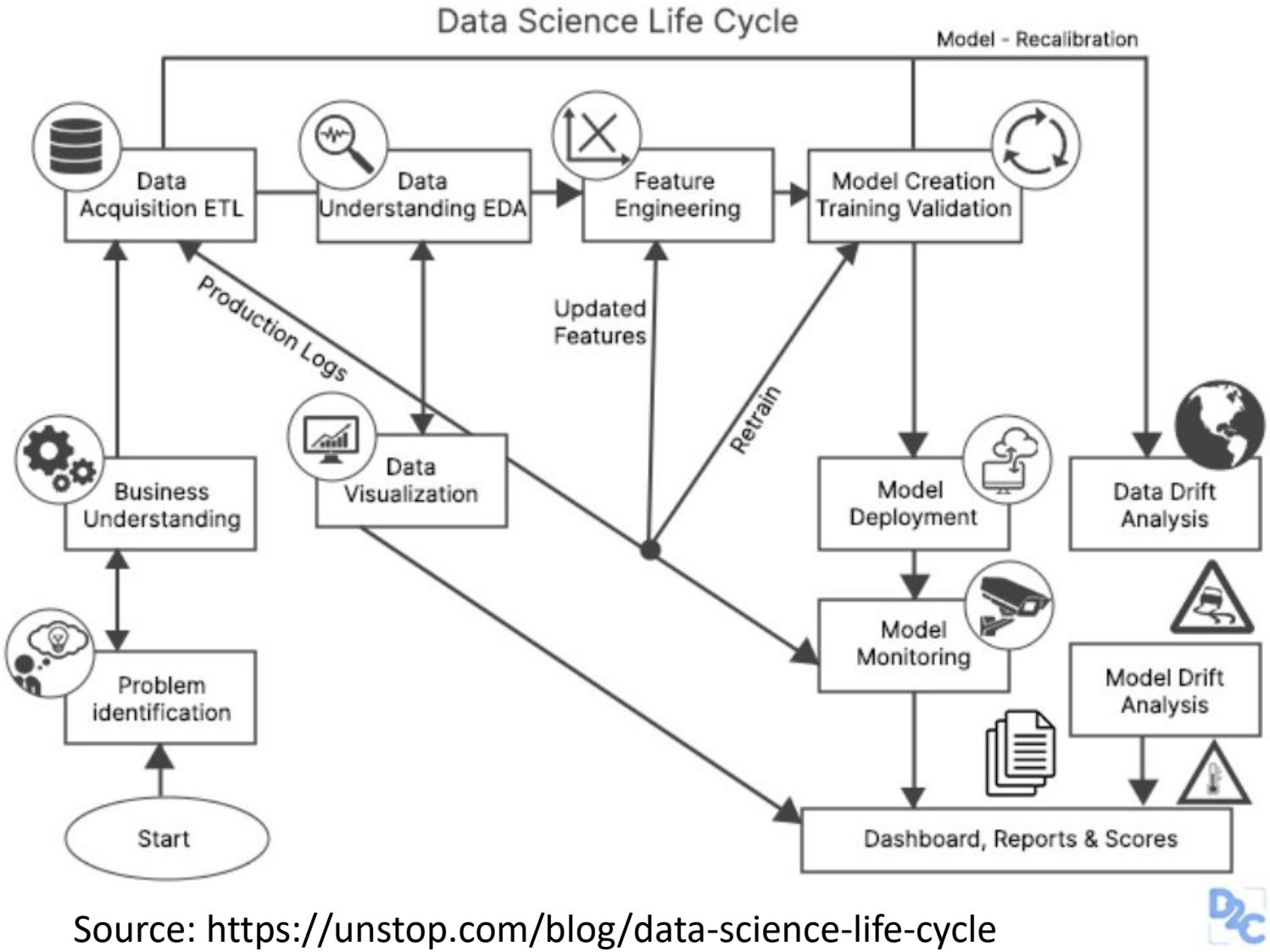
(2) Transform:

- **Definition**: Once the data is extracted, it goes through a transformation process where it is cleaned, structured, and enriched.
- **Purpose**: Data transformation involves tasks like data cleaning, normalization, aggregation, and feature engineering to prepare the data for analysis.

(3) Load:

- **Definition**: The final step is loading the transformed data into a target database, data warehouse, or data mart for storage and analysis.
- **Purpose**: The loaded data is now in a format that is optimized for querying, reporting, and analysis.

Data Science Life Cycle

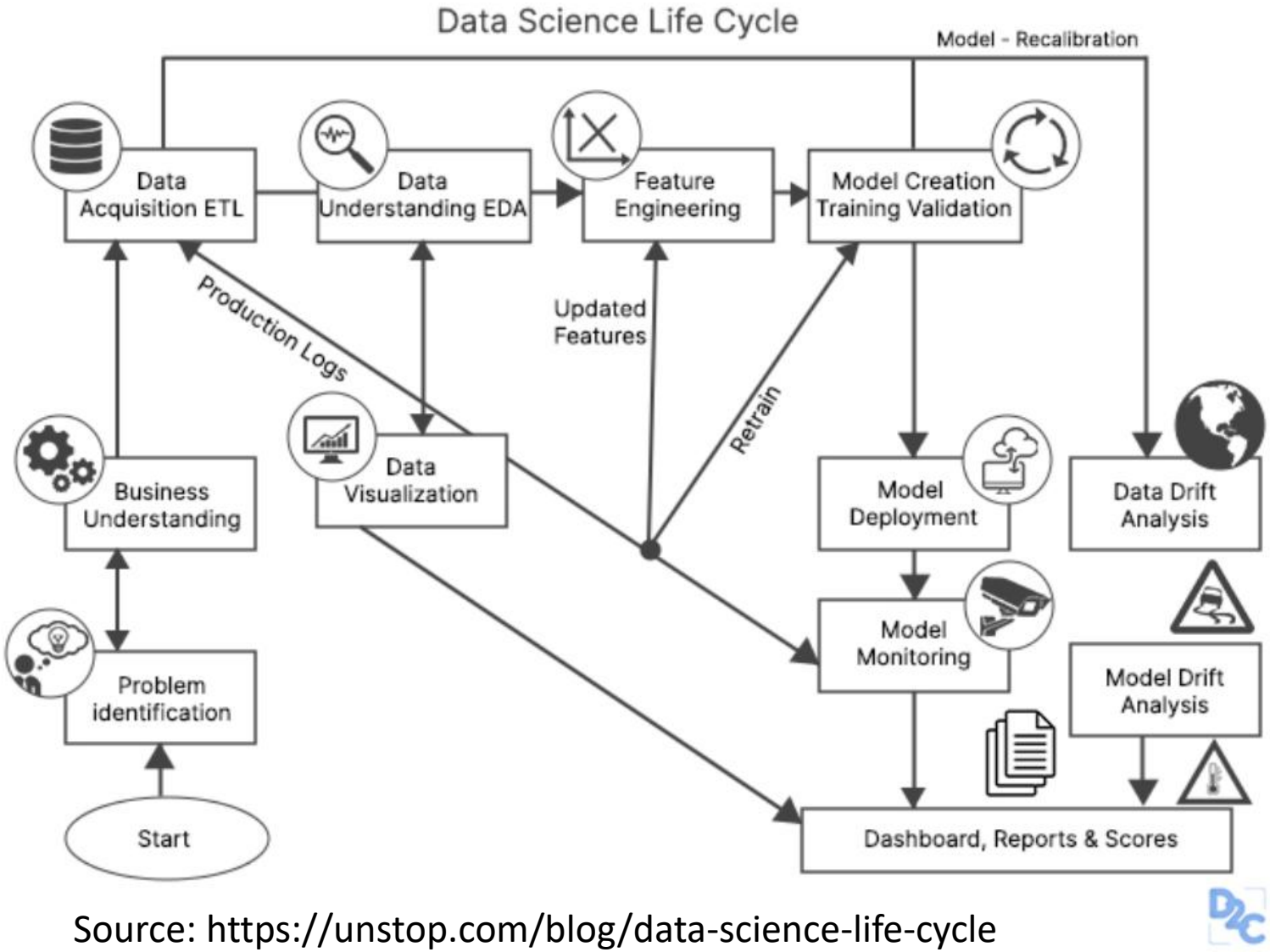


Source: https://unstop.com/blog/data-science-life-cycle

4. Data Exploration-Exploratory Data Analysis (EDA): EDA involves analyzing and visualizing the data to gain insights into patterns, relationships, and anomalies. This step helps in understanding the data better and identifying potential features for modeling.

Ref: https://en.wikipedia.org/wiki/Exploratory_data_analysis

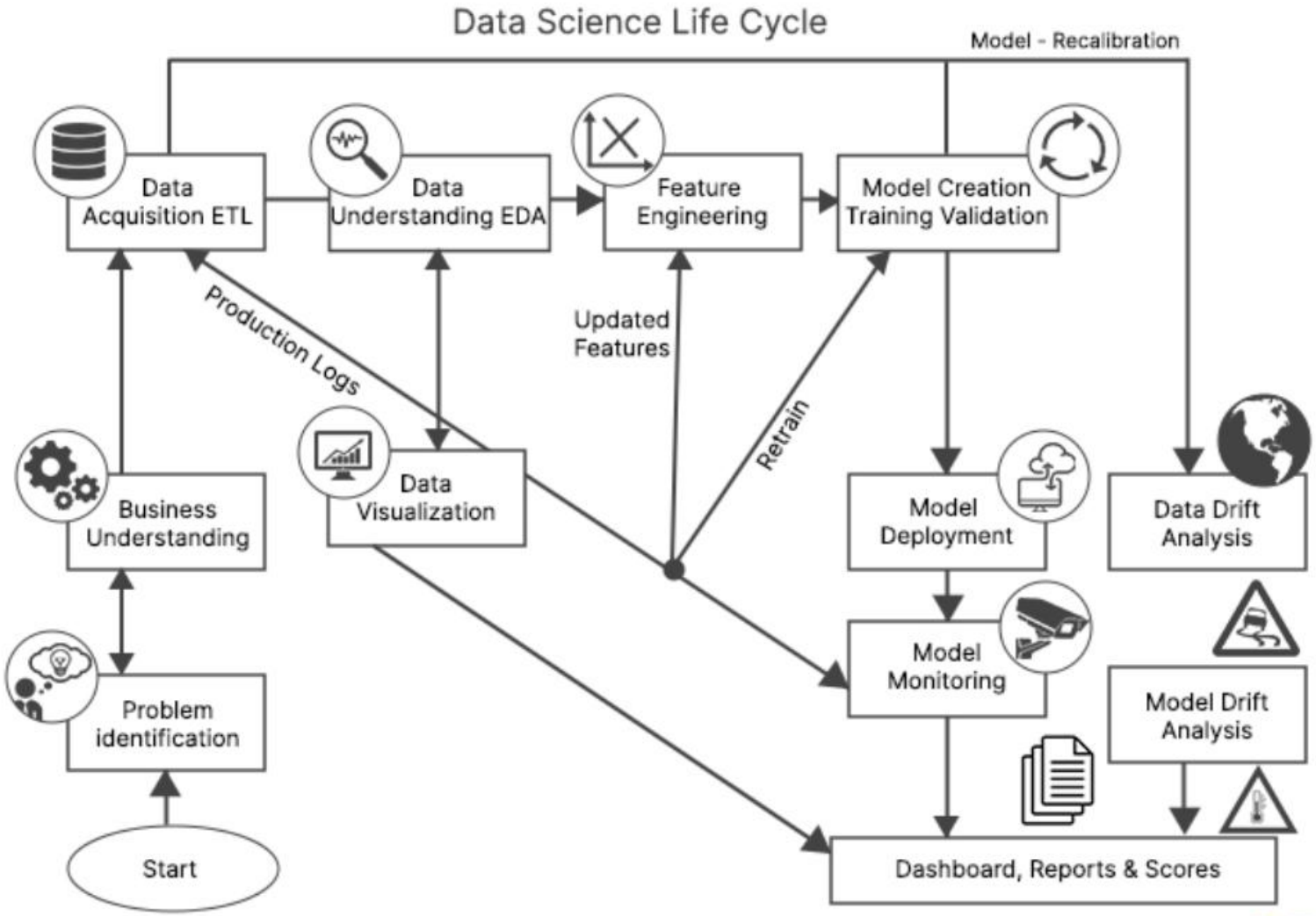
Data Science Life Cycle



Source: https://unstop.com/blog/data-science-life-cycle

5. Feature Engineering: Feature engineering involves creating new features or transforming existing features to improve the performance of machine learning models. It is a critical stage in the pipeline that can significantly impact model accuracy.

Data Science Life Cycle



Source: https://unstop.com/blog/data-science-life-cycle

5. Feature Engineering

5.1 Feature Creation:

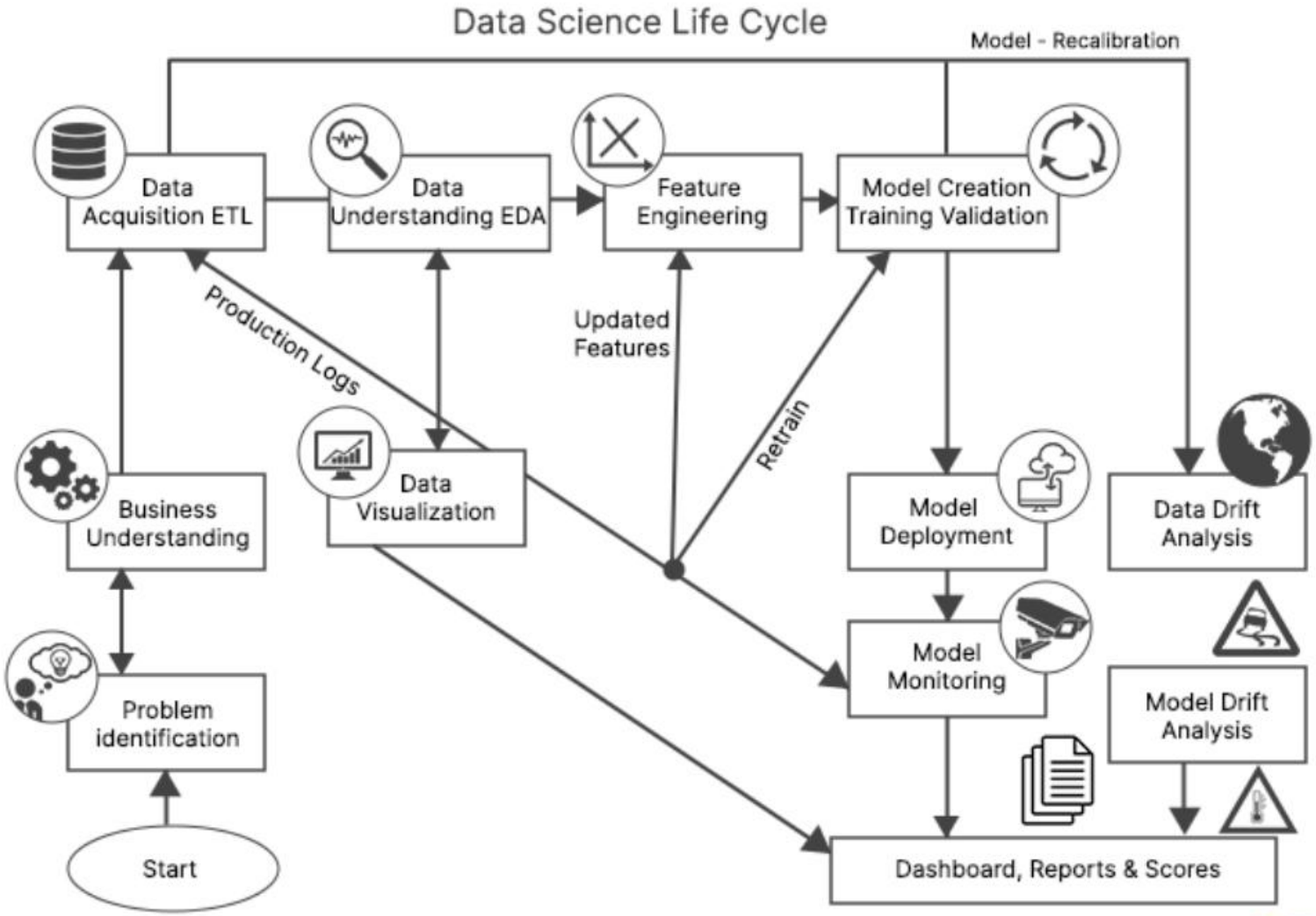
•**Definition**: Feature creation involves generating new features from the existing dataset to provide the model with additional relevant information.

•**Purpose**: Introducing new features can provide additional information to the model, capturing complex patterns and relationships within the data.

•**Methods**: This process includes techniques like polynomial features, interaction terms, and domain-specific feature engineering based on expert knowledge.

•**Impact**: Well-crafted features can significantly enhance model accuracy and predictive power by providing relevant information for better model understanding.

Data Science Life Cycle



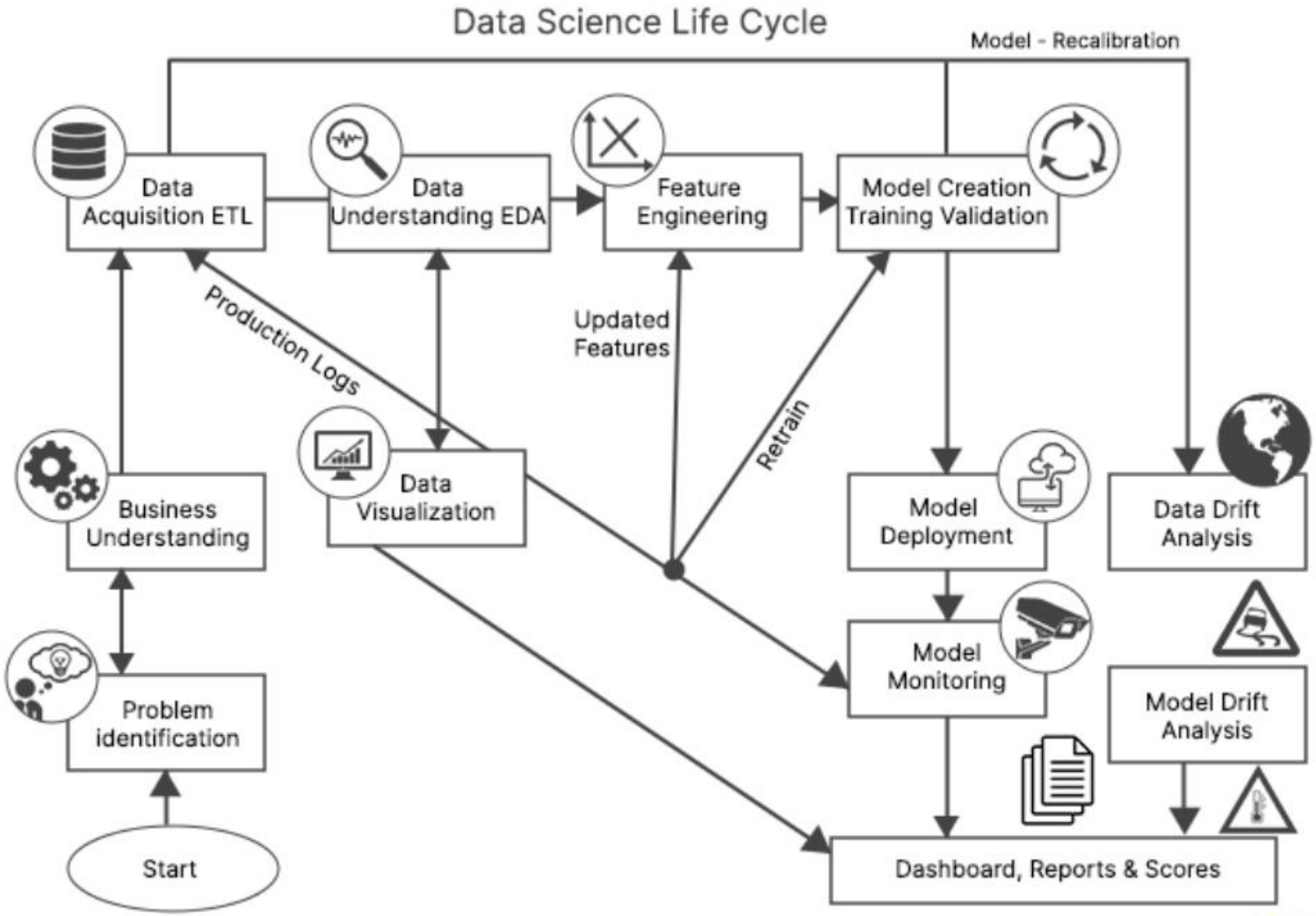
Source: https://unstop.com/blog/data-science-life-cycle

5. Feature Engineering

5.2 Feature Selection:

- **Definition**: Feature Selection is the process of choosing the most relevant features for model training while discarding irrelevant or redundant ones.
- **Purpose**: Selecting the right features helps in reducing overfitting, improving model interpretability, and reducing computational complexity.
- **Methods**: Feature selection can be done using filter methods (based on statistical measures), wrapper methods (using models for selection), or embedded methods (incorporating selection within model training).
- **Impact**: Proper feature selection can lead to simpler and more efficient models that focus on the most important aspects of the data, improving model performance and generalization.

Data Science Life Cycle



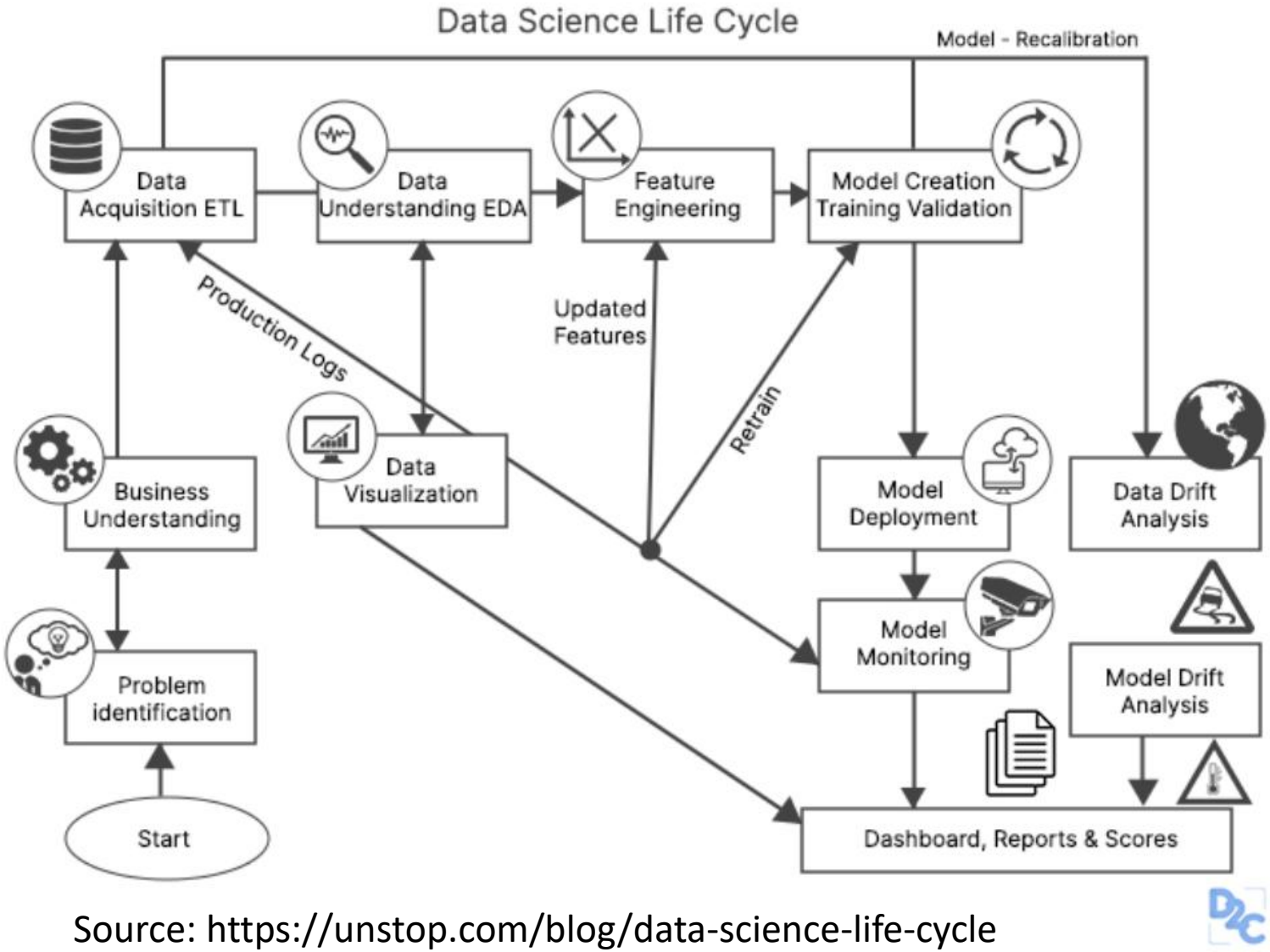
Source: https://unstop.com/blog/data-science-life-cycle

5. Feature Engineering

5.3 Feature Transformation:

- **Definition**: Feature transformation involves modifying the existing features to make them more suitable for the machine learning model.
- **Purpose**: Feature transformation optimizes model compatibility, reduces dimensionality, enhances interpretability, and eliminates noise in data, improving model performance by focusing on relevant patterns.
- **Methods**: Techniques like Principal Component Analysis (PCA), t-distributed Stochastic Neighbor Embedding (t-SNE), and Singular Value Decomposition (SVD), Scaling and Normalization.
- **Impact**: Feature transformation streamlines data handling by reducing dimensionality and noise, boosts interpretability for easier analysis, and enhances model performance through improved accuracy, generalization, and efficiency.

Data Science Life Cycle



Source: https://unstop.com/blog/data-science-life-cycle

6. **Model Creation**: Select appropriate machine learning algorithms, train models, and evaluate their performance.

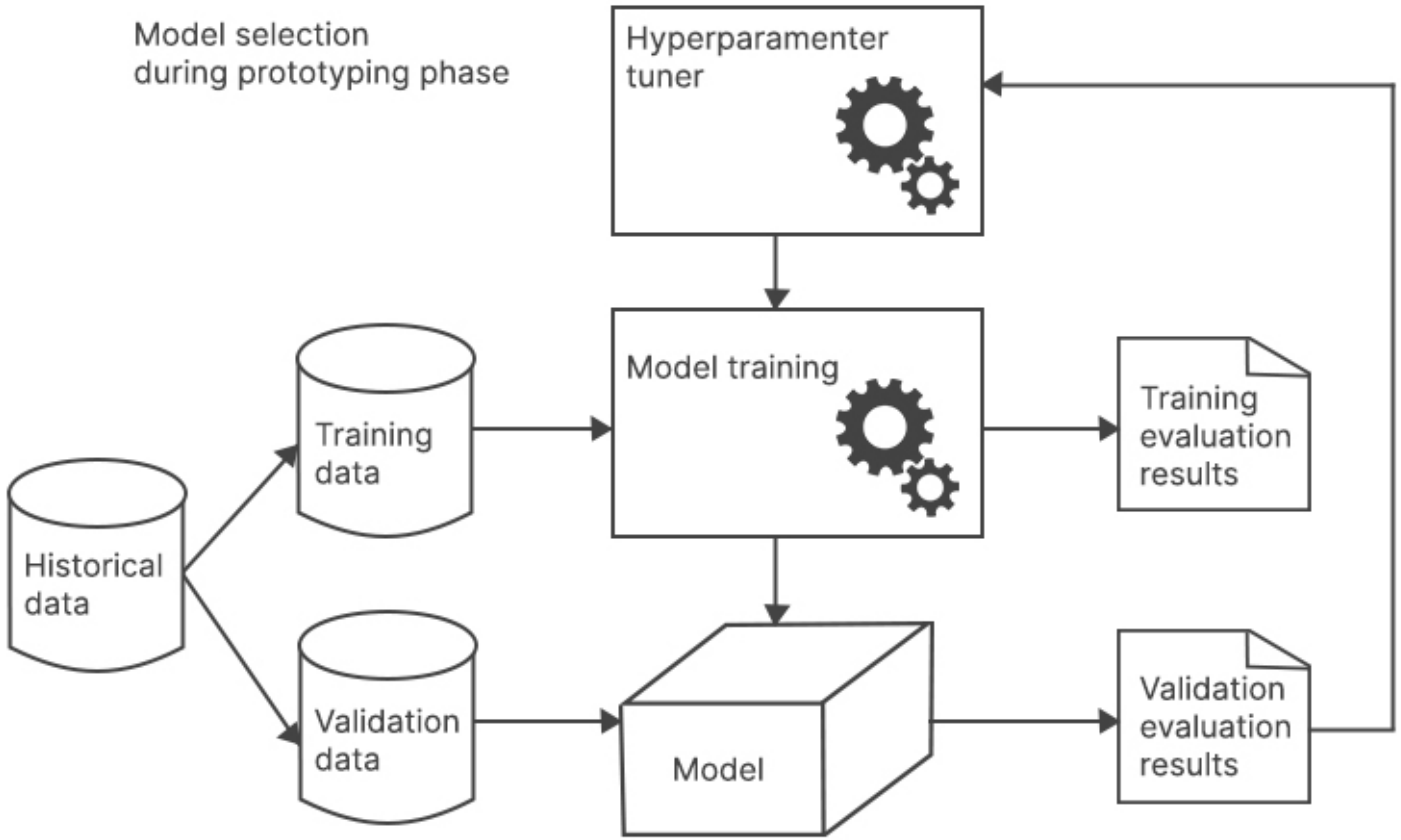
7. **Model Evaluation**: Test models on unseen data, tune hyperparameters, and assess model performance.

8. **Model Deployment**: Implement the model into production for real-world use, monitoring its performance over time.

9. **Communication and Visualization**: Present insights and results effectively using visualizations and reports to communicate findings to stakeholders.

10. **Iterative Process**: The Data Science Pipeline is often iterative, with feedback from evaluation and deployment stages informing further iterations to improve models and insights.

Data Science Life Cycle

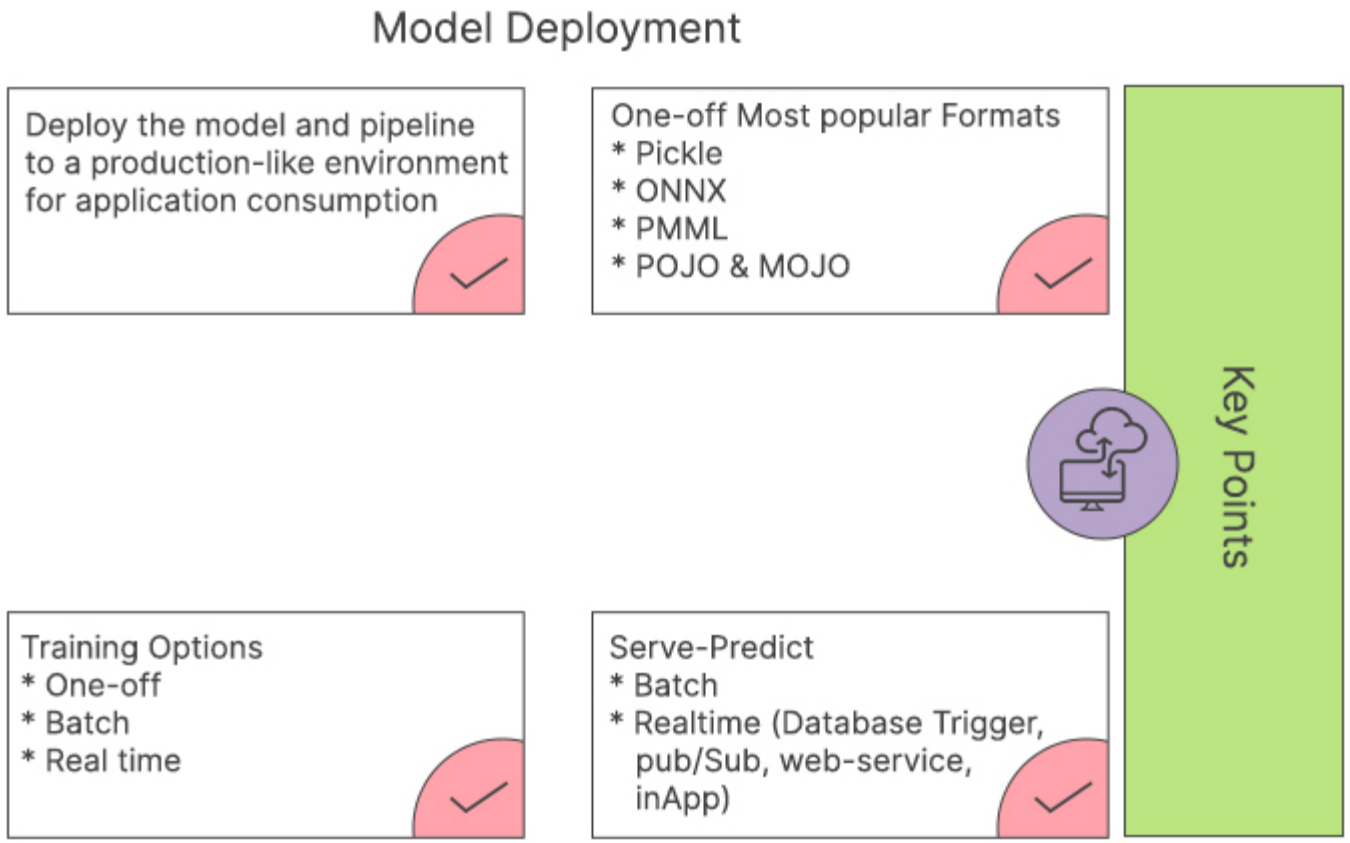


6. Model Creation: Select appropriate machine learning algorithms, train models, and evaluate their performance.

7. Model Evaluation: Test models on unseen data, tune hyperparameters, and assess model performance.

Source: https://unstop.com/blog/data-science-life-cycle

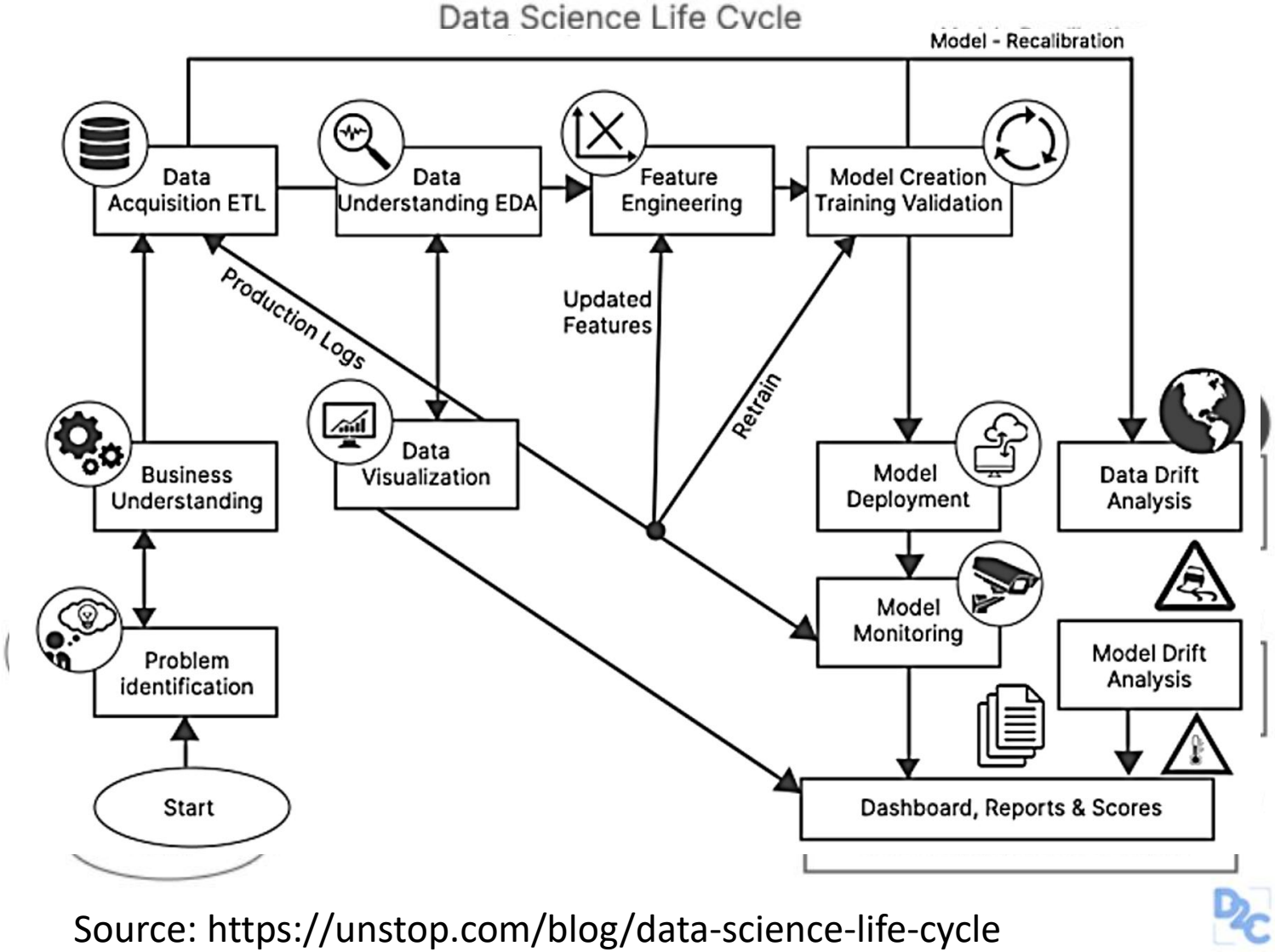
Data Science Life Cycle



8. Model Deployment: Implement the model into production for real-world use, monitoring its performance over time.

Source: https://unstop.com/blog/data-science-life-cycle

Data Science Life Cycle



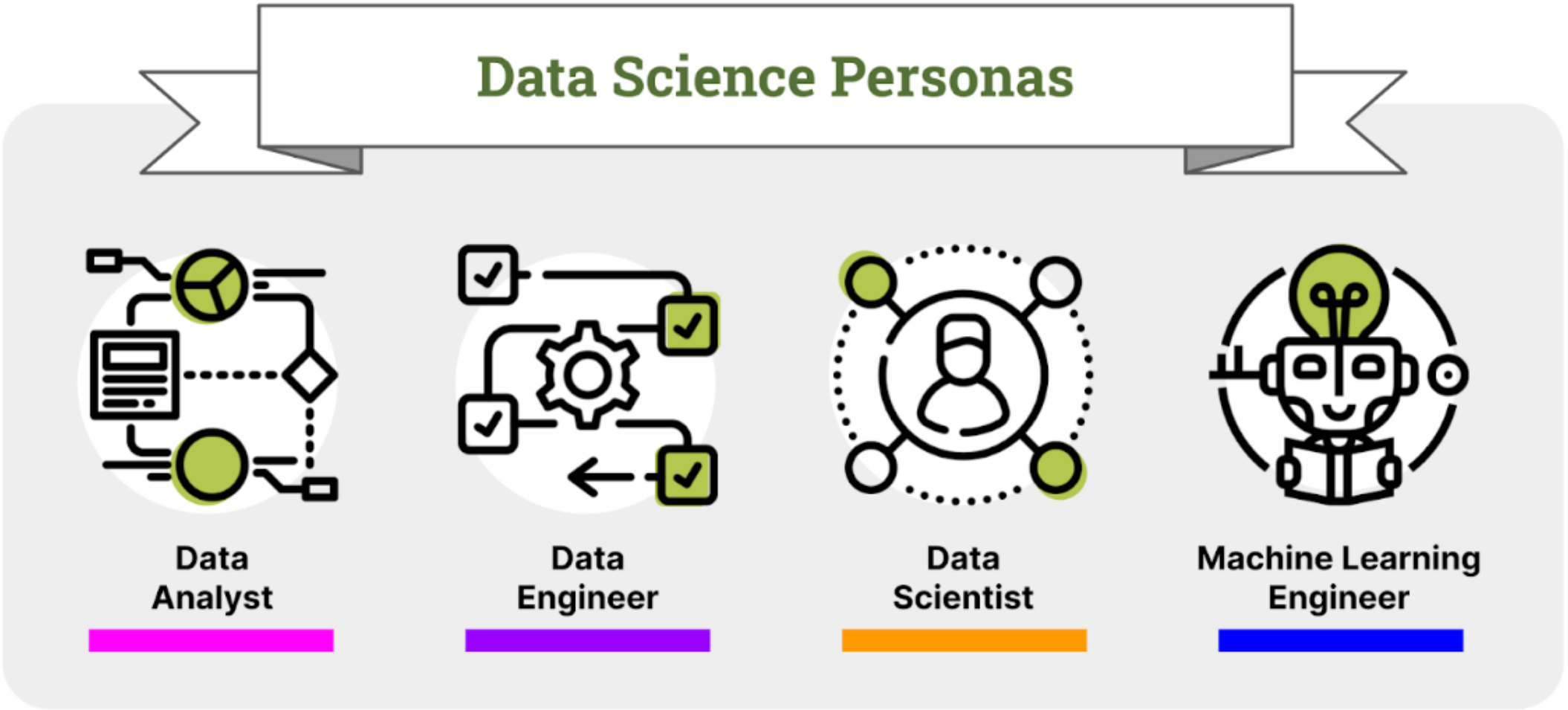
Source: https://unstop.com/blog/data-science-life-cycle

- **Data Drift Analysis:** We compare the incoming data distribution from the actual world with the one that has been scored, and if there is a difference, we must choose a threshold.
- **Model Drift Analysis:** we compare the difference between the score of the real-world model distribution and the scored model distribution, and this information is fed into a system that reports and outputs to the business user on a regular basis.

Data drift can be caused by a variety of factors, including:

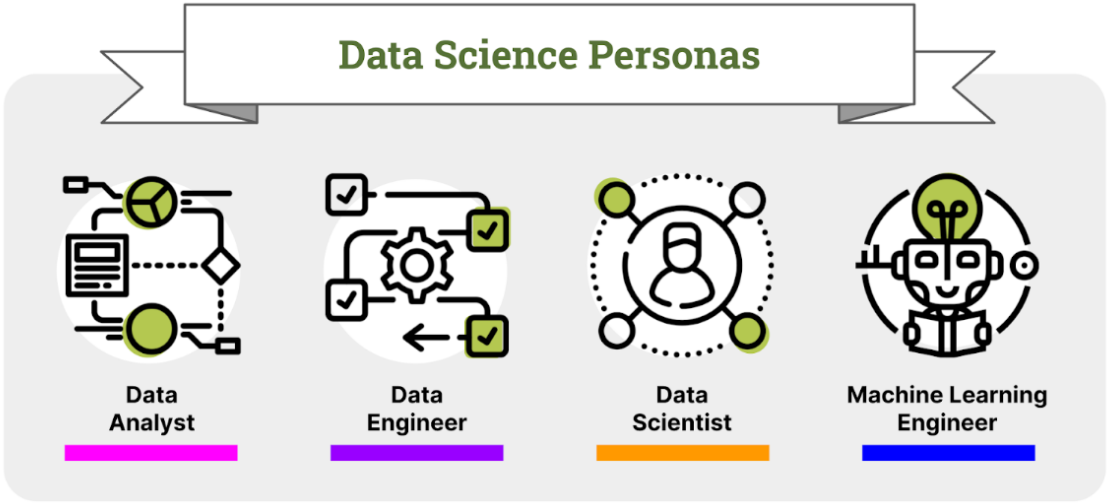
- Alters in upstream processes, such as the replacement of a sensor that changes the units of measurement, or a damaged sensor that always reads 0.
- A natural variation in the data, such as the change in mean temperature with the seasons.
- Changes in the nature of the relationship between characteristics.
- What can you do to help?
- Erroneous labeling should be removed.
- Model retraining/calibration
- Early detection and correction of upstream data issues

Data Science Life Cycle



Source: https://arangodb.com/2022/04/whos-who-in-data-science/

Data Science Life Cycle



Source: https://arangodb.com/2022/04/whos-who-in-data-science/

- Data Scientist: Analyzes data, builds models, and communicates insights.
- Data Engineer: Ensures data availability and quality, builds data pipelines.
- Data Science Architect: Designs and maintains data science infrastructure.
- Data Science Developer: Develops and deploys data science applications.
- Data Science Manager: Oversees the data science team, ensuring project alignment and delivery.

Source: https://www.genspark.ai/spark/data-science-life-cycle-diagram/

Data Science Life Cycle

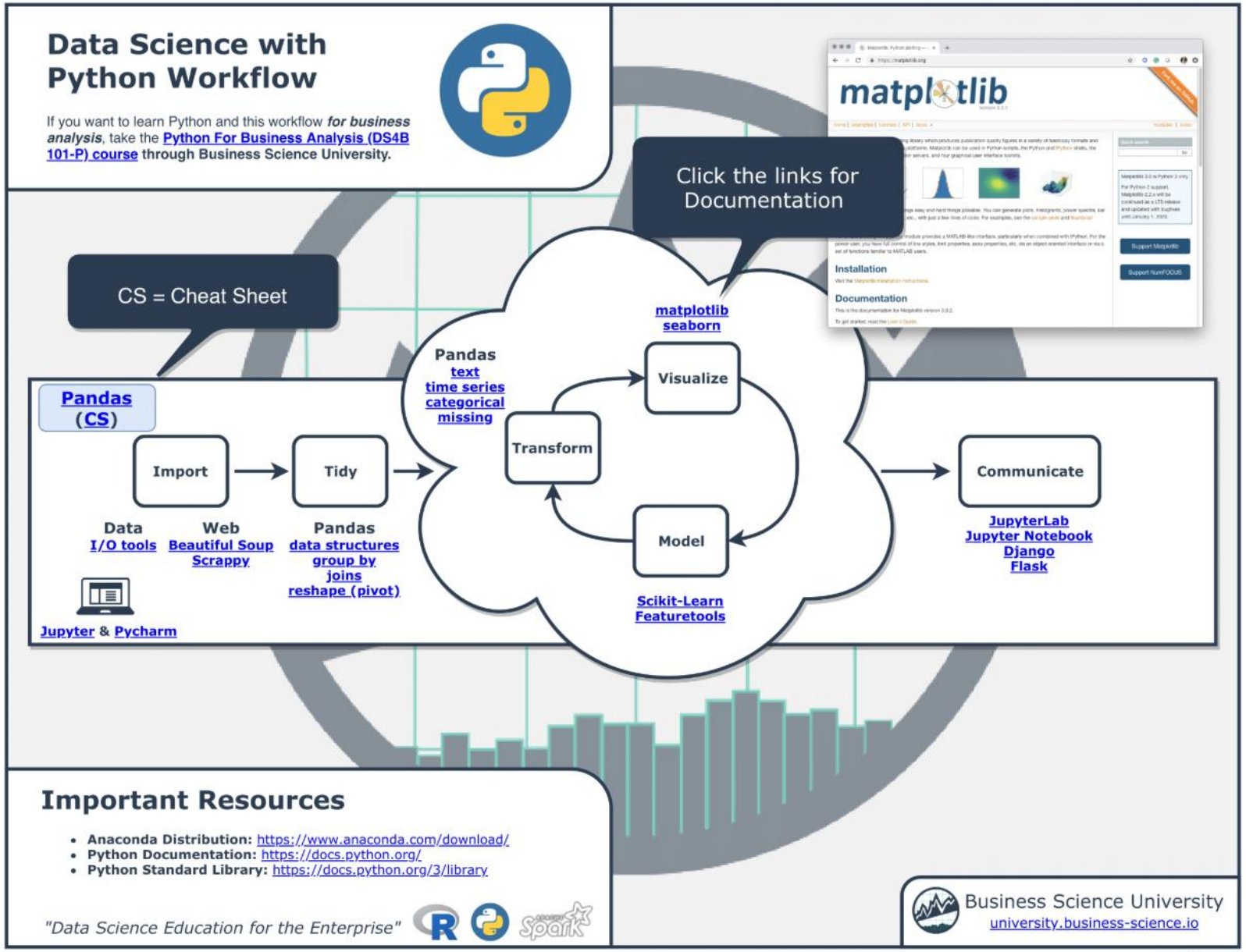
	Business Understanding	Data Understanding	Data Preparation	Modeling	Evaluation	Deployment
Data Scientist	Mid	High	Mid	Top	High	Lower
Data Engineer	Mid	High	Top	Lower	Lower	Mid
Data Analyst	Mid	Top	Mid	Mid	Lower	Lower
ML Engineer	Mid	Lower	Mid	Mid	High	Top
Product Owner	Top	Mid	Lower	Lower	Top	Lower
Project Manager	High	Lower	Lower	Lower	Mid	Mid

- Data Scientist: Analyzes data, builds models, and communicates insights.
- Data Engineer: Ensures data availability and quality, builds data pipelines.
- Data Science Architect: Designs and maintains data science infrastructure.
- Data Science Developer: Develops and deploys data science applications.
- Data Science Manager: Oversees the data science team, ensuring project alignment and delivery.

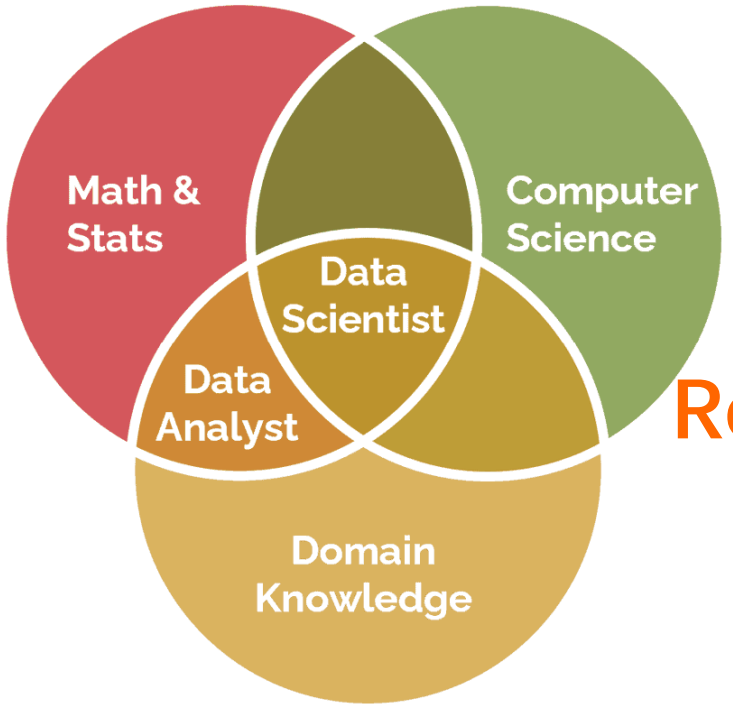
Source: https://www.genspark.ai/spark/data-science-life-cycle-diagram/



Data Science with Python Workflow

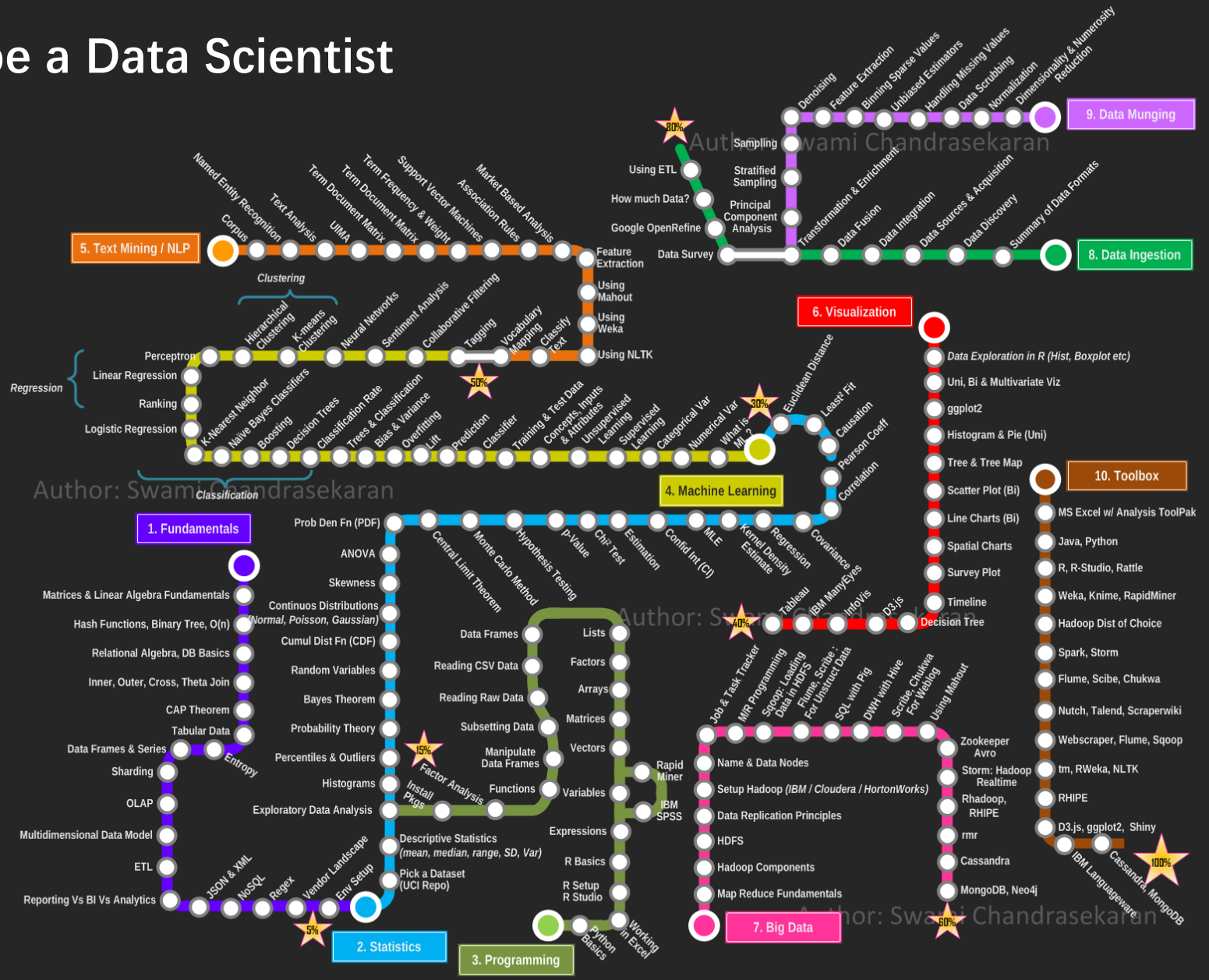


Source: https://github.com/FavioVazquez/ds-cheatsheets?tab=readme-ov-file



Roadmap to be a Data Scientist

Roadmap to be a Data Scientist

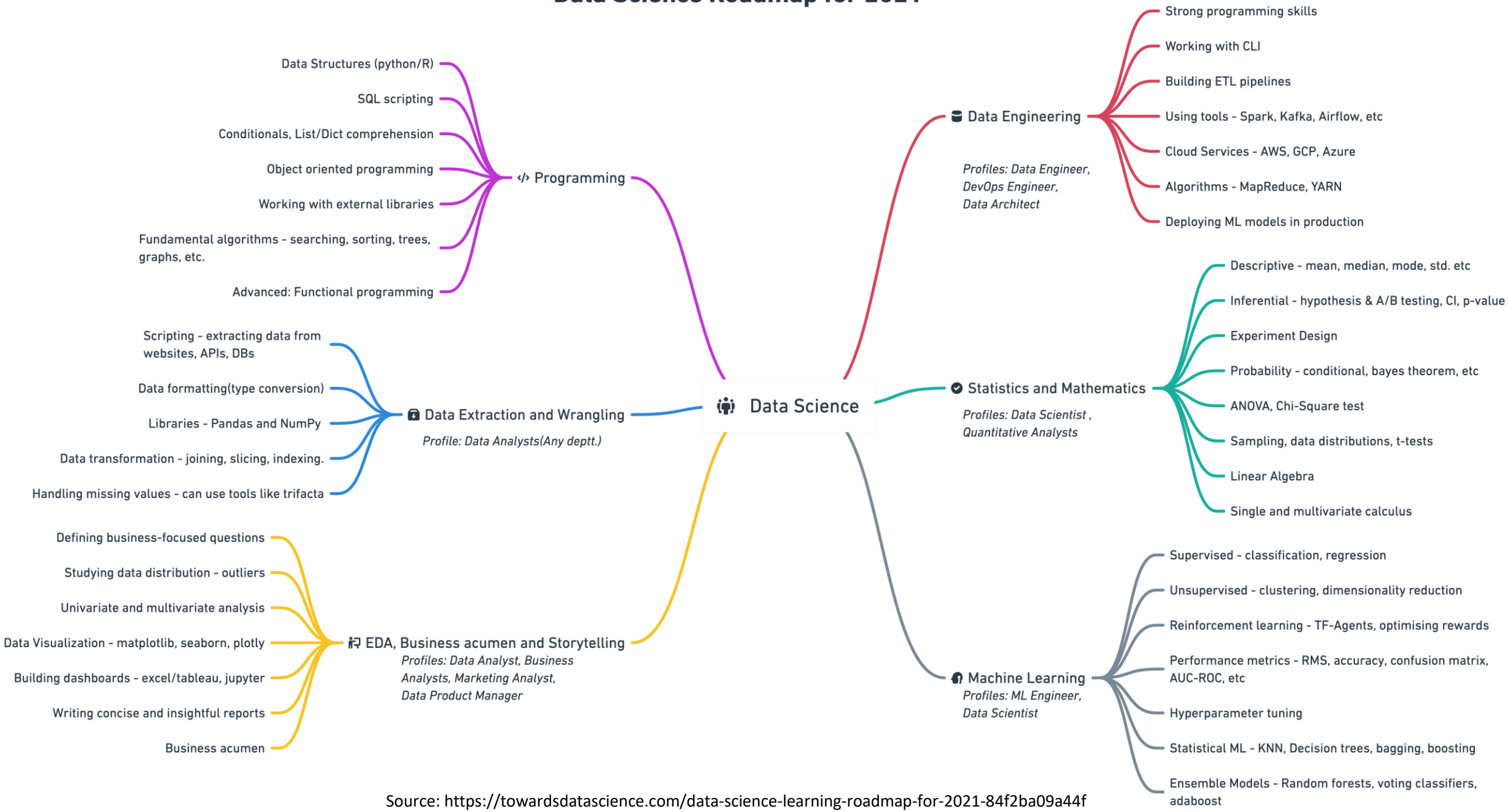


https://github.com/MrMimic/data-scientist-roadmap?tab=readme-ov-file

澳門理工大學
Universidade Politécnica de Macau
Macao Polytechnic University



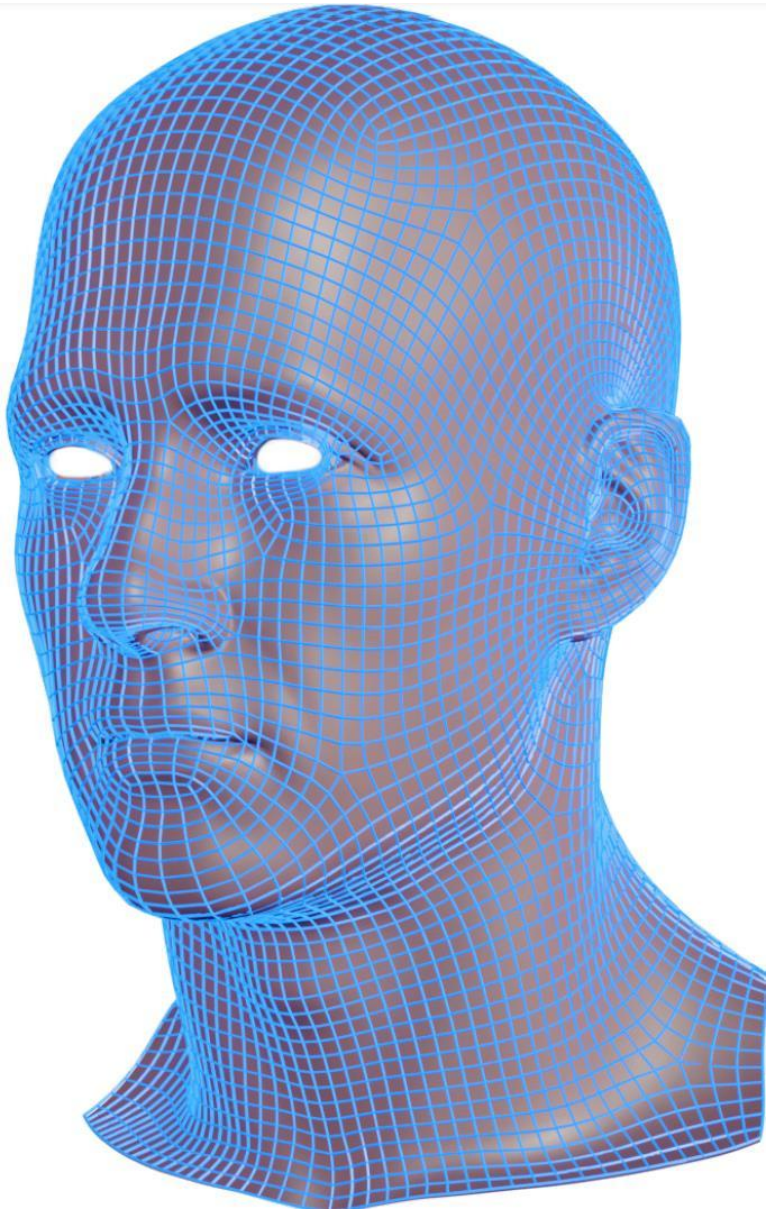
Data Science Roadmap for 2021



Source: https://towardsdatascience.com/data-science-learning-roadmap-for-2021-84f2ba09a44f

Case Study:

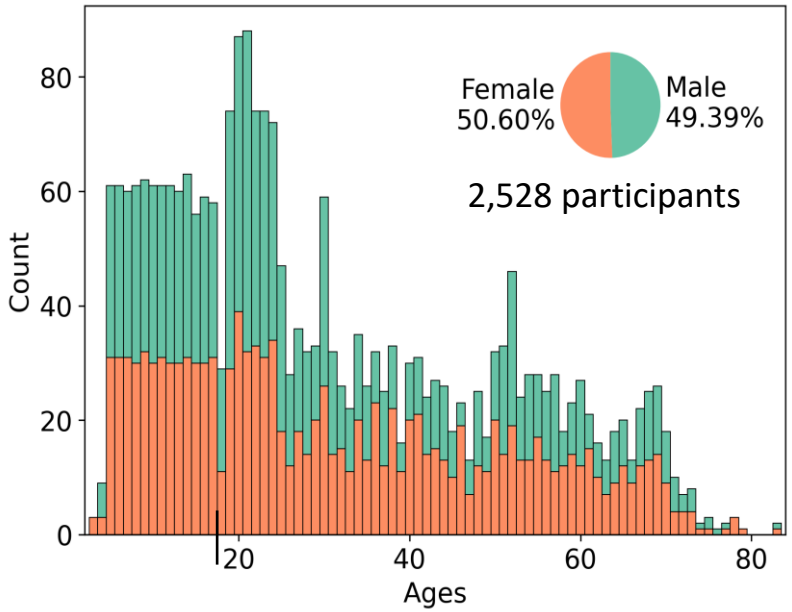
3D Head Shape Characteristics Analysis for Chinese





Case Study: 3D Head Shape Characteristics Analysis for Chinese

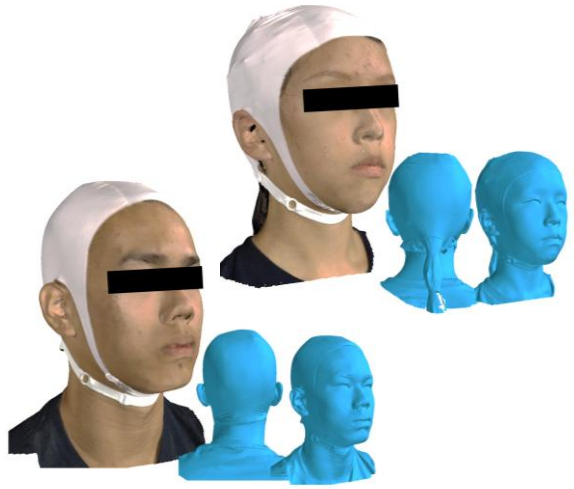
A Large-Scale 3D Head Dataset of Chinese



Participants' Locations



3D Scanning Examples



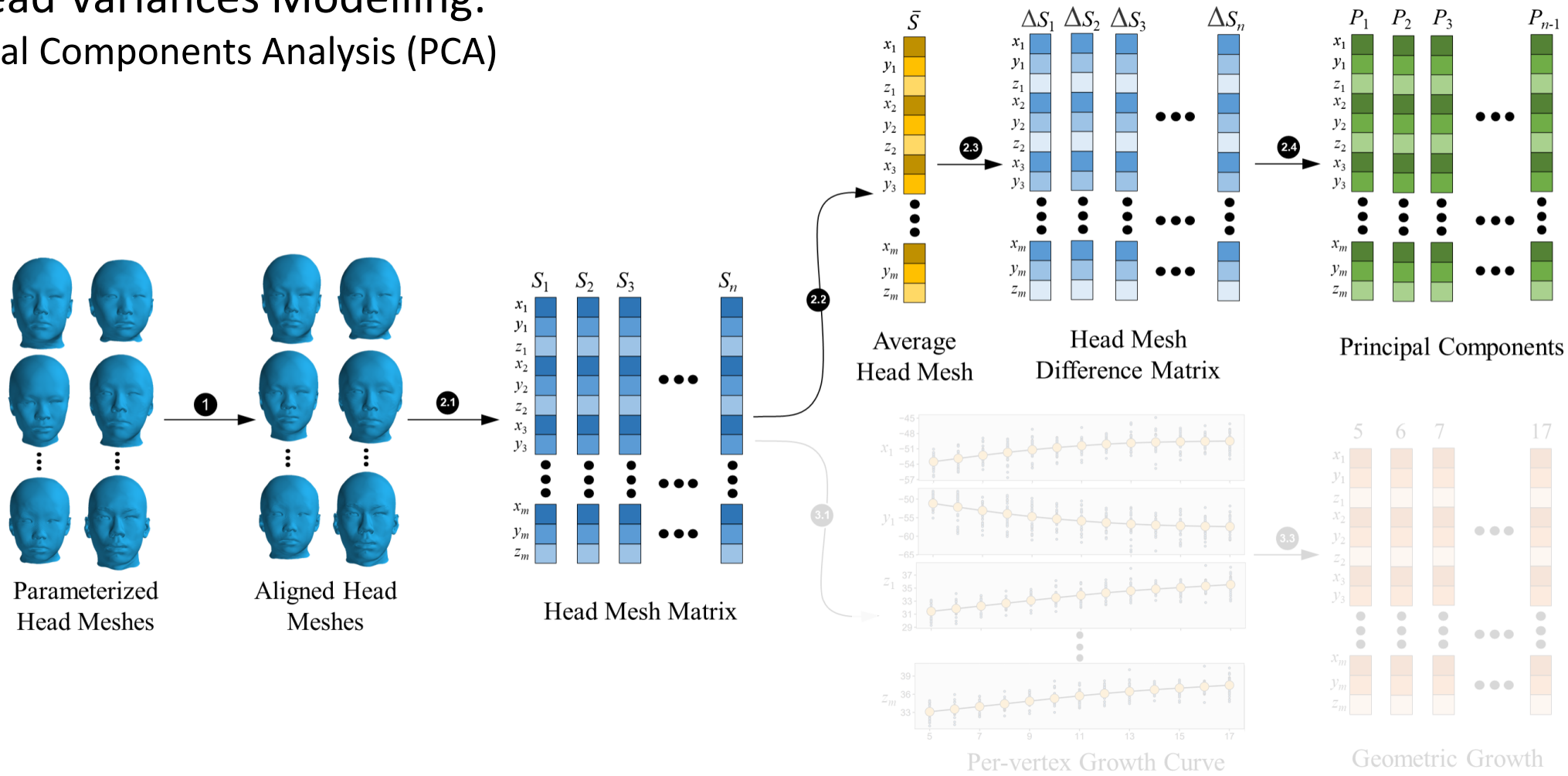
(Zhang, et al., Applied Ergonomics, 2022.)



Case Study: 3D Head Shape Characteristics Analysis for Chinese

3D Head Variances Modelling:

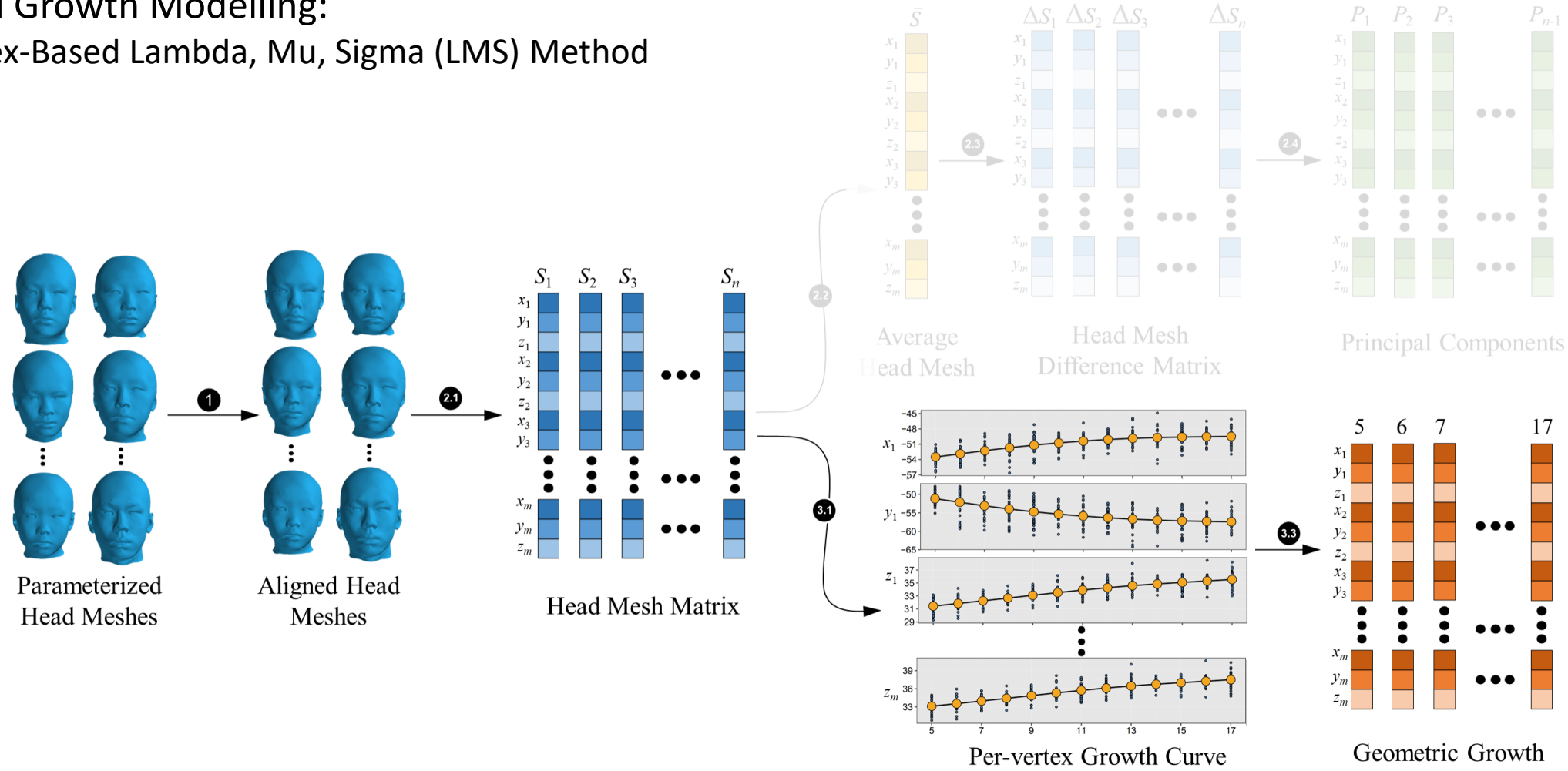
Principal Components Analysis (PCA)



Case Study: 3D Head Shape Characteristics Analysis for Chinese

3D Head Growth Modelling:

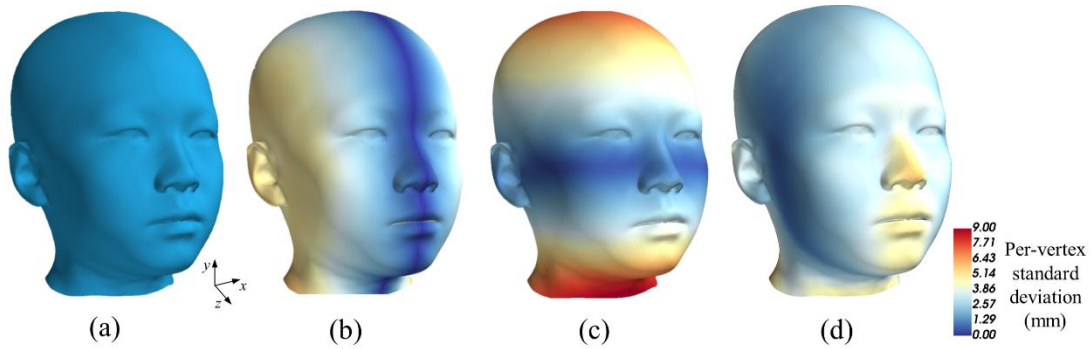
Per-Vertex-Based Lambda, Mu, Sigma (LMS) Method



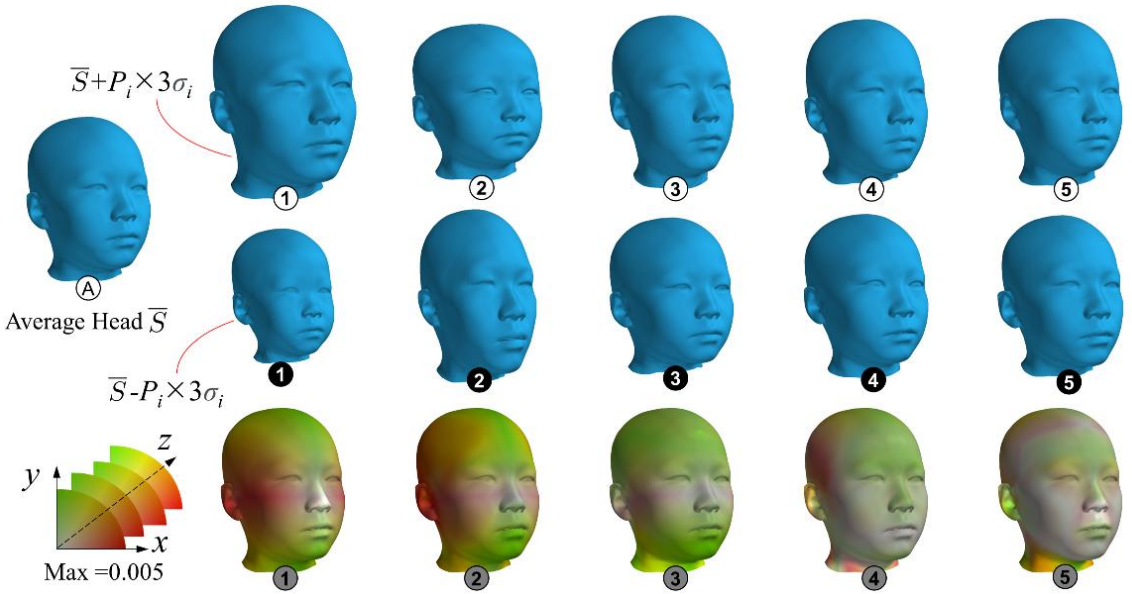
Case Study: 3D Head Shape Characteristics Analysis for Chinese

Results: Children (ages <18)

3D Statistical Shape Model (SSM): Global Model



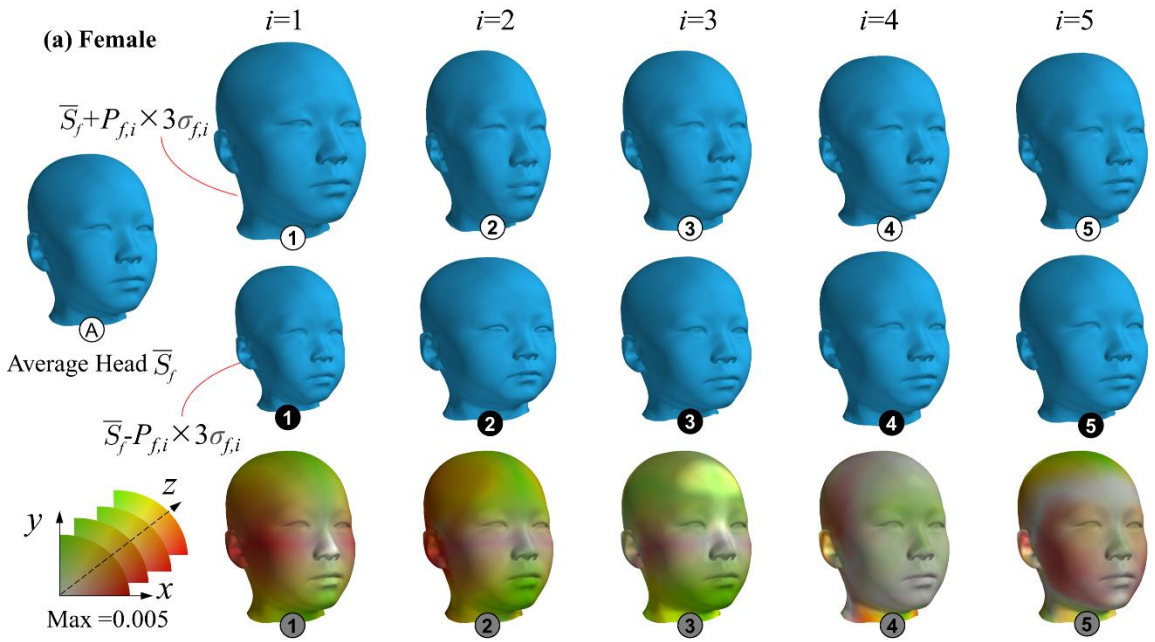
Global basic statistics of head meshes.
(a) average head mesh.
(b) per-vertex standard deviation along the x-axis.
(c) per-vertex standard deviation along the y-axis.
(d) per-vertex standard deviation along the z-axis.



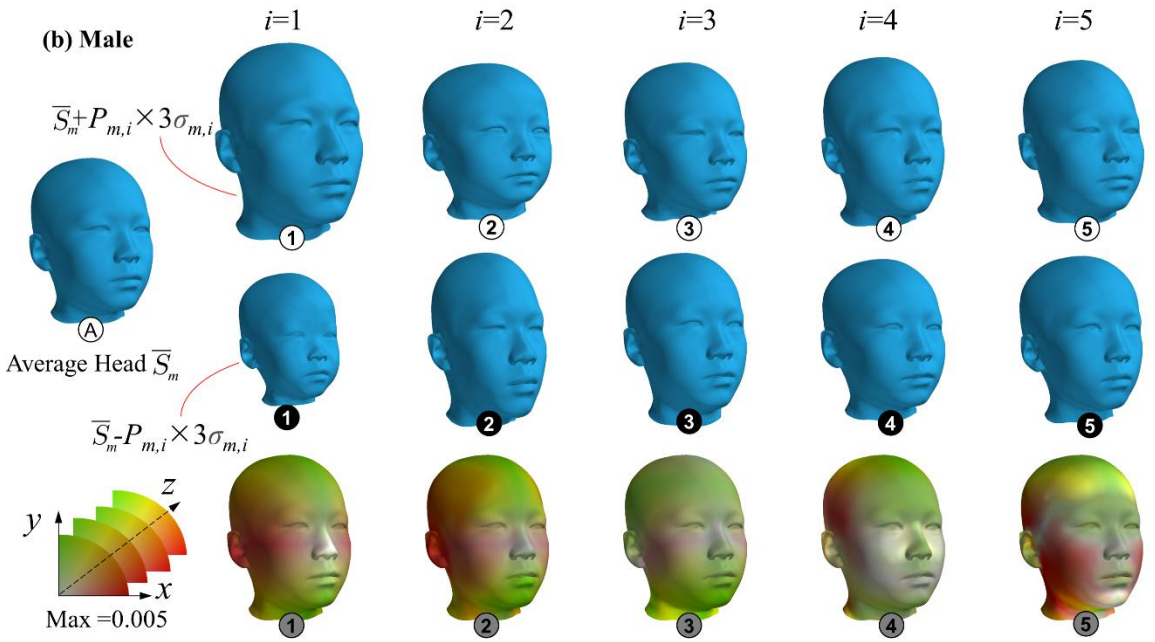
Average head model with the first five principal components

Case Study: 3D Head Shape Characteristics Analysis for Chinese

3D Statistical Shape Model: Subclass Models for Gender

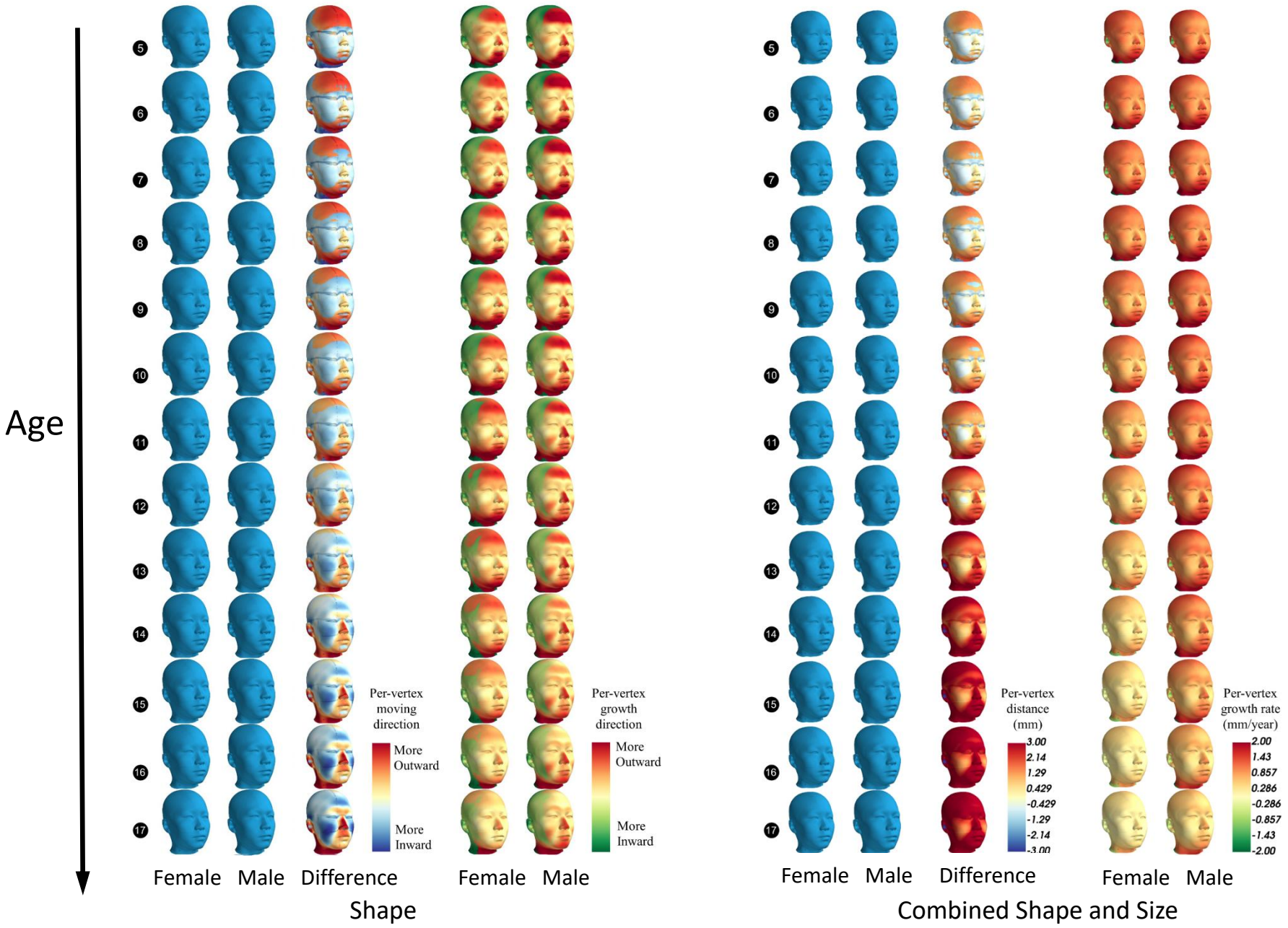


Average female head with the first five principal components



Average male head with the first five principal components

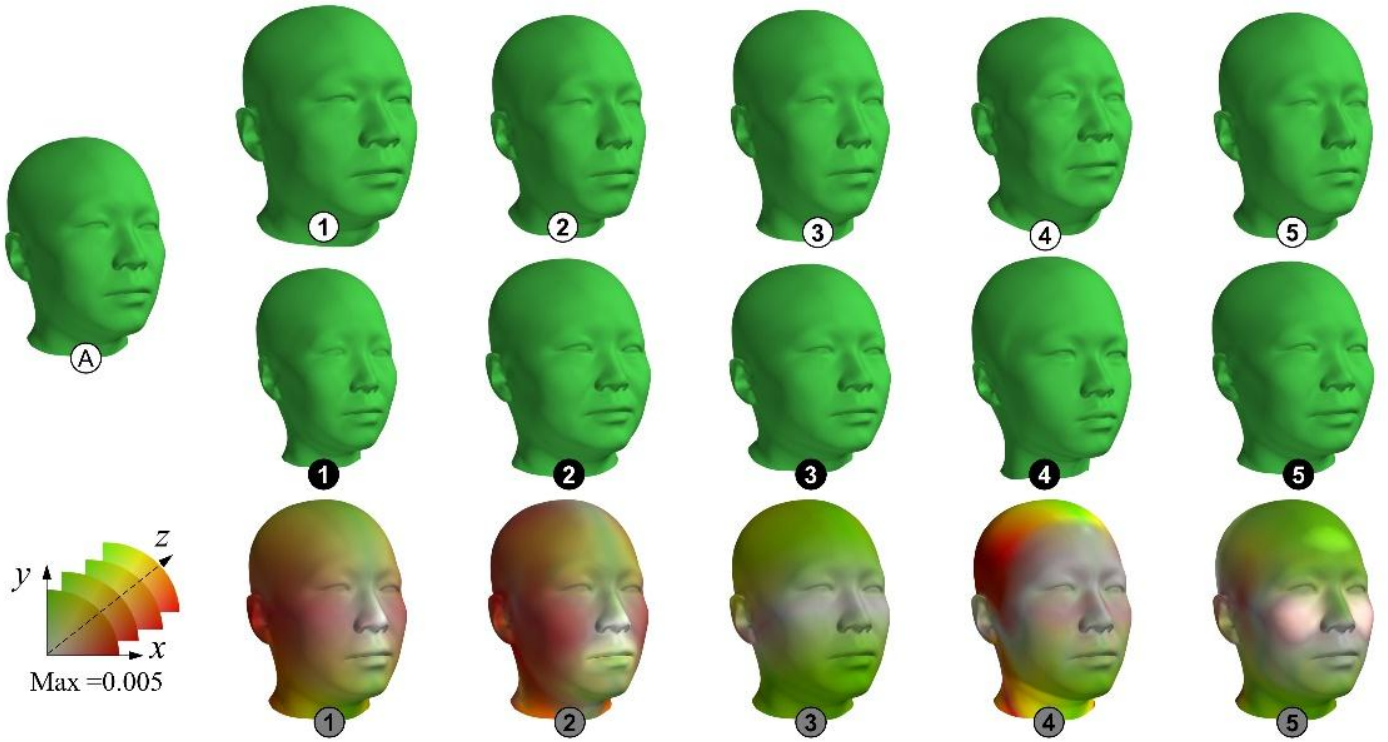
Growth Pattern for Age and Gender



Case Study: 3D Head Shape Characteristics Analysis for Chinese

Results: Adults (ages ≥18)

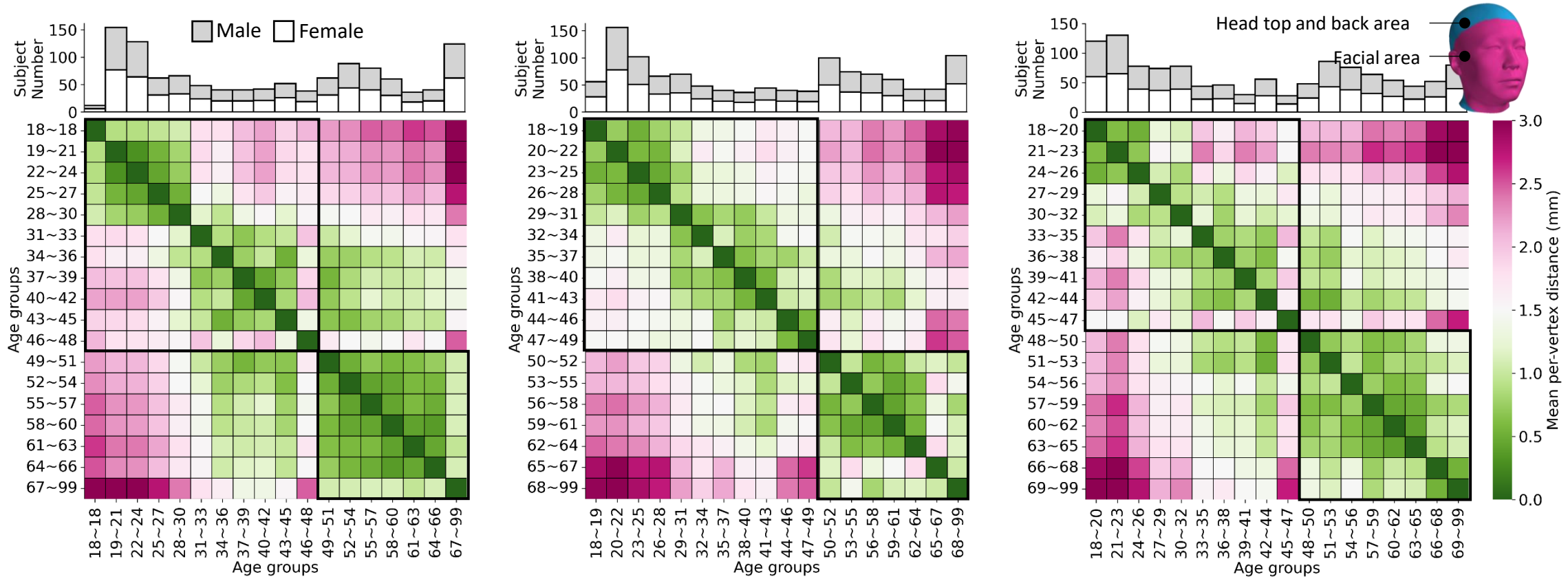
3D Statistical Shape Model (SSM):
Global Model



Average head model with the first five principal components

Case Study: 3D Head Shape Characteristics Analysis for Chinese

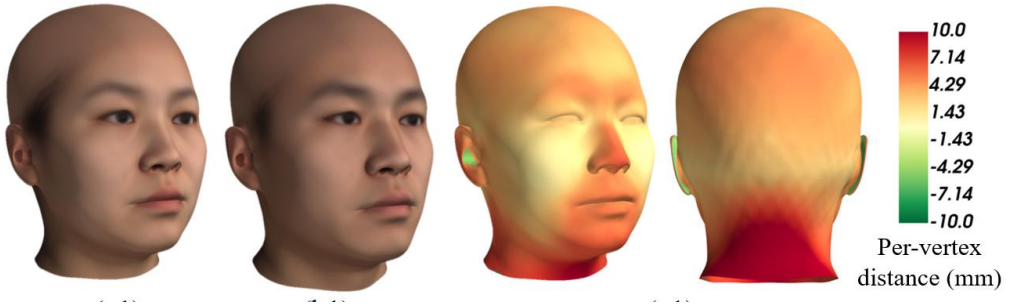
Age Group Classification for adults



Two adults‘ groups: 50- (less than 50) and 50+ (more than or equal to 50)

Case Study: 3D Head Shape Characteristics Analysis for Chinese

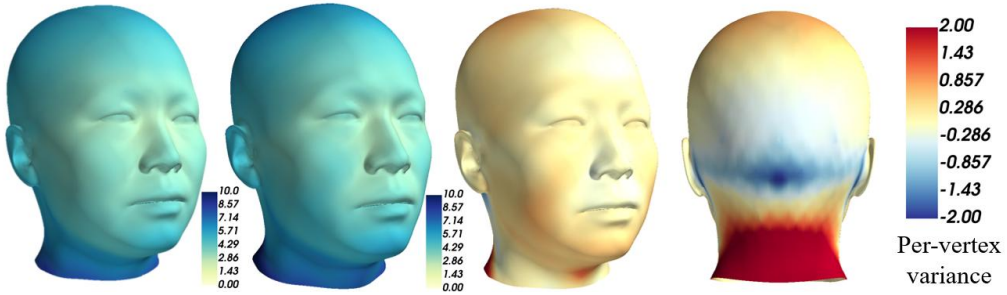
Gender Differences



Average female head mesh

Average male head mesh

Per-vertex distance distribution map

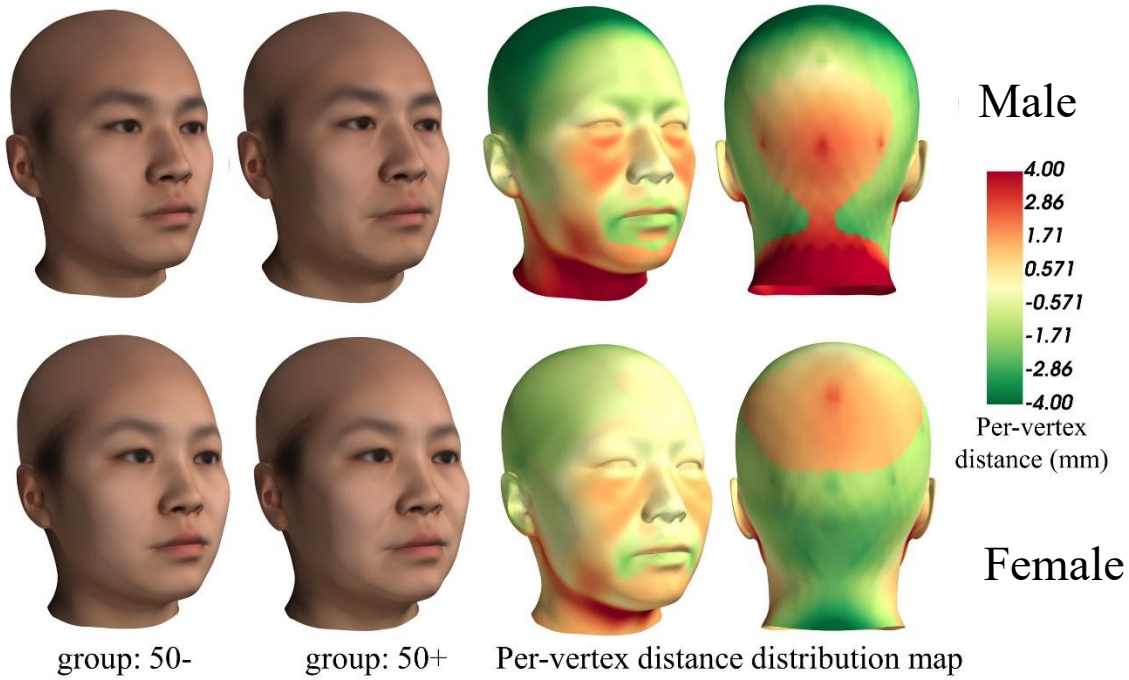


Variance per vertex of female head meshes

Variance per vertex of male head meshes

Per-vertex variance difference distribution map

Combined Gender and Age Differences



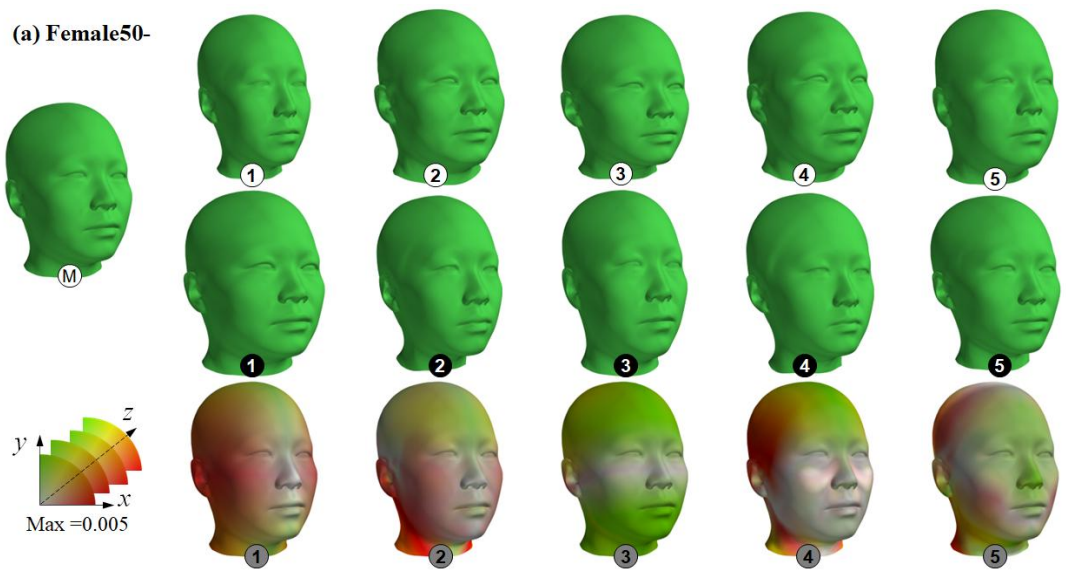
group: 50-

group: 50+

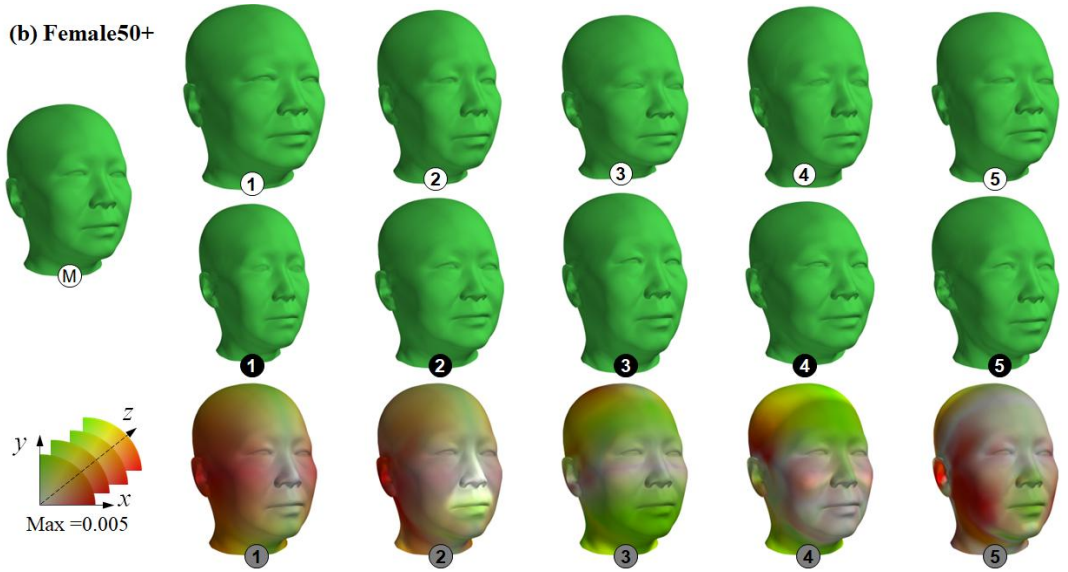
Per-vertex distance distribution map

Case Study: 3D Head Shape Characteristics Analysis for Chinese

3D Statistical Shape Model: Subclass Models for Age



Age <50, Average head model with the first five principal components

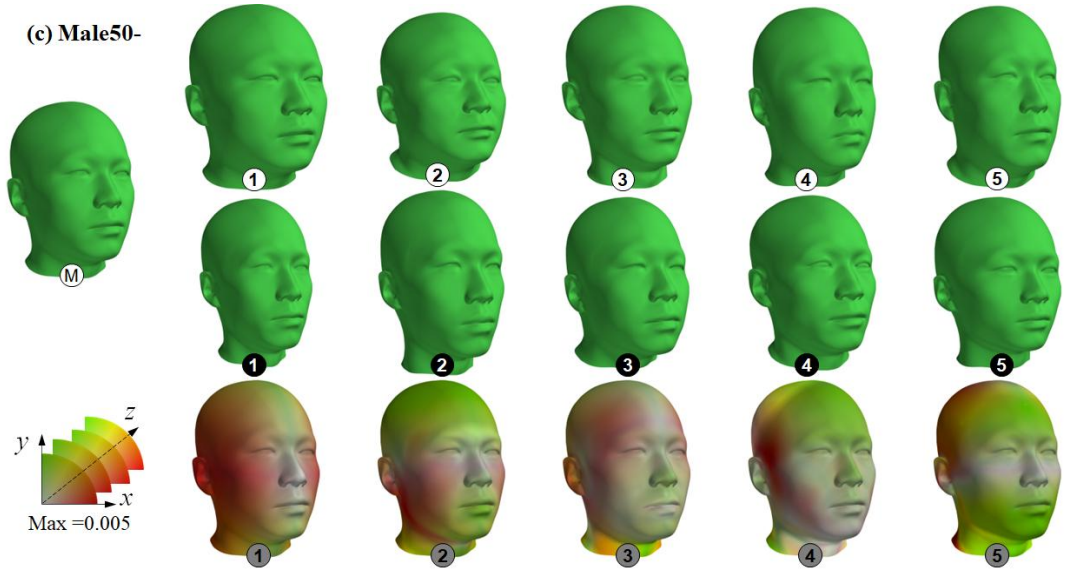


Age >=50, Average head model with the first five principal components

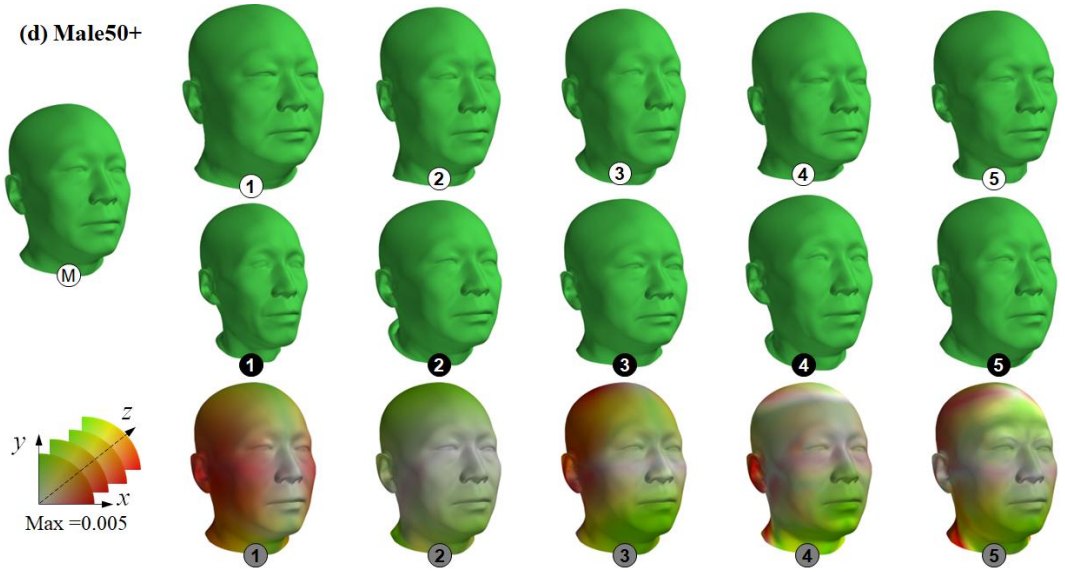
Female

Case Study: 3D Head Shape Characteristics Analysis for Chinese

3D Statistical Shape Model: Subclass Models for Age



Age <50, Average head model with the first five principal components

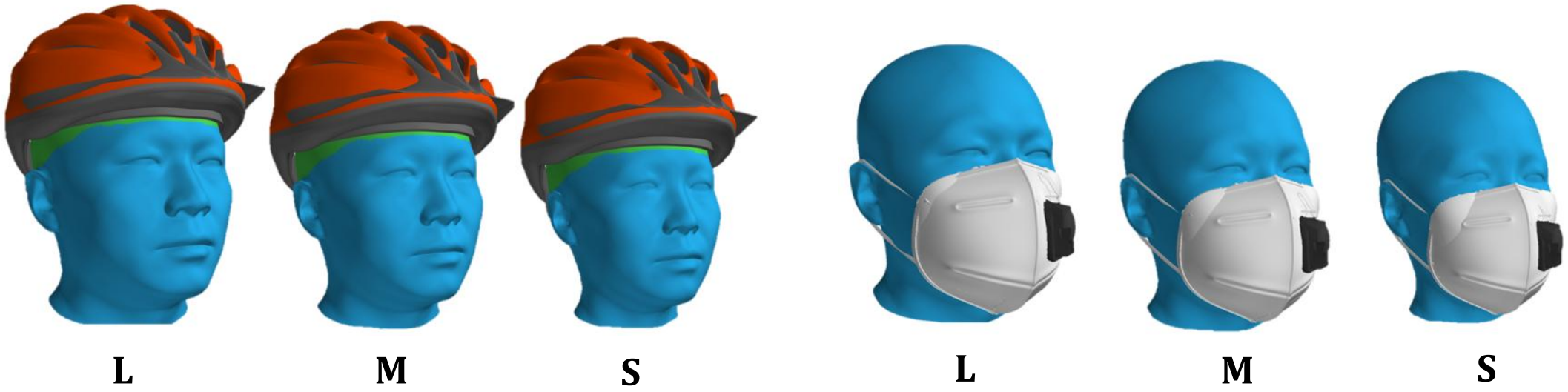


Age >=50, Average head model with the first five principal components

Male

Case Study: 3D Head Shape Characteristics Analysis for Chinese

3D Head and Face forms for Mass Customization (sizing)



Thanks

